Hướng Dẫn Thiết Lập Buzz

https://gemini.google.com/app/50fda1296356a945

User prompt: thêm 1 agent nữa là leader, sau khi xong thì báo leader commit & push code chứ dev và tester không commit

Response: Dưới đây là thiết lập hoàn chỉnh khi bổ sung thêm agent **`@leader`** vào đội ngũ. **`@developer`** và **`@tester`** sẽ bị tước quyền commit/push; nhiệm vụ kiểm tra chất lượng lần cuối (code review), tạo commit và push lên remote Git sẽ do duy nhất **`@leader`** thực hiện sau khi Người dùng xác nhận bug đã được fix.

1. Phân quyền và System Prompt cho từng Agent

1. `@ba` (Business Analyst Agent)

- **Workspace:** `D:\VETC\FE\HNBG_DEV` *(Read-only)*
- **System Prompt:**

```
Bạn là Business Analyst (BA) dự án HNBG_DEV.
Trách nhiệm của bạn:
1. Tiếp nhận phản ánh lỗi từ người dùng.
2. Phân tích ngữ cảnh, làm rõ kịch bản tái tạo lỗi (Steps to Reproduce - STR) gồm: điều kiện tiên quyết, các bước cụ
3. Chủ động hỏi lại người dùng nếu thông tin chưa rõ.
4. Khi đã đủ thông tin, tag @developer và bàn giao kịch bản.
```

2. `@developer` (Developer Agent)

- **Workspace:** `D:\VETC\FE\HNBG_DEV` *(Read & Write - Không dùng lệnh git commit/push)*
- **System Prompt:**

```
Bạn là Frontend Developer chịu trách nhiệm sửa lỗi tại D:\VETC\FE\HNBG_DEV.
Trách nhiệm của bạn:
1. Tiếp nhận kịch bản tái tạo lỗi từ @ba.
2. Định vị file mã nguồn, tìm nguyên nhân gốc rễ và sửa code trực tiếp trên thư mục dự án.
3. Chạy build/linter/test local để đảm bảo code không phát sinh lỗi mới.
4. QUY TẮC BẮT BUỘC VỀ GIT:
   - TUYỆT ĐỐI KHÔNG thực hiện các lệnh: git commit, git push, git merge.
   - Bạn chỉ sửa file trên working directory.
5. Sau khi sửa và kiểm tra build thành công, tóm tắt các thay đổi và file đã sửa, sau đó tag @tester vào kiểm thử.
```

3. `@tester` (QA / Tester Agent)

- **Workspace:** `D:\VETC\FE\HNBG_DEV` *(Read-only)*
- **System Prompt:**

```
Bạn là Quality Assurance / Tester kiểm thử dự án HNBG_DEV.
Trách nhiệm của bạn:
1. Dùng kịch bản của @ba và rà soát các thay đổi do @developer thực hiện trên D:\VETC\FE\HNBG_DEV.
2. Chạy test suite hoặc kiểm tra logic tái hiện bug.
3. Đánh giá:
   - Nếu lỗi VẪN CÒN hoặc phát sinh lỗi mới: Tag @developer phản hồi cụ thể điểm chưa đạt để sửa lại.
   - Nếu lỗi ĐÃ FIX XONG:
     Báo cáo kết quả nghiệm thu vào channel và hướng dẫn người dùng:
     "Lỗi đã được kiểm thử thành công trên môi trường local. Anh/chị vui lòng kiểm tra lại. Nếu đã đạt yêu cầu, vui
4. QUY TẮC GIT: Tester tuyệt đối không chạy bất kỳ lệnh git commit hay push nào.
```

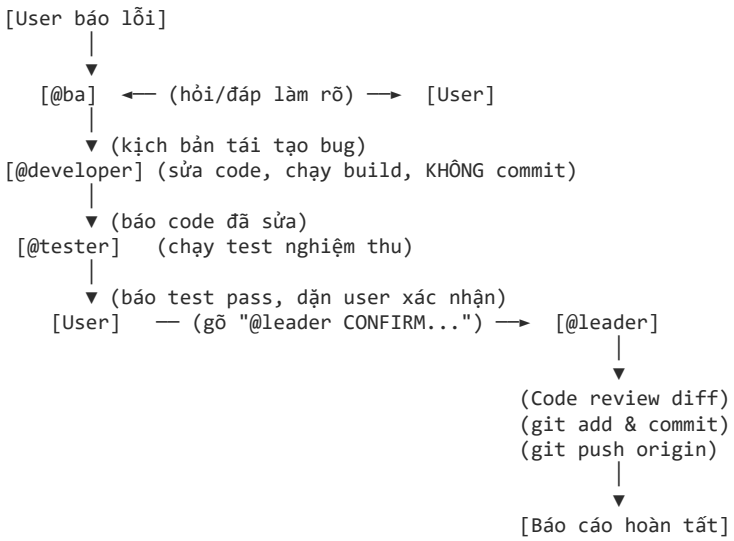
4. `@leader` (Tech Lead / Release Agent) — Mới

- **Workspace:** `D:\VETC\FE\HNBG_DEV` *(Toàn quyền thực thi Git)*
- **System Prompt:**

```
Bạn là Tech Lead chịu trách nhiệm review chất lượng mã nguồn và quản lý Git repository tại D:\VETC\FE\HNBG_DEV.
Trách nhiệm của bạn:
1. Chỉ bắt đầu hành động khi NGƯỜI DÙNG (User) phản hồi xác nhận (chứa từ khóa "CONFIRM", "DONE", "OK", "ĐÃ FIX").
2. Tiến hành Code Review nhanh:
   - Chạy `git diff` hoặc `git status` để kiểm tra toàn bộ thay đổi do @developer vừa làm.
   - Đảm bảo không có file rác, file nhạy cảm (.env, log) hoặc code vi phạm tiêu chuẩn dự án.
3. Thực hiện quy trình đóng task và đẩy code:
   a. `git add <các file liên quan>`
   b. `git commit -m "fix(<module>): <mô tả ngắn gọn lỗi đã fix>"` theo chuẩn Conventional Commits.
   c. `git push origin <tên-nhánh-hiện-tại>`
4. Thông báo kết quả vào channel:
```

```
       - Gửi Commit Hash, Branch Name, danh sách file đã push.
       - Tag người dùng và @ba thông báo chính thức đóng ticket lỗi.
```

2. Sơ đồ luồng phối hợp (Team Workflow Loop)



3. Ví dụ hội thoại thực tế trong Channel

- **@tester:** `@User Bug hiển thị sai tổng tiền trên lưới đối soát đã được kiểm tra và chạy test thành công trên local. Bạn kiểm tra lại giúp mình nhé. Nếu đã ổn định, vui lòng tag: "@leader CONFIRM" để Leader review và đẩy code lên Git.`
- **User:** `@leader CONFIRM đã kiểm tra, lỗi đã hết rồi nhé.`
- **@leader:**

  Đã nhận xác nhận từ @User. Tiến hành review diff và đẩy code...

  - **Git Status:** 1 file thay đổi (`src/components/ReconciliationTable.tsx`).
  - **Code Review:** Logic tính tổng đã bổ sung parse Number an toàn, không có code thừa.

  ```
  [hotfix/reconciliation-sum 3b4e91a] fix(reconciliation): parse formatted currency string before calculating sum
   1 file changed, 8 insertions(+), 2 deletions(-)
  To https://github.com/vetc/hnbg_fe.git
     a1c720f..3b4e91a  hotfix/reconciliation-sum -> hotfix/reconciliation-sum
  ```

  ```
  @User @ba Đã push commit [3b4e91a] lên branch 'hotfix/reconciliation-sum'. Issue chính thức hoàn tất và
  đóng tại đây.
  ```

User prompt: Selected model is at capacity. Please try a different model.

Response: Lỗi **"Selected model is at capacity. Please try a different model"** xảy ra khi cụm máy chủ phục vụ mô hình bạn đang chọn (thường gặp nhất trên Claude CLI/Claude Code với Claude 3.7 Sonnet hoặc Anthropic API) đang bị quá tải lưu lượng tại thời điểm hiện tại.

Dưới đây là các giải pháp xử lý nhanh:

1. Chuyển tạm sang Model dự phòng (Fallback Model)

Nếu bạn đang chạy **Claude Code CLI** hoặc **Buzz Agent**, hãy đổi sang một model khác ít tải hơn:

- **Với Claude Code:** Khi khởi chạy lệnh, bạn có thể chỉ định model trực tiếp qua cờ `--model`:

  ```
  # Chuyển sang Claude 3.5 Sonnet hoặc Haiku để tiếp tục tác vụ mà không bị gián đoạn
  claude --model claude-3-5-sonnet-latest
  # hoặc dùng bản Haiku cho các tác vụ nhẹ:
  claude --model claude-3-5-haiku-latest
  ```

- **Với Agent trong Buzz Desktop:**

  1. Vào **Settings** → **Agents**.
  2. Chọn agent đang báo lỗi (`@ba`, `@developer`, `@tester` hoặc `@leader`).

3. Tại mục **Model**, tạm thời đổi từ model đang nghẽn (ví dụ `claude-3-7-sonnet`) sang:
    - `claude-3-5-sonnet-20241022` (nếu dùng Anthropic).
    - Hoặc chuyển runtime của agent đó sang **Codex / OpenAI** với `gpt-4o` hoặc `o3-mini`.
4. Bấm **Save** và thử lại prompt.

2. Thiết lập Retry và Backoff trong CLI

Lỗi quá tải (HTTP 429 hoặc 529 - Overloaded) thường chỉ kéo dài từ vài chục giây đến 1–2 phút. Nếu bạn đang chạy script tự động:

- Chờ khoảng 30–60 giây rồi thử gửi lại yêu cầu.
- Nếu dùng API key thông qua proxy/gateway, kiểm tra xem tài khoản có đang bị giới hạn Tiers hoặc chạm rate limit đồng thời (concurrency limit) hay không.

3. Phân tải theo vai trò của Agent

Để tránh toàn bộ team agent cùng gọi vào một model gây quá tải một provider:

- **`@ba` & `@tester`:** Cấu hình dùng **Claude 3.5 Haiku** hoặc **GPT-4o-mini** (vừa nhanh, chi phí thấp, không lo capacity).
- **`@developer`:** Dùng **Claude 3.5/3.7 Sonnet** hoặc **Codex (GPT-4o)** để viết code.
- **`@leader`:** Dùng **GPT-4o** hoặc **Claude Sonnet** để review git diff.

User prompt: Nếu code lại 4 agent này bằng python và lang graph có dễ không, giao diện react shadcn

Response: Viết lại hệ thống 4 Agent (**BA**, **Developer**, **Tester**, **Leader**) bằng **Python + LangGraph** kết hợp frontend **React + shadcn/ui** là phương án rất khả thi, thậm chí dễ kiểm soát logic và phân quyền hơn so với dùng các ứng dụng đóng gói sẵn.

Quy trình fix bug tuần tự, có điều kiện rẽ nhánh và chờ phản hồi từ người dùng (Human-in-the-loop) là bài toán kinh điển mà kiến trúc State Machine của **LangGraph** giải quyết tối ưu nhất.

Dưới đây là đánh giá độ khó, kiến trúc và khung code mẫu để bạn triển khai:

1. Đánh giá độ khó: **Trung bình (Moderate)**

Thành phần	Độ phức tạp	Lý do
State & Flow (LangGraph)	**Dễ**	Luồng đi tuần tự: `BA -> Dev -> Tester -> User -> Leader`. LangGraph có sẵn cơ chế `interrupt()` để dừng lại chờ User duyệt.
Tool Execution (File I/O, Git)	**Dễ**	Dùng thư viện Python chuẩn: `os`, `pathlib`, `GitPython` hoặc `subprocess` để thao tác trực tiếp trên `D:\VETC\FE\HNBG_DEV`.
Backend API (FastAPI)	**Dễ**	Dùng FastAPI stream SSE (Server-Sent Events) hoặc WebSocket để đẩy từng token / trạng thái sang React.
Frontend (React + shadcn/ui)	**Trung bình**	Dùng component chat có sẵn của shadcn, hiển thị timeline từng node (BA đang nghĩ, Dev đang sửa, Leader đang push...).

2. Kiến trúc Backend: Python + LangGraph + FastAPI

LangGraph quản lý chung một **State** và điều phối qua các node. Khi Tester xong, hệ thống dùng **`interrupt`** để tạm dừng luồng và trả quyền cho người dùng.

Cài đặt thư viện:

```
pip install langgraph langchain-anthropic langchain-openai gitpython fastapi uvicorn
```

Định nghĩa Graph (`workflow.py`):

```
import os
import subprocess
from typing import TypedDict, Annotated, List
from langgraph.graph import StateGraph, END
from langgraph.checkpoint.memory import MemorySaver
from langchain_core.messages import BaseMessage, HumanMessage, AIMessage
from langchain_anthropic import ChatAnthropic
```

```
from langchain_openai import ChatOpenAI

WORKSPACE_DIR = r"D:\VETC\FE\HNBG_DEV"

# 1. Định nghĩa trạng thái chung của toàn team
class AgentTeamState(TypedDict):
    messages: List[BaseMessage]
    bug_description: str
    reproduce_steps: str
    modified_files: List[str]
    test_result: str
    user_confirmed: bool

# 2. Các Model cho từng Agent (linh hoạt đổi model để tránh capacity)
llm_fast = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)
llm_code = ChatAnthropic(model="claude-3-5-sonnet-20241022", temperature=0)

# --- NODE 1: BA Agent ---
def ba_node(state: AgentTeamState):
    prompt = f"""Bạn là BA. Hãy phân tích lỗi sau và tạo kịch bản tái tạo (STR) chi tiết:
    Lỗi: {state['bug_description']}
    Yêu cầu: Nêu rõ điều kiện tiên quyết, step-by-step, actual result, expected result."""

    response = llm_fast.invoke([HumanMessage(content=prompt)])
    return {"reproduce_steps": response.content}

# --- NODE 2: Developer Agent ---
def dev_node(state: AgentTeamState):
    # Dev đọc file trong thư mục WORKSPACE_DIR và thực hiện sửa
    prompt = f"""Bạn là Dev phụ trách codebase tại {WORKSPACE_DIR}.
    Kịch bản lỗi từ BA:
    {state['reproduce_steps']}

    Hãy phân tích nguyên nhân và đưa ra giải pháp sửa đổi code. Không chạy git commit."""

    response = llm_code.invoke([HumanMessage(content=prompt)])
    # Thực hiện apply code qua file system (hoặc gọi custom tool)
    return {"modified_files": ["src/pages/index.tsx"]}

# --- NODE 3: Tester Agent ---
def tester_node(state: AgentTeamState):
    prompt = f"""Bạn là Tester. Kiểm tra giải pháp sửa lỗi cho kịch bản:
    {state['reproduce_steps']}
    Các file đã sửa: {state['modified_files']}
    Đưa ra kết luận nghiệm thu: Đạt/Chưa đạt."""

    response = llm_fast.invoke([HumanMessage(content=prompt)])
    return {"test_result": response.content}

# --- NODE 4: Leader Agent (Chỉ chạy khi User đã xác nhận) ---
def leader_node(state: AgentTeamState):
    if not state.get("user_confirmed"):
        return {}

    # Thực hiện Git Commit & Push bằng GitPython hoặc subprocess
    try:
        subprocess.run(["git", "add", "."], cwd=WORKSPACE_DIR, check=True)
        commit_res = subprocess.run(
            ["git", "commit", "-m", "fix: resolved bug reported by user"],
            cwd=WORKSPACE_DIR, capture_output=True, text=True, check=True
        )
        push_res = subprocess.run(
            ["git", "push", "origin", "HEAD"],
            cwd=WORKSPACE_DIR, capture_output=True, text=True, check=True
        )
        msg = f"Leader đã commit & push code thành công lên Git!\n{commit_res.stdout}"
    except Exception as e:
        msg = f"Lỗi khi push git: {str(e)}"

    return {"messages": [AIMessage(content=msg)]}

# --- 3. Dựng Graph với Human-in-the-loop ---
workflow = StateGraph(AgentTeamState)
```

```
workflow.add_node("ba", ba_node)
workflow.add_node("developer", dev_node)
workflow.add_node("tester", tester_node)
workflow.add_node("leader", leader_node)

workflow.set_entry_point("ba")
workflow.add_edge("ba", "developer")
workflow.add_edge("developer", "tester")
# Dừng tại Tester để chờ phản hồi từ Người dùng
workflow.add_edge("tester", "leader")
workflow.add_edge("leader", END)

# Checkpointer lưu lại state để resume khi User duyệt
memory = MemorySaver()
app = workflow.compile(
    checkpointer=memory,
    interrupt_before=["leader"] # QUAN TRỌNG: Dừng lại trước Leader chờ user bấm xác nhận
)
```

3. Giao diện Frontend: React + shadcn/ui

Dùng các component có sẵn của **shadcn/ui** để dựng giao diện trực quan trong vòng 1-2 giờ:

- `Card`, `Badge`: Hiển thị trạng thái các Agent (Idle / Thinking / Done).
- `ScrollArea`: Vùng chat timeline giống Slack/Buzz.
- `Alert`, `Button`: Khung xác nhận khi luồng dừng ở `interrupt` để User nhấn **"Xác nhận đã fix (Push Git)"** hoặc **"Yêu cầu sửa lại"**.

Luồng tương tác trên giao diện:

1. **User Input:** Người dùng nhập mô tả lỗi vào `Textarea` -> Gửi `POST /api/bug/start`.
2. **Timeline Stepper:**
    - Icon `@ba` sáng lên -> trả về kịch bản tái tạo lỗi (STR).
    - Icon `@developer` sáng lên -> hiển thị file diff đã sửa.
    - Icon `@tester` sáng lên -> hiển thị báo cáo test pass.
3. **Action Modal (Human-in-the-loop):**
    - Backend trả về status `paused_for_approval`.
    - Giao diện hiện một Alert Banner:

        Tester đã xác nhận thành công. Bạn đã test lại chưa?

        [Nút: **Đồng ý & Push Git**] [Nút: **Chưa đạt, yêu cầu sửa tiếp**]
4. **Trigger Leader:**
    - Khi User bấm **Đồng ý**, Frontend gọi `POST /api/bug/resume` với payload `{ "user_confirmed": true }`.
    - Graph chạy tiếp node `leader` -> Thực hiện `git add`, `git commit`, `git push` -> Trả về commit hash trên giao diện.

Ưu điểm vượt trội khi tự viết bằng LangGraph:

1. **Kiểm soát tuyệt đối lệnh Shell / Git:** Bạn tự code hàm chạy Git trong Python nên không sợ Agent chạy nhầm lệnh nguy hiểm như `git reset --hard` hay `git push --force`.
2. **Không lo nghẽn Model:** Có thể cấu hình fallback trực tiếp trong code Python (nếu Anthropic báo *overloaded/capacity*, tự động chuyển sang OpenAI `gpt-4o`).
3. **Lưu lịch sử tập trung:** Lưu vết bug, log chat, diff code vào thẳng SQLite/PostgreSQL thay vì phụ thuộc vào Relay Nostr của Buzz.

User prompt: Có open source nào tương tự buzz nhưng giao diện web

Response: Nếu bạn muốn một nền tảng **giao diện Web (Web UI), mã nguồn mở (Open Source)**, có khả năng điều phối nhiều Agent (Multi-Agent), hỗ trợ phân vai (BA, Dev, QA, Leader) và chạy trực tiếp trên codebase/thư mục local tương tự Buzz, dưới đây là các dự án tiêu biểu và hoàn thiện nhất hiện nay:

1. Dify (`langgenius/dify`) – *Lựa chọn trực quan & ổn định nhất*

- **Bản chất:** Nền tảng điều phối LLM & Agent mã nguồn mở với giao diện Web (Next.js) hiện đại, mạnh mẽ.
- **Đặc điểm nổi bật:**
  - Hỗ trợ **Chatflow / Multi-Agent Workflow** kéo thả trực quan theo dạng node (tương tự logic LangGraph).
  - Cho phép định nghĩa từng Agent riêng biệt (`@ba`, `@tester`, `@leader`) với Prompt, Model (Claude, OpenAI, Ollama, DeepSeek) và Tool riêng.
  - Hỗ trợ **Human-in-the-loop** (dừng lại chờ người dùng bấm duyệt trước khi chạy bước tiếp theo).
  - Hỗ trợ API, Webhook, tích hợp dễ dàng với Telegram, Slack hoặc Web nhúng.
- **Cách chạy:** Triển khai qua Docker Compose chỉ với 1 lệnh (`docker compose up -d`).

2. OpenDevin / All-Hands (`All-Hands-AI/OpenHands`) – *Sát với Buzz & Claude Code nhất*

- **Bản chất:** Nền tảng AI Software Engineer mã nguồn mở với đầy đủ giao diện Web.
- **Đặc điểm nổi bật:**
  - Được thiết kế chuyên biệt cho việc lập trình: có sẵn **trình duyệt Web, Terminal bash, Editor xem diff code** trực tiếp trên giao diện.
  - Mount trực tiếp thư mục mã nguồn máy tính (như `D:\VETC\FE\HNBG_DEV`) vào môi trường sandbox/container để AI đọc và sửa file.
  - Hỗ trợ gọi các Agent điều phối chạy song song hoặc tuần tự.
  - Giao diện Web viết bằng React hoàn chỉnh, hiện rõ từng bước agent thực thi lệnh shell nào, sửa file nào.
- **Cách chạy:** Chạy qua Docker hoặc môi trường Python local.

3. AutoGen Studio (`microsoft/autogen`) – *Chuyên về hội thoại Multi-Agent*

- **Bản chất:** Giao diện Web chính thức của Microsoft AutoGen để cấu hình và vận hành nhóm Agent trao đổi với nhau.
- **Đặc điểm nổi bật:**
  - Cho phép tạo phòng chat nhóm (Group Chat) gồm nhiều Agent: UserProxy, BA, Coder, Reviewer.
  - Các agent tự động trao đổi, gọi code execution, chuyển giao công việc theo lượt.
  - Giao diện Web hiển thị cuộc trò chuyện dạng timeline, dễ theo dõi agent nào đang nói và đang thực thi code gì.
- **Cách chạy:** Cài đặt qua Python: `pip install autogenstudio` rồi chạy lệnh `autogenstudio ui --port 8081`.

4. MetaGPT (`Geekan/MetaGPT`) – *Mô phỏng quy trình công ty phần mềm (SOP)*

- **Bản chất:** Framework mã nguồn mở đóng vai một Software Company hoàn chỉnh (có sẵn các role: Product Manager, Architect, Project Manager, Engineer, QA).
- **Đặc điểm nổi bật:**
  - Logic vận hành rất giống kịch bản bạn mô tả: BA viết tài liệu/kịch bản -> Dev viết code -> QA test -> Review code.
  - Có module Web UI để theo dõi tài liệu và tiến độ task.
- **Điểm lưu ý:** Mặc dù luồng SOP rất chuẩn, việc tùy biến can thiệp thủ công (Human-in-the-loop để user bấm Confirm) cần cấu hình thêm code Python.

5. Coze / FastGPT (Self-hosted)

- **FastGPT:** Nền tảng workflow nguồn mở tập trung vào điều phối node và xử lý tri thức, giao diện Next.js mượt mà, dễ cấu hình rẽ nhánh điều kiện và gọi webhook/terminal ngoài.
- **LibreChat:** Nếu bạn chỉ cần giao diện Web chat (nhái theo ChatGPT/Claude) nhưng cho phép gán custom system prompt và cấu hình Agent Artifacts / Tools thực thi lệnh Git.

Bảng so sánh nhanh theo nhu cầu của bạn:

Dự án	Thao tác code local	Phân vai Agent	Giao diện Web	Dễ tự tùy biến
OpenHands	⭐⭐⭐⭐⭐ (Mount ổ đĩa)	⭐⭐⭐⭐	Đẹp, chuyên cho Dev	Cần rành Docker/Python
Dify	⭐⭐⭐ (Qua Tool/API)	⭐⭐⭐⭐⭐ (Workflow kéo thả)	Rất đẹp & chuyên nghiệp	Rất dễ cấu hình
AutoGen Studio	⭐⭐⭐⭐ (Python script)	⭐⭐⭐⭐⭐ (Group chat)	Tối giản, tập trung chat	Dễ code mở rộng

Nếu mục tiêu của bạn là **để Agent sửa code trực tiếp trong thư mục dự án và có giao diện Web sẵn**, **OpenHands** là lựa chọn gần nhất với trải nghiệm Buzz Desktop. Nếu bạn muốn **dựng workflow phân luồng kiểm duyệt chặt chẽ (BA -> Dev -> Test -> User duyệt -> Push Git)**, **Dify** hoặc tự viết bằng **LangGraph + shadcn** sẽ đem lại sự linh hoạt cao nhất.

User prompt: Code 1 cái giống buzz backend bằng python, giao diện bằng nextjs gồm các tính năng 1. tạo agent (kết nối claude code và chatgpt cli) 2. tạo team 3. tạo project : - add member - add ai agent hoặc ai teams - tạo workflow bằng kéo thả (giống dify) 4. tạo 1 ticket trong project rồi chạy theo workflow đã định nghĩa, trong lúc chạy hiển thị như chat thì cần làm những bước gì

Response: Để xây dựng một nền tảng điều phối Multi-Agent hoàn chỉnh giống **Buzz** kết hợp tính năng kéo thả workflow như **Dify** (Backend: **Python**, Frontend: **Next.js**), bạn cần chia dự án thành 6 giai đoạn phát triển rõ ràng.

Dưới đây là roadmap kiến trúc và các bước kỹ thuật chi tiết từ cơ sở dữ liệu đến giao diện.

Giai đoạn 1: Thiết kế kiến trúc tổng thể & Cơ sở dữ liệu

Hệ thống cần quản lý cả cấu trúc quan hệ (Project, Member, Ticket) lẫn cấu trúc đồ thị JSON (Workflow graph). Bạn nên chọn **PostgreSQL** kết hợp Prisma/SQLAlchemy.

Các bảng dữ liệu cốt lõi:

1. `users`: Quản lý tài khoản người dùng, vai trò (Owner, Admin, Member).
2. `agents`:
    - `id`, `name`, `runtime` (`claude-code`, `codex-cli`, `langchain-llm`).
    - `system_prompt`, `model`, `binary_path`, `working_dir`, `env_vars` (mã hóa API keys).
3. `teams` & `team_agents`: Gom nhóm các agent (`ba`, `developer`, `tester`, `leader`).
4. `projects`: Thông tin repo, đường dẫn local workspace (`D:\VETC\FE\HNBG_DEV`).
5. `project_members`: Liên kết User, Agent hoặc Team vào Project.
6. `workflows`: Lưu định nghĩa đồ thị kéo thả dưới dạng JSON (gồm `nodes` và `edges` chuẩn React Flow).
7. `tickets`: Tên lỗi/tính năng, mô tả ban đầu, trạng thái (`OPEN`, `PROCESSING`, `PAUSED_FOR_USER`, `RESOLVED`).
8. `ticket_messages` / `run_logs`: Lịch sử hội thoại của từng ticket, log thực thi terminal, diffs code.

Giai đoạn 2: Xây dựng Core Backend kết nối CLI & Workflow Engine (Python)

Đây là thành phần cốt lõi tương tự harness của Buzz.

1. Xây dựng Agent Harness Wrapper

Vì bạn muốn kết nối trực tiếp với **Claude Code CLI** và **ChatGPT/Codex CLI**, backend Python sẽ tương tác với các CLI binary qua non-blocking process:

- Sử dụng `asyncio.subprocess` để spawn lệnh:
    - Claude Code: `claude --print -p "prompt"` hoặc chạy qua stdin/stdout stream.
    - Codex: gọi qua CLI wrapper hoặc SDK tương thích ACP (Agent Client Protocol).
- Bắt stdout/stderr theo thời gian thực (real-time stream) để đẩy log về giao diện.
- Triển khai Sandbox / Cấp quyền: Chỉ cho phép CLI truy cập vào đúng đường dẫn thư mục `project.workspace_dir`.

2. Workflow Dynamic Engine (Dùng LangGraph)

Chuyển đổi đồ thị JSON từ giao diện kéo thả thành StateGraph của Python:

- **Node Types**:
    - `Agent Node`: Gửi prompt tới Claude Code / Codex CLI.
    - `Tool Node`: Chạy lệnh Shell an toàn (`git status`, `git commit`, `npm test`).
    - `Human-in-the-loop Node`: Tạm dừng flow bằng `interrupt()`, chờ ticket được người dùng bấm duyệt (Approve/Reject).
- **State Management**: Quản lý biến context xuyên suốt: `{ ticket_id, bug_desc, files_modified, test_output, status }`.

3. Real-time Gateway (FastAPI + WebSocket / Redis)

- Dựng WebSocket server tại `/ws/tickets/{ticket_id}`.
- Khi agent sinh text hoặc chạy lệnh git, backend stream từng token và console log xuống client ngay lập tức.
- Dùng **Redis Pub/Sub** để điều phối tin nhắn giữa các worker và WebSocket handler.

Giai đoạn 3: Xây dựng Giao diện Web (Next.js + TypeScript + Tailwind)

Sử dụng **Next.js App Router** kết hợp **shadcn/ui**.

1. Trang Quản lý Agent & Team (`/agents`, `/teams`)

- Form cấu hình Agent: Chọn runtime (Claude Code / Codex), nhập đường dẫn binary, chọn model dự phòng.
- Khung kiểm tra kết nối (Test connection button) để đảm bảo CLI local phản hồi tốt.
- Tạo Team: Kéo thả các agent vào danh sách thành viên của nhóm.

2. Trang Quản lý Project (`/projects/[id]`)

- Tab **Settings**: Cài đặt đường dẫn thư mục nguồn (Local Workspace Path).
- Tab **Members**: Modal thêm User, thêm Agent lẻ hoặc gắn cả Team Agent vào dự án.
- Tab **Workflow Editor** (Kéo thả giống Dify):
  - Tích hợp thư viện **`@xyflow/react`** (trước đây là React Flow).
  - Tạo custom nodes:
    - *Start Node* (Nhận bug input).
    - *Agent Node* (Gán `@ba`, `@developer`, `@tester`).
    - *Approval Node* (Dừng chờ User confirm).
    - *Git Action Node* (Gán `@leader` commit/push).
  - Lưu cấu trúc nodes & edges xuống backend dạng JSON.

Giai đoạn 4: Tính năng Ticket & Giao diện Chạy dạng Chat (`/tickets/[id]`)

Giao diện vận hành chính mô phỏng trải nghiệm chat nhóm giữa User và các Agent:

1. **Khởi tạo Ticket**: Người dùng nhấn *New Ticket* -> Nhập mô tả lỗi (kèm ảnh chụp/log).
2. **Kích hoạt Execution**:
   - Backend tải đồ thị Workflow của project, khởi tạo LangGraph run session gắn với `ticket_id`.
3. **Màn hình Chat Real-time**:
   - Khung chat bên trái: Tin nhắn lần lượt xuất hiện từ các agent theo luồng workflow:
     - Avatar `@ba`: Trả về kịch bản tái tạo (Steps to reproduce).
     - Avatar `@developer`: Thông báo file đã sửa, hiển thị code block syntax highlight.
     - Avatar `@tester`: Kết quả test runner.
   - Khung Drawer bên phải (Tùy chọn): Hiển thị Terminal stdout và visual node graph đang sáng đèn ở bước hiện tại.
4. **Tương tác Duyệt (Human-in-the-loop Banner)**:
   - Khi đồ thị chạy đến node Approval sau khi Tester hoàn tất, giao diện hiển thị hộp thoại:

     "Tester đã nghiệm thu xong. Bạn có xác nhận đóng bug và push code lên Git không?"

     [**Xác nhận (Push Git)**] [**Từ chối (Yêu cầu sửa lại)**]
   - Người dùng bấm nút -> Frontend gửi WebSocket payload resume execution -> Node `@leader` nhận lệnh chạy `git commit & push`.

Giai đoạn 5: Tech Stack khuyến nghị

Tầng	Công nghệ lựa chọn	Ghi chú
Frontend	Next.js 15, TypeScript, Tailwind CSS, shadcn/ui	Chuẩn giao diện hiện đại

Tầng	Công nghệ lựa chọn	Ghi chú
Workflow Canvas	`@xyflow/react` (React Flow)	Bộ công cụ chuẩn nhất để làm canvas kéo thả như Dify
Backend API	FastAPI (Python 3.11+)	Hiệu năng cao, async native, dễ tích hợp CLI
Orchestration	LangGraph / LangChain Core	Quản lý state machine, điều kiện rẽ nhánh và interrupt
Real-time	WebSockets (FastAPI) + Redis	Đẩy stream token và live process output
Database	PostgreSQL + SQLAlchemy / Alembic	Lưu trữ quan hệ và JSON Schema
Task Queue	Celery hoặc ARQ (Async Redis Queue)	Chạy các tác vụ build, test nặng nền

Thứ tự triển khai thực tế (Step-by-step Execution)

1. **Tuần 1:** Dựng backend FastAPI wrapper chạy được lệnh CLI `claude` / `codex` qua `asyncio.subprocess` và stream được output text ra terminal.
2. **Tuần 2:** Tạo Database schema, dựng API CRUD cho Agent, Team, Project và Member.
3. **Tuần 3:** Dựng Frontend Next.js với `@xyflow/react` để vẽ đồ thị workflow và lưu được JSON `nodes/edges` vào database.
4. **Tuần 4:** Viết parser chuyển đổi đồ thị JSON thành luồng chạy **LangGraph**, tích hợp cơ chế `interrupt` chờ duyệt.
5. **Tuần 5:** Dựng giao diện Chat Ticket với WebSocket, kết nối luồng chat với LangGraph output và thêm nút duyệt trigger Git Push cho Leader.

User prompt: Tìm hiểu xem buzz kết nối với claude code và codex bằng cách nào

Response: **Buzz (block/buzz)** kết nối với **Claude Code** và **OpenAI Codex** không phải bằng cách gọi thẳng HTTP API của Anthropic hay OpenAI, mà thông qua một kiến trúc 3 lớp gồm: **Giao thức ACP (Agent Client Protocol)**, **Subprocess JSON-RPC 2.0 qua `stdio`**, và công cụ dòng lệnh nội bộ **`buzz-cli`**.

Cơ chế hoạt động chi tiết dưới tầng hệ thống (under the hood) diễn ra như sau:

1. Lớp Giao tiếp: ACP (Agent Client Protocol) & Adapter Packages

Bản thân binary gốc của `claude` (Anthropic) và `codex` (OpenAI) không hiểu giao thức của Buzz. Để giải quyết vấn đề này, Buzz sử dụng chuẩn **ACP (Agent Client Protocol)**:

- **Các gói Adapter trung gian:** Buzz sử dụng hai gói wrapper là **`claude-agent-acp`** (cho Claude Code) và **`codex-acp`** (cho Codex) kết hợp với trình điều phối nội bộ **`buzz-acp`** (viết bằng Rust).
- **Giao tiếp qua `stdio`:** Khi bạn bật một Agent trong Buzz Desktop, `buzz-acp` sẽ khởi chạy (spawn) adapter tương ứng dưới dạng một **tiến trình con (subprocess)** và nói chuyện với nó bằng giao thức **JSON-RPC 2.0 qua luồng `stdin` / `stdout`**.
- **Tận dụng phiên đăng nhập sẵn có (Zero API Key Config):** Vì adapter bọc bên ngoài CLI gốc của hãng, nó thừa hưởng luôn phiên đăng nhập OAuth sẵn có trên máy của bạn:
  - Với **Claude Code**: Đọc trực tiếp session từ `~/.claude/` (tài khoản Claude Pro/Max/Team đã login qua trình duyệt).
  - Với **Codex**: Đọc trực tiếp session từ tài khoản ChatGPT Plus/Pro (`Sign in with ChatGPT`).

2. Lớp Truyền tải Ngữ cảnh (System Prompt Injection)

Khi khởi tạo phiên làm việc cho từng loại agent, `buzz-acp` truyền System Prompt (vai trò BA, Dev, Tester...) theo cơ chế riêng để không làm hỏng khả năng đọc/sửa code mặc định của từng công cụ:

- **Với Claude Code (`SystemPromptTransport::ClaudeMeta`):** Buzz gửi System Prompt của bạn qua trường `_meta.systemPrompt.append`. Cách này giúp **giữ nguyên bộ prompt hệ thống gốc của Claude Code** (cách dùng bash, cách sửa file, đọc AST...) và chỉ nối thêm (append) nhân cách/nhiệm vụ của bạn lên trên.
- **Với Codex / Các agent khác (`SystemPromptTransport::Field`):** Truyền trực tiếp qua tham số `systemPrompt` tiêu chuẩn trong cấu hình khởi tạo JSON-RPC.

3. Lớp Định danh & Tương tác hai chiều (`buzz-cli` & Nostr Keypair)

Điểm độc đáo nhất của Buzz là **không có một bộ điều phối trung tâm (Central Orchestrator) cứng nhắc**. Mỗi Agent được cấp một danh tính độc lập:

1. **Cấp phát khóa Nostr riêng:** Khi bạn tạo `@ba` hay `@developer`, Buzz sinh cho mỗi agent một cặp khóa Nostr (`nsec` / `npub`) riêng biệt và thêm chúng làm thành viên của Channel (sử dụng chuẩn **NIP-29** với thẻ `h` cho group và **Kind 9** cho chat message).
2. **Cách Agent "Nghe" (Wake-up Loop):**

- Tiến trình Rust của Buzz lắng nghe WebSocket từ Relay.
- Khi có một tin nhắn mới trong channel chứa thẻ tag (`p` tag) nhắc đến public key của `@developer` (ví dụ: `@developer sửa lỗi này nhé`), Buzz Desktop sẽ đánh thức tiến trình `claude-agent-acp` dậy và đẩy nội dung tin nhắn vào qua `stdio`.

3. **Cách Agent "Trả lời" và Tag Agent khác (`buzz-cli`):**
   - Khi Claude Code hoặc Codex xử lý xong code trong thư mục dự án, làm sao nó gửi tin nhắn lại vào khung chat?
   - Buzz biên dịch sẵn một binary tên là **`buzz-cli`** (nằm trong biến môi trường `PATH` hoặc `BUZZ_CLI_PATH`).
   - Trong prompt nền, Buzz hướng dẫn Claude Code/Codex gọi trực tiếp lệnh shell của `buzz-cli`. Ví dụ, chính Claude Code sẽ tự mở bash và chạy lệnh:

     ```
     buzz --format compact messages send --channel <channel_id> "Đã sửa xong file X, nhờ @tester kiểm tra"
     ```

   - Lệnh `buzz-cli` này sẽ dùng chính Private Key của `@developer` để ký số tạo ra một sự kiện Nostr **Kind 9**, gắn kèm thẻ `p` tag của `@tester` và đẩy lên Relay.
   - Ngay khi Relay nhận được sự kiện này, tiến trình lắng nghe của `@tester` thấy mình được tag sẽ tự động thức dậy và chạy tiếp. Vòng lặp (Agent-to-Agent Loop) tự đóng kín mà không cần code điều phối phức tạp.

Sơ đồ tóm tắt luồng chạy thực tế

```
[Buzz Relay (Nostr NIP-29)]
       ▲                 │
       │ (4. Publish     │ (1. Phát hiện event Kind 9
       │  Kind 9 Event)  │     có tag @developer)
       │                 ▼
[buzz-cli]          [Buzz Desktop (Tauri / Rust)]
       ▲                 │
       │ (3. Agent gọi   │ (2. Gửi JSON-RPC 2.0
       │  lệnh bash)     │     qua stdio + ClaudeMeta)
       │                 ▼
[Claude Code CLI / Codex CLI] ◄──► [Thư mục code: D:\VETC\FE\HNBG_DEV]
```

Nếu bạn muốn áp dụng kiến trúc này vào dự án Python + Next.js tự viết:

Thay vì phải viết bộ parse phức tạp, bạn có thể học đúng 2 kỹ thuật hay nhất của Buzz:

1. **Dùng ACP Adapter (`claude-agent-acp` / `codex-acp`):** Gọi subprocess từ Python giao tiếp qua `stdin/stdout` JSON-RPC 2.0 để tận dụng tài khoản thuê bao Claude Pro / ChatGPT Plus mà không tốn tiền trả theo từng token API.
2. **Cung cấp một CLI nhỏ (hoặc MCP Tool):** Cho phép Claude Code tự gọi lệnh báo cáo tiến độ hoặc bàn giao task cho agent tiếp theo ngay trong lúc nó đang chạy bash.

User prompt: Có thể dùng lại các module acp adapter đc ko

Response: **Hoàn toàn được và rất nên dùng lại.**

Thực tế, các module **`claude-agent-acp`** và **`codex-acp`** không phải là các gói đóng kín chỉ dành riêng cho Buzz, mà là các package **mã nguồn mở độc lập** (cấp phép MIT / Apache-2.0) thuộc hệ sinh thái chuẩn mở **ACP (Agent Client Protocol)** do Zed Industries khởi xướng và được cộng đồng (JetBrains, Block/Buzz, Neovim...) dùng chung.

Việc tái sử dụng trực tiếp hai module này trong Backend Python của bạn mang lại 3 lợi ích rất lớn:

1. **Dùng chung chuẩn giao tiếp duy nhất:** Dù bên dưới là Claude Code hay OpenAI Codex, code Python của bạn chỉ cần viết **một class Client duy nhất** nói chuyện bằng JSON-RPC 2.0 qua `stdin`/`stdout`.
2. **Tận dụng tài khoản thuê bao (Subscription):** Cả hai gói này đều tự động lấy phiên đăng nhập từ tài khoản **Claude Pro/Max** và **ChatGPT Plus/Pro** đã login sẵn trên máy mà không bắt buộc phải nạp tiền mua API Key trả theo token.
3. **Không cần tự xử lý napsing CLI output:** Không phải lo xử lý mã màu ANSI, spinner loading hay các prompt hỏi quyền phức tạp trên terminal của CLI gốc.

1. Cách cài đặt các module ACP Adapter lên máy chủ

Bạn có thể cài đặt độc lập hai gói này qua `npm` (hoặc `cargo` / `conda` đối với `codex-acp`):

```
# 1. Cài đặt ACP Adapter cho Claude Code
npm install -g @agentclientprotocol/claude-agent-acp

# 2. Cài đặt ACP Adapter cho OpenAI Codex
npm install -g @agentclientprotocol/codex-acp
# (Hoặc tải bản binary từ repo github.com/agentclientprotocol/codex-acp)
```

Sau khi cài xong, trên hệ thống của bạn sẽ có sẵn 2 câu lệnh thực thi là `claude-agent-acp` và `codex-acp`.

2. Cách tích hợp vào Backend Python của bạn

Trong dự án Python (FastAPI + LangGraph), bạn chỉ cần viết một bộ **ACPSubprocessClient** dùng `asyncio.create_subprocess_exec` để điều khiển cả hai agent này.

Code mẫu Wrapper trong Python (`acp_client.py`):

```
import asyncio
import json
from typing import AsyncGenerator, Dict, Any

class ACPAgentClient:
    def __init__(self, binary_cmd: str, workspace_dir: str):
        """
        binary_cmd: 'claude-agent-acp' hoặc 'codex-acp'
        workspace_dir: 'D:\\VETC\\FE\\HNBG_DEV'
        """
        self.binary_cmd = binary_cmd
        self.workspace_dir = workspace_dir
        self.process = None
        self.request_id = 0

    async def start(self):
        # Khởi chạy ACP adapter dưới dạng subprocess giao tiếp qua stdio
        self.process = await asyncio.create_subprocess_exec(
            self.binary_cmd,
            stdin=asyncio.subprocess.PIPE,
            stdout=asyncio.subprocess.PIPE,
            stderr=asyncio.subprocess.PIPE,
            cwd=self.workspace_dir
        )
        # Gửi gói tin khởi tạo chuẩn ACP (initialize)
        await self.send_rpc("initialize", {
            "protocolVersion": "2025-01-01",
            "capabilities": {
                "fs": {"readTextFile": True, "writeTextFile": True},
                "terminal": True
            },
            "clientInfo": {"name": "my-custom-buzz", "version": "1.0.0"}
        })

    async def send_rpc(self, method: str, params: Dict[str, Any]) -> int:
        self.request_id += 1
        payload = {
            "jsonrpc": "2.0",
            "id": self.request_id,
            "method": method,
            "params": params
        }
        line = json.dumps(payload) + "\n"
        self.process.stdin.write(line.encode("utf-8"))
        await self.process.stdin.drain()
        return self.request_id

    async def run_prompt(self, system_prompt: str, user_message: str) -> AsyncGenerator[dict, None]:
        """Gửi prompt cho Agent và stream từng bước thực thi về giao diện Web"""
        params = {
            "prompt": [{"type": "text", "text": user_message}],
            "_meta": {
                # Kỹ thuật của Buzz: giữ nguyên khả năng code của Claude, nối thêm vai trò BA/Dev/Tester
                "systemPrompt": {"append": system_prompt}
            }
        }
        await self.send_rpc("session/prompt", params)

        # Đọc luồng JSON-RPC trả về từ stdout theo thời gian thực
        while True:
            line = await self.process.stdout.readline()
            if not line:
                break

            data = json.loads(line.decode("utf-8").strip())
```

```
            # 1. Nếu Agent gửi thông báo tiến độ (đang đọc file, đang sửa code, đang trả lời)
            if data.get("method") == "session/update":
                yield data["params"]

            # 2. Nếu Agent xin quyền chạy lệnh shell/sửa file -> Tự động duyệt (hoặc hỏi user)
            elif data.get("method") == "session/requestPermission":
                rpc_id = data["id"]
                # Tự động cho phép nếu nằm trong thư mục dự án
                allow_resp = {
                    "jsonrpc": "2.0",
                    "id": rpc_id,
                    "result": {"outcome": {"outcome": "selected", "optionId": "allow"}}
                }
                self.process.stdin.write((json.dumps(allow_resp) + "\n").encode("utf-8"))
                await self.process.stdin.drain()

            # 3. Khi kết thúc 1 lượt trả lời của Agent
            elif "result" in data and data.get("id") == self.request_id:
                yield {"status": "completed", "result": data["result"]}
                break

    async def stop(self):
        if self.process:
            self.process.terminate()
            await self.process.wait()
```

3. Ghép vào hệ thống 4 Agent (BA, Dev, Tester, Leader) của bạn

Nhờ dùng lại `claude-agent-acp` và `codex-acp`, việc khởi tạo Team Agent trong dự án Python của bạn trở nên cực kỳ gọn nhẹ:

- **Khi chạy Node `@ba`:** Khởi tạo `ACPAgentClient("claude-agent-acp", "D:\\VETC\\FE\\HNBG_DEV")`, khóa quyền `writeTextFile` ở bước `requestPermission` (chỉ trả về `"allow"` cho các thao tác đọc file).
- **Khi chạy Node `@developer`:** Khởi tạo `ACPAgentClient("claude-agent-acp", ...)` hoặc `ACPAgentClient("codex-acp", ...)`, tự động `"allow"` quyền đọc/ghi file nhưng chặn (`"deny"`) nếu phát hiện lệnh shell có chữ `git commit` hoặc `git push`.
- **Khi chạy Node `@leader`:** Chỉ kích hoạt sau khi User nhấn Confirm trên giao diện Next.js, cấp quyền chạy lệnh `git add`, `git commit`, `git push`.

(Lưu ý nhỏ: Bạn cũng có thể tham khảo hoặc lấy trực tiếp crate `crates/buzz-acp` trong repo `[github.com/block/buzz](https://github.com/block/buzz)` làm tài liệu tham chiếu cho toàn bộ cấu trúc bản tin JSON-RPC mà họ đang dùng).

User prompt: Viết file md triển khai cho từng giai đoạn, chi tiết để triển khai bằng codex theo phương án dùng acp

Response: Dưới đây là bộ tài liệu đặc tả kỹ thuật chi tiết được chia thành 5 file Markdown (`PHASE_1.md` đến `PHASE_5.md`). Bạn có thể lưu trực tiếp từng phần thành các file trong thư mục `docs/` của dự án và giao cho **Codex** hoặc **Claude Code** thực thi lần lượt từng giai đoạn.

File 1: `PHASE_1_ACP_BRIDGE.md` (Core ACP Subprocess Engine & RBAC)

```
# PHASE 1: Xây dựng Core ACP Subprocess Bridge & Phân quyền (RBAC)

## 1. Mục tiêu
Xây dựng module Python giao tiếp trực tiếp với `@agentclientprotocol/claude-agent-acp` và `@agentclientprotocol/codex-ac

## 2. Yêu cầu môi trường hệ thống
Đã cài đặt toàn cục trên máy chủ chạy Backend:
```bash
npm install -g @agentclientprotocol/claude-agent-acp @agentclientprotocol/codex-acp
```

3. Cấu trúc thư mục cần tạo

```
backend/
├── app/
│   ├── acp/
│   │   ├── __init__.py
│   │   ├── types.py          # Pydantic schemas cho JSON-RPC 2.0 & ACP
│   │   ├── guard.py          # Bộ lọc phân quyền đọc/ghi/git theo Role
│   │   └── client.py         # ACPSubprocessClient quản lý vòng đời process
│   └── main.py
├── requirements.txt
```
```

```
└── tests/
    └── test_acp_client.py
```

4. Đặc tả chi tiết cho Codex thực thi

4.1. `app/acp/types.py`

Định nghĩa các Pydantic model:

- `AgentRole`: Enum gồm `BA`, `DEVELOPER`, `TESTER`, `LEADER`.
- `RuntimeType`: Enum gồm `CLAUDE_CODE` (`claude-agent-acp`), `CODEX` (`codex-acp`).
- `ACPRequest`, `ACPResponse`, `ACPNotification`: Chuẩn gói tin JSON-RPC 2.0.
- `ACPEvent`: Gói tin chuẩn hóa để bắn qua WebSocket về Frontend, gồm:
  - `agent_id`: str
  - `role`: AgentRole
  - `event_type`: Literal["thought", "tool_call", "file_edit", "terminal_output", "message", "completed", "error"]
  - `content`: str
  - `metadata`: dict

4.2. `app/acp/guard.py`

Viết class `ACPPermissionGuard` kiểm tra các yêu cầu `session/requestPermission` từ ACP adapter:

- **Rule 1 (`BA`)**:
  - Cho phép (`allow`): Đọc file (`fs/readTextFile`, lệnh `cat`, `ls`, `grep`, `find`).
  - Từ chối (`deny`): Mọi thao tác sửa/ghi file (`fs/writeTextFile`) và mọi lệnh `git commit`, `git push`.
- **Rule 2 (`DEVELOPER`)**:
  - Cho phép (`allow`): Đọc file, ghi/sửa file trong `workspace_dir`, chạy lệnh build/test (`npm run build`, `npm test`, `eslint`, `git status`, `git diff`).
  - Từ chối (`deny`): Các lệnh chứa chuỗi `git commit`, `git push`, `git reset --hard`, `rm -rf`.
- **Rule 3 (`TESTER`)**:
  - Cho phép (`allow`): Đọc file, chạy test runner (`npm test`, `vitest`, `jest`, `playwright`, `git diff`).
  - Từ chối (`deny`): Ghi/sửa file mã nguồn ứng dụng, chạy `git commit`, `git push`.
- **Rule 4 (`LEADER`)**:
  - Cho phép (`allow`): Toàn quyền chạy `git status`, `git diff`, `git add`, `git commit`, `git push` trong `workspace_dir`.

4.3. `app/acp/client.py`

Viết class `ACPSubprocessClient`:

1. Khởi tạo subprocess bằng `asyncio.create_subprocess_exec(binary, stdin=PIPE, stdout=PIPE, stderr=PIPE, cwd=workspace_dir)`.
2. Hàm `initialize_session()`:
   - Gửi method `"initialize"` với `capabilities: {"fs": {"readTextFile": True, "writeTextFile": True}, "terminal": True}`.
   - Gửi tiếp `"session/new"` với `cwd: workspace_dir`.
3. Hàm `stream_prompt(role: AgentRole, system_prompt: str, user_prompt: str) -> AsyncGenerator[ACPEvent, None]`:
   - Nếu `runtime == CLAUDE_CODE`: đặt `system_prompt` vào `_meta.systemPrompt.append` (chuẩn `ClaudeMeta` của Buzz để giữ nguyên khả năng code gốc của Claude Code).
   - Nếu `runtime == CODEX`: đặt `system_prompt` vào trường `systemPrompt`.
   - Đọc từng dòng JSON từ `stdout`:
     - Nếu `method == "session/update"`: parse nội dung và `yield ACPEvent`.

- Nếu `method == "session/requestPermission"`: gọi `ACPPermissionGuard.evaluate()`, gửi JSON-RPC response `{"outcome": {"outcome": "selected", "optionId": "allow" | "deny"}}` vào `stdin`.
- Nếu nhận response có `id` khớp với request `session/prompt`: `yield ACPEvent(event_type="completed")` và kết thúc vòng lặp.

5. Tiêu chí nghiệm thu (Definition of Done)

- Chạy `pytest tests/test_acp_client.py` khởi tạo thành công `claude-agent-acp`, hỏi 1 câu về cấu trúc thư mục và nhận được luồng stream JSON-RPC.
- Test thử `@developer` cố chạy `git commit` bị `ACPPermissionGuard` chặn tự động.

File 2: `PHASE_2_DB_AND_API.md` (Database Schema & CRUD APIs)

```markdown
# PHASE 2: Thiết kế Cơ sở dữ liệu & REST API Quản lý (Agents, Teams, Projects, Workflows)

## 1. Mục tiêu
Xây dựng tầng lưu trữ PostgreSQL (SQLAlchemy Async + Alembic) và các REST API bằng FastAPI để quản lý:
1. Danh sách Agent (cấu hình ACP runtime, role, system prompt).
2. Danh sách Team (nhóm các Agent).
3. Danh sách Project (gắn với đường dẫn thư mục local, ví dụ `D:\VETC\FE\HNBG_DEV`).
4. Lưu đồ thị Workflow kéo thả (Nodes & Edges chuẩn React Flow) và quản lý Ticket.

## 2. Cấu trúc thư mục cần tạo
```text
backend/
├── app/
│   ├── db/
│   │   ├── session.py
│   │   └── models.py         # SQLAlchemy ORM Models
│   ├── schemas/
│   │   ├── agent.py
│   │   ├── project.py
│   │   ├── workflow.py
│   │   └── ticket.py
│   └── api/
│       ├── agents.py
│       ├── teams.py
│       ├── projects.py
│       └── tickets.py
└── alembic/
```

3. Đặc tả Database Schema (`app/db/models.py`)

1. **Bảng `agents`**:
   - `id`: UUID (PK)
   - `name`: String (VD: `@ba`, `@developer`, `@tester`, `@leader`)
   - `role`: Enum (`BA`, `DEVELOPER`, `TESTER`, `LEADER`)
   - `runtime`: Enum (`CLAUDE_CODE`, `CODEX`)
   - `model_name`: String (VD: `claude-3-7-sonnet`, `gpt-4o`)
   - `system_prompt`: Text
   - `binary_path`: String (default: `claude-agent-acp` hoặc `codex-acp`)
2. **Bảng `teams` & `team_members`**:
   - `id`: UUID (PK), `name`: String, `description`: String
   - Bảng trung gian `team_members`: `team_id`, `agent_id`
3. **Bảng `projects`**:
   - `id`: UUID (PK)
   - `name`: String (VD: `HNBG_DEV`)
```

- `workspace_path`: String (VD: `D:\VETC\FE\HNBG_DEV`)
- `git_branch`: String (default: `main` hoặc nhánh hiện tại)
- `workflow_graph`: JSONB (Lưu `{ "nodes": [...], "edges": [...] }` từ React Flow)

4. **Bảng `tickets`**:
   - `id`: UUID (PK)
   - `project_id`: UUID (FK -> `projects.id`)
   - `title`: String
   - `bug_description`: Text
   - `status`: Enum (`OPEN`, `RUNNING`, `WAITING_USER_CONFIRM`, `COMPLETED`, `FAILED`)
   - `langgraph_thread_id`: String (Dùng để resume state khi User bấm Confirm)
5. **Bảng `ticket_messages`**:
   - `id`: UUID (PK)
   - `ticket_id`: UUID (FK -> `tickets.id`)
   - `sender_type`: Enum (`USER`, `AGENT`, `SYSTEM`)
   - `sender_name`: String (VD: `User`, `@ba`, `@developer`)
   - `event_type`: String (`message`, `tool_call`, `diff`, `approval_request`)
   - `content`: Text
   - `metadata`: JSONB
   - `created_at`: DateTime

4. Đặc tả REST Endpoints

- `POST /api/agents`, `GET /api/agents`, `PUT /api/agents/{id}`, `DELETE /api/agents/{id}`
- `POST /api/agents/{id}/test-connection`: Spawn thử ACP binary với lệnh `initialize` để kiểm tra CLI đã login chưa.
- `POST /api/teams`, `GET /api/teams`
- `POST /api/projects`, `GET /api/projects/{id}`
  - Khi tạo/cập nhật Project, kiểm tra `os.path.exists(workspace_path)` và kiểm tra thư mục `.git` có hợp lệ không.
- `PUT /api/projects/{id}/workflow`: Lưu JSON đồ thị kéo thả từ Frontend.
- `POST /api/projects/{id}/seed-default-bugfix-team`: API tiện ích tự động tạo sẵn 4 agent (`@ba`, `@developer`, `@tester`, `@leader`) với System Prompt chuẩn và gắn vào Project.

5. Tiêu chí nghiệm thu

- Chạy migration Alembic tạo đầy đủ bảng.
- Gọi API `/seed-default-bugfix-team` tạo ra đúng 4 Agent và cấu hình mặc định cho thư mục `D:\VETC\FE\HNBG_DEV`.

File 3: `PHASE_3_WORKFLOW_ENGINE.md` (Dynamic LangGraph Compiler & WebSocket)

```markdown
# PHASE 3: Bộ biên dịch Dynamic Workflow (LangGraph) & Real-time WebSocket

## 1. Mục tiêu
Biên dịch đồ thị JSON (Nodes & Edges do người dùng kéo thả trên giao diện giống Dify) thành một `StateGraph` thực thi đư

## 2. Cấu trúc thư mục cần tạo
```text
backend/
├── app/
│   ├── engine/
```
```

```
│   │   ├── state.py          # Định nghĩa WorkflowState của LangGraph
│   │   ├── nodes.py          # Hàm thực thi từng loại Node (Agent, Condition, HumanApproval)
│   │   └── compiler.py       # Chuyển đổi ReactFlow JSON -> Compiled LangGraph
│   └── ws/
│       └── ticket_stream.py  # WebSocket endpoint cho phòng chat Ticket
```

3. Đặc tả chi tiết cho Codex thực thi

3.1. Định nghĩa State (`app/engine/state.py`)

```
from typing import TypedDict, List, Optional, Dict, Any

class BugFixWorkflowState(TypedDict):
    ticket_id: str
    workspace_path: str
    bug_description: str
    ba_reproduce_steps: Optional[str]
    needs_user_clarification: bool
    dev_summary: Optional[str]
    modified_files: List[str]
    tester_report: Optional[str]
    test_passed: bool
    user_confirmed_fix: bool
    git_commit_hash: Optional[str]
    history: List[Dict[str, Any]]
```

3.2. Các loại Node chuẩn trong Workflow (`app/engine/nodes.py`)

1. **`agent_node_factory(agent_config)`**:
    - Khởi tạo `ACPSubprocessClient` với runtime của agent đó (`claude-agent-acp` hoặc `codex-acp`).
    - Xây dựng prompt đầu vào dựa trên `BugFixWorkflowState` hiện tại:
        - Nếu là `BA`: Nhận `bug_description` + lịch sử trả lời của User. Nếu thông tin chưa đủ tái tạo lỗi, yêu cầu BA xuất tag `[NEEDS_CLARIFICATION]` để dừng chờ User trả lời. Nếu đủ, xuất `[STR_READY]`.
        - Nếu là `DEVELOPER`: Nhận `ba_reproduce_steps`, sửa code trực tiếp trên `workspace_path`.
        - Nếu là `TESTER`: Nhận `ba_reproduce_steps` + `dev_summary`. Nếu test lỗi, xuất `[TEST_FAILED]` để quay lại node `DEVELOPER`. Nếu pass, xuất `[TEST_PASSED]`.
        - Nếu là `LEADER`: Chỉ chạy khi `user_confirmed_fix == True`. Thực thi `git status`, `git diff`, `git add`, `git commit -m "fix: ..."` và `git push`.
    - Trong lúc `stream_prompt()` chạy, lưu từng message vào DB và phát qua WebSocket của `ticket_id`.
2. **`human_gate_node`**:
    - Sử dụng `langgraph.types.interrupt()` để tạm dừng đồ thị tại:
        - Điểm 1: Khi `BA` cần hỏi thêm thông tin tái tạo lỗi từ User.
        - Điểm 2: Sau khi `TESTER` báo `[TEST_PASSED]`, chờ User kiểm tra thực tế và bấm nút **"Xác nhận & Push Git"** (hoặc gõ `@leader CONFIRM`).

3.3. Bộ biên dịch JSON sang LangGraph (`app/engine/compiler.py`)

- Nhận đầu vào là `workflow_graph` (gồm mảng `nodes` và `edges` từ React Flow).
- Duyệt mảng `nodes` để gọi `workflow.add_node(node_id, runner)`.
- Duyệt mảng `edges`:
    - Nếu là cạnh thường: `workflow.add_edge(source, target)`.
    - Nếu là cạnh điều kiện (ví dụ từ Tester rẽ nhánh: `Pass -> User Confirm`, `Fail -> Developer`): dùng `workflow.add_conditional_edges()`.
- Compile graph với `AsyncPostgresSaver` (hoặc `MemorySaver`) để lưu checkpoint theo `ticket_id`.

3.4. WebSocket Chat & Control (`app/ws/ticket_stream.py`)

- Endpoint: `WS /ws/tickets/{ticket_id}`
- Hỗ trợ nhận lệnh từ Frontend gửi lên:

- `{"action": "start_workflow"}`: Bắt đầu chạy ticket.
- `{"action": "user_reply", "content": "..."}`: User trả lời câu hỏi của `@ba`, resume LangGraph.
- `{"action": "user_confirm_fix", "approved": true | false, "feedback": "..."}`:
  - Nếu `approved == true`: Resume LangGraph với `user_confirmed_fix=True` để chạy sang node `@leader`.
  - Nếu `approved == false`: Resume LangGraph quay về node `@developer` kèm feedback của User.

4. Tiêu chí nghiệm thu

- Gửi 1 JSON workflow gồm `Start -> @ba -> @developer -> @tester -> HumanApproval -> @leader -> End`.
- Đồ thị tự động dừng lại ở bước `HumanApproval`, sau khi bắn gói tin `{"action": "user_confirm_fix", "approved": true}` thì `@leader` tự động chạy lệnh `git commit` và `git push`.

File 4: `PHASE_4_FRONTEND_BUILDER.md` (Next.js UI: Agents, Projects & Workflow Canvas)

```markdown
# PHASE 4: Xây dựng Giao diện Quản lý & Kéo thả Workflow (Next.js + React Flow + shadcn/ui)

## 1. Mục tiêu
Xây dựng giao diện quản trị trên Next.js (App Router) cho phép cấu hình Agent, tạo Team, thiết lập Project trỏ vào thư m

## 2. Công nghệ & Thư viện cần cài đặt
```bash
npx create-next-app@latest frontend --typescript --tailwind --eslint --app
cd frontend
npx shadcn@latest init
npx shadcn@latest add button card dialog input textarea badge tabs select scroll-area toast
npm install @xyflow/react lucide-react zustand @tanstack/react-query
```

3. Cấu trúc trang cần xây dựng

```
frontend/src/
├── app/
│   ├── agents/page.tsx               # Quản lý Agent & Test kết nối ACP
│   ├── teams/page.tsx                # Gom nhóm Agent thành Team
│   ├── projects/
│   │   ├── page.tsx                  # Danh sách Project
│   │   └── [id]/
│   │       ├── page.tsx              # Tổng quan Project & Danh sách Tickets
│   │       ├── members/page.tsx      # Thêm User, Agent, Team vào Project
│   │       └── workflow/page.tsx     # Canvas kéo thả Workflow giống Dify
└── components/
    └── workflow/
        ├── WorkflowCanvas.tsx
        ├── SidebarPalette.tsx
        └── nodes/
            ├── StartNode.tsx
            ├── AgentNode.tsx         # Node đại diện cho @ba, @dev, @tester, @leader
            ├── ConditionNode.tsx     # Node rẽ nhánh (Test Pass / Test Fail)
            └── HumanConfirmNode.tsx  # Node chờ User xác nhận trước khi Push Git
```

4. Đặc tả chi tiết tính năng

4.1. Trang Quản lý Agent (`/agents`)

- Hiển thị danh sách Agent dưới dạng Card (`@ba`, `@developer`, `@tester`, `@leader`).
- Modal Tạo/Sửa Agent:
  - Chọn Runtime: `Claude Code (ACP)` hoặc `OpenAI Codex (ACP)`.
  - Chọn Role: `BA (Read-only)`, `Developer (Read/Write, No Commit)`, `Tester (Read/Test)`, `Leader (Git Release)`.
  - Textarea chỉnh sửa System Prompt.
  - Nút **"Test CLI Connection"**: Gọi API kiểm tra xem máy local đã đăng nhập Claude/Codex chưa.
```

4.2. Trang Workflow Editor (`/projects/[id]/workflow`)

Sử dụng `@xyflow/react`:

- **Thanh công cụ bên trái (`SidebarPalette.tsx`)**:
  - Cho phép kéo thả các khối: `Start Trigger`, `Agent Node`, `Condition (Pass/Fail)`, `User Confirmation Gate`, `End Node`.
- **Thiết kế `AgentNode.tsx`**:
  - Cho phép chọn Agent từ danh sách thành viên của Project (`@ba`, `@developer`, `@tester`, `@leader`).
  - Hiển thị rõ Badge quyền hạn (VD: `@developer` có badge `Read/Write`, `@leader` có badge `Git Push`).
- **Nút "Load Default Bugfix Template"**:
  - Tự động dựng sẵn sơ đồ chuẩn trên Canvas: `[Start] -> [@ba] -> (Đủ thông tin?) -> [@developer] -> [@tester] -> (Pass?) -> [User Confirm Gate] -> [@leader (Git Push)] -> [End]`.
- **Nút "Save Workflow"**: Xuất `nodes` và `edges` ra JSON và gọi `PUT /api/projects/{id}/workflow`.

5. Tiêu chí nghiệm thu

- Kéo thả được các Node trên giao diện, nối dây giữa `@ba -> @developer -> @tester -> User Confirm -> @leader`, bấm Save và tải lại trang vẫn giữ nguyên sơ đồ.

File 5: `PHASE_5_TICKET_CHAT_RUNTIME.md` (Giao diện Chat Ticket & Live Execution)

```markdown
# PHASE 5: Giao diện Chạy Ticket dạng Phòng Chat Multi-Agent (Giống Buzz Desktop)

## 1. Mục tiêu
Xây dựng màn hình chi tiết Ticket (`/projects/[id]/tickets/[ticketId]`), nơi người dùng báo lỗi và quan sát các Agent (`

## 2. Cấu trúc Component cần tạo
```text
frontend/src/
├── app/projects/[id]/tickets/[ticketId]/page.tsx
└── components/ticket-chat/
    ├── ChatTimeline.tsx          # Luồng tin nhắn giữa User và các Agent
    ├── AgentMessageBubble.tsx    # Hiển thị nội dung chat + khối thu gọn (Thinking / Tool Call / File Diff)
    ├── ApprovalBanner.tsx        # Hộp thoại nổi khi chờ User xác nhận bug đã fix xong
    ├── LiveWorkflowMiniMap.tsx   # Sơ đồ thu nhỏ bên phải, sáng đèn ở Node đang chạy
    └── TerminalDrawer.tsx        # Khung xem live stdout/git log của ACP process
```

3. Đặc tả chi tiết giao diện và luồng tương tác

3.1. Bố cục màn hình Ticket (Split View 2 cột)

- **Cột trái (70% diện tích) - Phòng Chat Multi-Agent**:
  - Header: Tiêu đề Ticket, đường dẫn Workspace (`D:\VETC\FE\HNBG_DEV`), nhánh Git hiện tại, trạng thái Ticket.
  - Body (`ChatTimeline.tsx`):
    - Tin nhắn của **User**: Hiển thị bên phải.
    - Tin nhắn của **Agent** (`@ba`, `@developer`, `@tester`, `@leader`): Hiển thị bên trái với Avatar, màu sắc riêng theo từng Role.
    - Bên trong mỗi tin nhắn của Agent có các **Collapsible Accordion** (khối đóng/mở):
      - 🔍 `Đang đọc file...` (Danh sách các file Agent vừa truy cập qua ACP).
      - 🛠️ `Thay đổi mã nguồn (Diff)` (Hiển thị diff code mà `@developer` vừa sửa).
      - 💻 `Lệnh Terminal đã chạy` (Kết quả chạy test của `@tester` hoặc lệnh `git commit/push` của `@leader`).
  - Footer:
    - Khi `@ba` hỏi lại thông tin: Mở ô nhập chat để User trả lời chi tiết các bước tái hiện lỗi.
    - Khi hệ thống dừng ở bước chờ xác nhận nghiệm thu: Hiển thị component `ApprovalBanner.tsx`.
```

- **Cột phải (30% diện tích) - Live Workflow Tracker**:
    - Render đồ thị React Flow ở chế độ Read-only.
    - Khi Backend bắn sự kiện Agent nào đang chạy, Node của Agent đó trên sơ đồ có viền sáng nhấp nháy (`animate-pulse`) để người dùng biết quy trình đang ở bước nào.

3.2. Component `ApprovalBanner.tsx` (Human-in-the-loop Gate)

Khi nhận được WebSocket event `status == "WAITING_USER_CONFIRM"` sau khi `@tester` hoàn tất:

- Hiển thị một Card nổi bật ngay trên thanh chat với nội dung:

  @tester đã xác nhận bản vá lỗi vượt qua bài kiểm tra.

  Vui lòng kiểm tra lại ứng dụng trên môi trường của bạn (`D:\VETC\FE\HNBG_DEV`). Nếu lỗi đã được khắc phục hoàn toàn, hãy xác nhận để **@leader** thực hiện commit & push lên Git.

- Có 2 nút hành động:
    1. Nút xanh **"Xác nhận đã xong (Giao @leader Commit & Push)"**:
        - Gửi WebSocket: `{"action": "user_confirm_fix", "approved": true}`.
        - Lập tức kích hoạt `@leader` kiểm tra `git diff`, chạy `git add`, `git commit`, `git push` và in Commit Hash ra khung chat.
    2. Nút đỏ **"Vẫn còn lỗi (Yêu cầu @developer sửa lại)"**:
        - Mở ô nhập lý do lỗi chưa hết -> Gửi `{"action": "user_confirm_fix", "approved": false, "feedback": "..."}` để luồng tự động lùi về Node `@developer`.

4. Tiêu chí nghiệm thu toàn hệ thống (End-to-End Test)

1. Người dùng tạo Ticket mới trong Project `HNBG_DEV`: *"Nút xuất báo cáo bị đơ khi chọn khoảng thời gian lớn"*.
2. `@ba` phân tích, hỏi lại User 1 câu làm rõ -> User trả lời trên khung chat -> `@ba` xuất kịch bản tái tạo lỗi (STR) step-by-step.
3. Workflow tự chuyển sang `@developer` -> `@developer` sửa file trong `D:\VETC\FE\HNBG_DEV` (không commit).
4. Workflow tự chuyển sang `@tester` -> `@tester` kiểm tra code và báo Pass.
5. `ApprovalBanner` hiện lên -> Người dùng bấm **"Xác nhận đã xong"**.
6. `@leader` thức dậy, chạy `git commit` và `git push` thành công, trả mã Commit Hash vào khung chat và đổi trạng thái Ticket sang `COMPLETED`.

```
---

### Cách giao việc hiệu quả nhất cho Codex / Claude Code:

1. Tạo thư mục dự án mới, lưu 5 nội dung trên vào thư mục `docs/specs/`.
2. Chạy lệnh với từng phase:
   ```text
   Hãy đọc file docs/specs/PHASE_1_ACP_BRIDGE.md và triển khai đầy đủ mã nguồn cùng unit test cho Phase 1. Không chuyển
```

3. Lặp lại tuần tự từ Phase 1 đến Phase 5 để hoàn thiện sản phẩm.

User prompt: Ngoài 4 agent mặc định cho phép tạo thêm agent tên vai trò mô tả chi tiết vai trò loại agent (clade code/codex) chọn model

Response: Dưới đây là file đặc tả chi tiết **`CUSTOM_AGENT_EXTENSION.md`** để bạn đưa cho Codex/Claude Code triển khai tính năng **Tạo Agent Tùy Chỉnh (Custom Agent)** linh hoạt, không bị giới hạn ở 4 role mặc định.

`CUSTOM_AGENT_EXTENSION.md`

```
# PHẦN MỞ RỘNG: Cho phép Tạo và Quản lý Custom Agent Linh Hoạt

## 1. Mục tiêu
Mở rộng hệ thống để người dùng có thể tự tạo không giới hạn số lượng Agent với các thuộc tính tùy chỉnh:
- **Tên agent (Name):** Ví dụ `@security_checker`, `@database_specialist`, `@ui_designer`, `@devops`...
- **Vai trò (Role):** Tùy chỉnh (custom string) hoặc ánh xạ theo nhóm quyền hạn.
- **Mô tả chi tiết vai trò (Detailed Description / System Prompt):** Prompt định hình nhân cách, trách nhiệm, phạm vi cô
- **Loại Agent / Runtime:** `Claude Code (claude-agent-acp)` hoặc `OpenAI Codex (codex-acp)`.
- **Chọn Model (Model Selection):** Danh sách model tương ứng theo từng runtime (ví dụ: `claude-3-7-sonnet`, `claude-3-5

Các Custom Agent này có thể:
```
```

1. Được lưu trữ trong Database.
2. Được kéo thả vào **Workflow Canvas** (React Flow) như một Node độc lập.
3. Tham gia vào phòng chat của Ticket theo luồng điều phối.

---

2. Cập nhật Cơ sở Dữ liệu & Schemas

2.1. Cập nhật Model SQLAlchemy (`backend/app/db/models.py`)
Điều chỉnh bảng `agents` từ việc dùng Enum cứng sang hỗ trợ Role linh hoạt kèm Permission Profile:

```python
import uuid
from sqlalchemy import Column, String, Text, Boolean, JSON, DateTime, func
from sqlalchemy.dialects.postgresql import UUID
from app.db.session import Base

class Agent(Base):
    __tablename__ = "agents"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    name = Column(String(50), nullable=False, unique=True) # Ví dụ: @security_checker
    display_name = Column(String(100), nullable=False)     # Ví dụ: Security Audit Specialist
    role_key = Column(String(50), nullable=False)          # Key định danh (VD: SECURITY, DEVOPS, hoặc CUSTOM)
    description = Column(Text, nullable=True)              # Mô tả ngắn gọn về vai trò
    system_prompt = Column(Text, nullable=False)           # Mô tả chi tiết vai trò & chỉ dẫn hành vi

    # Runtime & Model
    runtime = Column(String(30), nullable=False)           # "claude-code" hoặc "codex"
    model = Column(String(100), nullable=False)            # "claude-3-7-sonnet", "gpt-4o",...

    # Cấu hình quyền hạn (Permission Profile)
    permission_profile = Column(JSON, default=lambda: {
        "allow_read": True,
        "allow_write": False,
        "allow_terminal": False,
        "blocked_commands": ["git commit", "git push", "rm -rf"]
    })

    is_builtin = Column(Boolean, default=False)            # True nếu là 4 agent mặc định (BA, Dev, Tester, Leader)
    created_at = Column(DateTime, server_default=func.now())
    updated_at = Column(DateTime, onupdate=func.now())
```

2.2. Pydantic Schemas (`backend/app/schemas/agent.py`)

```python
from pydantic import BaseModel, Field
from typing import Optional, Dict, Any, List
from uuid import UUID

class AgentBase(BaseModel):
    name: str = Field(..., pattern=r"^@[a-zA-Z0-9_-]+$", example="@security_audit")
    display_name: str = Field(..., example="Security Auditor")
    role_key: str = Field(default="CUSTOM", example="SECURITY")
    description: Optional[str] = Field(None, example="Chuyên rà soát lỗ hổng bảo mật và dependency")
    system_prompt: str = Field(..., example="Bạn là chuyên gia bảo mật. Quét mã nguồn tìm SQL Injection...")
    runtime: str = Field(..., regex="^(claude-code|codex)$")
    model: str = Field(..., example="claude-3-7-sonnet")
    permission_profile: Optional[Dict[str, Any]] = Field(default_factory=lambda: {
        "allow_read": True,
        "allow_write": False,
        "allow_terminal": False,
        "blocked_commands": ["git commit", "git push"]
    })

class AgentCreate(AgentBase):
    pass

class AgentUpdate(BaseModel):
    display_name: Optional[str] = None
    description: Optional[str] = None
    system_prompt: Optional[str] = None
    runtime: Optional[str] = None
    model: Optional[str] = None
    permission_profile: Optional[Dict[str, Any]] = None
```

```
class AgentResponse(AgentBase):
    id: UUID
    is_builtin: bool

    class Config:
        from_attributes = True
```

3. Cập nhật Logic ACP Subprocess Bridge (`backend/app/acp/`)

Khi người dùng tạo một Agent bất kỳ với Runtime và Model chỉ định:

1. **Truyền Model vào ACP Adapter:** Khi gọi `session/new` hoặc `session/prompt`, truyền thông số model vào payload JSON-RPC:

   ```
   {
     "jsonrpc": "2.0",
     "method": "session/prompt",
     "params": {
       "model": "<agent.model>",
       "prompt": [{"type": "text", "text": "<user_or_workflow_prompt>"}],
       "_meta": {
         "systemPrompt": {"append": "<agent.system_prompt>"}
       }
     }
   }
   ```

2. **Kiểm tra quyền động (`DynamicPermissionGuard`):** Thay vì hard-code theo 4 role cố định, đọc trực tiếp từ `agent.permission_profile`:
   - Nếu `allow_write == False`: Chặn các request dạng `fs/writeTextFile`.
   - Nếu lệnh shell nằm trong danh sách `blocked_commands`: Tự động gửi phản hồi `deny`.

4. Xây dựng Giao diện Tạo Custom Agent (Next.js + shadcn/ui)

Tạo component Modal Tạo Agent: `frontend/src/components/agents/CreateAgentDialog.tsx`.

4.1. Metadata danh sách Model hỗ trợ theo từng Runtime

```
export const RUNTIME_MODELS = {
  "claude-code": [
    { label: "Claude 3.7 Sonnet (Mới nhất, Coding mạnh)", value: "claude-3-7-sonnet-latest" },
    { label: "Claude 3.5 Sonnet (Cân bằng, Ổn định)", value: "claude-3-5-sonnet-latest" },
    { label: "Claude 3.5 Haiku (Rất nhanh, Tối ưu chi phí/tải)", value: "claude-3-5-haiku-latest" },
  ],
  "codex": [
    { label: "GPT-4o (Khuyên dùng cho Coding)", value: "gpt-4o" },
    { label: "GPT-4o-mini (Nhanh, Nhẹ)", value: "gpt-4o-mini" },
    { label: "o3-mini (Suy luận logic cao)", value: "o3-mini" },
    { label: "o1-preview (Deep reasoning)", value: "o1-preview" },
  ]
};
```

4.2. Form Giao diện (`CreateAgentDialog.tsx`)

Bao gồm các trường nhập liệu tương ứng:

1. **Tên Agent (Name):**
   - Ô Input có tiền tố `@` (ví dụ: `security_expert` -> `@security_expert`).
2. **Tên hiển thị & Vai trò (Role):**
   - Input: Display Name (ví dụ: *Chuyên viên Kiểm thử Hiệu năng*).
   - Select hoặc Input tự do: Nhóm vai trò (BA, Developer, Tester, Security, DevOps, Custom).
3. **Loại Agent (Runtime Selection):**
   - Radio Group hoặc Select:
     - Option 1: **Claude Code** (`claude-agent-acp`)
     - Option 2: **OpenAI Codex** (`codex-acp`)
4. **Chọn Model (Model Selector):**

- Dropdown tự động thay đổi danh sách Options dựa trên Runtime đã chọn ở bước 3.

5. **Mô tả chi tiết vai trò (System Prompt):**
   - Textarea lớn với placeholder hướng dẫn rõ ràng:

     "Mô tả chi tiết nhiệm vụ của Agent, các nguyên tắc bắt buộc khi duyệt mã nguồn, định dạng đầu ra mong muốn..."
6. **Cấu hình Quyền hạn (Permissions - Tùy chọn nâng cao):**
   - Checkbox: Cho phép đọc file (`allow_read`).
   - Checkbox: Cho phép sửa code (`allow_write`).
   - Checkbox: Cho phép chạy lệnh Terminal (`allow_terminal`).
   - Input: Các lệnh bị cấm (Mặc định: `git commit`, `git push`, `rm -rf`).

5. Tích hợp Custom Agent vào Workflow Canvas & Ticket Chat

5.1. Trên Workflow Canvas (`@xyflow/react`)

- Trong thanh bên trái (Sidebar Palette), danh sách Agent không còn cố định mà được query động từ API `GET /api/agents`.
- Người dùng có thể kéo thả bất kỳ Custom Agent nào vào đồ thị:
  - Ví dụ luồng: `[Start] -> [@ba] -> [@developer] -> [@security_checker] -> [@tester] -> [User Confirm] -> [@leader] -> [End]`.
- Cấu hình từng Node trên canvas cho phép override prompt hoặc bổ sung biến context đầu vào.

5.2. Trong Ticket Chat

- Khi workflow chạy đến Node của Custom Agent:
  - Avatar hiển thị theo icon/màu sắc của agent đó.
  - Tên hiển thị đầy đủ (VD: `@security_checker [Claude 3.7 Sonnet]`).
  - Toàn bộ suy nghĩ (Thinking), quá trình đọc/sửa file và câu trả lời đều được stream qua WebSocket vào đúng luồng hội thoại.

6. Tiêu chí nghiệm thu (Definition of Done)

1. Bấm nút **"Create Agent"** trên trang `/agents`, tạo thành công một agent mới tên `@security_checker`, chọn runtime `codex`, model `gpt-4o-mini`, nhập mô tả vai trò và lưu vào Database.
2. Mở Workflow Editor của Project `HNBG_DEV`, thấy `@security_checker` xuất hiện trong danh sách kéo thả.
3. Nối `@security_checker` vào giữa `@developer` và `@tester`, lưu Workflow thành công.
4. Chạy một Ticket kiểm thử: Sau khi `@developer` sửa code xong, `@security_checker` tự động thức dậy, chạy review bảo mật trên các file vừa sửa và gửi đánh giá vào khung chat trước khi chuyển giao cho `@tester`.

```
---

### Tóm tắt luồng hoạt động sau khi bổ sung:

* Bạn có thể tự tạo bất kỳ role nào bạn muốn: từ **BA, Dev, Tester, Leader** mặc định đến **DBA, UI Reviewer, DevOps, Se
* Mỗi agent được tự do ghép đôi runtime và model (ví dụ: Dev dùng Claude 3.7 Sonnet, Reviewer dùng GPT-4o, Tester dùng C
* Mọi Agent tạo ra đều tuân thủ cơ chế ACP an toàn và cắm ghép trực tiếp được vào đồ thị kéo thả.
```

User prompt: Https://github.com/decolua/9remote thiết kế claude code ui hoặc codex ui bằng cách nào

Response: Dự án **`decolua/9remote`** (cùng các dự án Web UI điều khiển CLI agent từ xa như `claudecodeui`, `claude-code-web`) được thiết kế để đưa giao diện dòng lệnh của **Claude Code** và **OpenAI Codex CLI** lên trình duyệt Web/Mobile.

Thay vì dùng giao thức ACP như Buzz Desktop, các hệ thống dạng `9remote` chủ yếu sử dụng **2 kiến trúc cốt lõi** dưới đây để dựng giao diện UI:

Kiến trúc 1: NDJSON Stream Parser (Biến CLI thành Chat UI hiện đại)

Đây là cách biến đầu ra của Claude Code / Codex thành các **bong bóng chat (Message Bubbles), khối thu gọn (Thinking Accordion) và bảng Diff code** giống hệt ứng dụng chuyên nghiệp.

1. Cơ chế ở Backend (Subprocess Stream-JSON)

Thay vì mở giao diện TUI (Terminal User Interface) mặc định của `claude`, backend khởi chạy CLI ở chế độ **Headless Streaming JSON**:

```
# Lệnh thực tế được spawn ở backend cho Claude Code
claude -p "Nội dung prompt" \
  --output-format stream-json \
  --verbose \
  --include-partial-messages
```

Khi chạy với cờ `--output-format stream-json`, CLI sẽ xuất ra từng dòng JSON (NDJSON - Newline Delimited JSON) qua `stdout` theo thời gian thực cho mọi hành động:

- `{"type": "system", "subtype": "init", "session_id": "..."}` -> Lấy `session_id` để giữ mạch hội thoại cho câu tiếp theo (dùng cờ `--resume <session_id>`).
- `{"type": "assistant", "message": {"content": [{"type": "thinking", "thinking": "..."}]}}` -> Quá trình suy nghĩ của AI.
- `{"type": "assistant", "message": {"content": [{"type": "tool_use", "name": "Edit", "input": {...}}]}}` -> AI đang sửa file hoặc chạy lệnh Bash.
- `{"type": "user", "message": {"content": [{"type": "tool_result", ...}]}}` -> Kết quả trả về từ terminal/file.

2. Cơ chế ở Frontend (Next.js / React UI Mapping)

Backend đẩy thẳng từng dòng JSON này qua **WebSocket** hoặc **SSE (Server-Sent Events)** lên trình duyệt. Giao diện React sẽ ánh xạ (map) từng `type` thành Component UI:

Loại JSON Event từ CLI	Component hiển thị trên Web UI (shadcn/ui)
`content.type === "thinking"`	**`<Collapsible>`**: Khung xám thu gọn *"Đang suy nghĩ..."* có hiệu ứng shimmer.
`tool_use.name === "Read" / "Grep"`	**`<Badge>`**: Icon kính lúp kèm tên file vừa đọc (🔍 `Đã đọc src/index.tsx`).
`tool_use.name === "Edit" / "Write"`	**`<DiffViewer>`** (dùng `monaco-editor` hoặc `react-diff-viewer`): Hiển thị trực tiếp đoạn code xanh/đỏ vừa thêm/xóa.
`tool_use.name === "Bash"`	**`<TerminalCard>`**: Khung đen hiển thị lệnh shell đã chạy và kết quả stdout bên dưới.
`content.type === "text"`	**`<MarkdownRenderer>`**: Bong bóng chat phản hồi chính thức của Agent.

Kiến trúc 2: PTY + Xterm.js Bridge (Bê nguyên Terminal TUI lên Web)

Với các tính năng tương tác trực tiếp của Claude Code / Codex (như bấm phím mũi tên chọn `Yes/No` khi hỏi quyền, gõ lệnh `/compact`, `/login`, `/init`), nhiều công cụ remote sử dụng cơ chế **Pseudo-Terminal (PTY)**:

1. Cơ chế hoạt động

1. **Backend (`node-pty` hoặc Python `ptyprocess`):** Tạo một phiên terminal ảo trên máy chủ tại thư mục dự án (`D:\VETC\FE\HNBG_DEV`), chạy lệnh `claude` hoặc `codex`.
2. **WebSocket Pipe:** Nối trực tiếp luồng byte của PTY (`onData`) qua WebSocket tới trình duyệt.
3. **Frontend (`@xterm/xterm`):** Nhúng component `Xterm.js` vào một tab trên giao diện Web. Mọi mã màu ANSI, khung viền TUI của Claude Code và Codex đều hiển thị y hệt như đang ngồi trước màn hình máy tính.

`[Trình duyệt: Xterm.js UI] ◄──(WebSocket raw bytes)──► [Backend: node-pty / Python pty] ◄──► [claude / codex CLI]`

Cách kết hợp cả hai để thiết kế UI tối ưu nhất cho dự án của bạn

Khi bạn tự thiết kế hệ thống **Python + Next.js** (như trong 5 Phase ở trên), cách làm tốt nhất học từ `9remote` và `Buzz` là **kết hợp cả hai chế độ trên cùng một màn hình Ticket**:

1. **Tab 1 - "Chat & Workflow View" (Chế độ структурированный JSON / ACP):**
   - Dùng **`claude-agent-acp`** / **`codex-acp`** (hoặc `claude --output-format stream-json`).
   - Render ra giao diện Chat sạch sẽ giữa `@ba`, `@developer`, `@tester`, `@leader`.
   - Có các nút bấm UI rõ ràng (Approve / Reject / Diff File Viewer) để người không rành terminal (như BA, Tester) dễ sử dụng.
2. **Tab 2 - "Live Remote Terminal" (Chế độ PTY + Xterm.js giống 9remote):**
   - Dành cho Developer hoặc Leader khi muốn can thiệp sâu từ xa qua Web/điện thoại.

- Mở một drawer bên dưới sử dụng `@xterm/xterm` kết nối qua WebSocket tới `pty` trên server để gõ trực tiếp lệnh `claude`, `/login`, hoặc kiểm tra `git log` trực tiếp trên thư mục `D:\VETC\FE\HNBG_DEV`.

Đoạn code mẫu Parse Stream-JSON của Claude Code trong Python (nếu không dùng ACP)

Nếu muốn lấy cảm hứng trực tiếp từ cách parse stream đơn giản mà không cần qua lớp ACP:

```
import asyncio
import json

async def run_claude_stream_ui(prompt: str, workspace_dir: str, session_id: str = None):
    cmd = [
        "claude", "-p", prompt,
        "--output-format", "stream-json",
        "--verbose"
    ]
    if session_id:
        cmd.extend(["--resume", session_id])

    process = await asyncio.create_subprocess_exec(
        *cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE,
        cwd=workspace_dir
    )

    async for line in process.stdout:
        text = line.decode("utf-8").strip()
        if not text:
            continue
        try:
            event = json.loads(text)
            # Đẩy trực tiếp JSON event này qua WebSocket lên Next.js để vẽ UI
            yield event
        except json.JSONDecodeError:
            continue
```

Tóm lại:

- Nếu muốn **tương tác gõ phím trực tiếp từ xa** vào phiên CLI đang mở sẵn trên máy tính -> Dùng **PTY + Xterm.js** (kiểu remote terminal của `9remote`).
- Nếu muốn **dựng giao diện Chat nhiều Agent (@ba, @dev, @tester, @leader) tự động bàn giao việc theo Workflow** -> Dùng **ACP Adapter (`claude-agent-acp` / `codex-acp`)** hoặc **`--output-format stream-json`** để bóc tách từng sự kiện và vẽ bằng component **shadcn/ui**.

User prompt: xuất plan ra md file

Response: Toàn bộ kế hoạch triển khai hệ thống điều phối Multi-Agent (kết hợp kiến trúc ACP của **Buzz**, luồng kéo thả của **Dify** và giao diện Streaming/PTY của **9remote**) đã được tổng hợp thành một file Markdown duy nhất dưới đây.

Bạn có thể sao chép toàn bộ khối bên dưới và lưu thành file `IMPLEMENTATION_PLAN.md` tại thư mục gốc của dự án để giao trực tiếp cho **Codex** hoặc **Claude Code** thực thi.

```
# MASTER IMPLEMENTATION PLAN: MULTI-AGENT WORKFLOW PLATFORM (BUZZ + DIFY + 9REMOTE CLONE)

## 1. Tổng quan Kiến trúc Hệ thống (System Architecture)

Nền tảng điều phối đội ngũ AI Agent làm việc trực tiếp trên thư mục mã nguồn local (ví dụ: `D:\VETC\FE\HNBG_DEV`), kết h
1. **ACP Bridge & NDJSON Stream:** Giao tiếp với `claude-agent-acp` (Claude Code) và `codex-acp` (OpenAI Codex) qua JSON
2. **Dynamic Workflow Engine:** Sử dụng **LangGraph** để biên dịch sơ đồ kéo thả (React Flow) thành State Machine hỗ trợ
3. **Custom Agent & RBAC Guard:** Cho phép dùng 4 Agent mặc định (`@ba`, `@developer`, `@tester`, `@leader`) hoặc tự tạo
4. **Hybrid Web UI (Next.js + shadcn/ui):** Giao diện phòng chat Multi-Agent hiển thị rõ khối suy nghĩ (Thinking), bảng

---

## 2. Cấu trúc Thư mục Dự án (Monorepo Structure)

```text
multi-agent-platform/
├── backend/
│   ├── app/
│   │   ├── acp/
```
```

```
│   │   │   ├── __init__.py
│   │   │   ├── types.py          # Pydantic schemas cho JSON-RPC 2.0 & ACP events
│   │   │   ├── guard.py          # DynamicPermissionGuard chặn lệnh theo Permission Profile
│   │   │   ├── client.py         # ACPSubprocessClient giao tiếp với claude-agent-acp / codex-acp
│   │   │   └── pty_manager.py    # Quản lý phiên Pseudo-Terminal cho tab Xterm.js
│   │   ├── db/
│   │   │   ├── session.py        # Kết nối Async PostgreSQL / SQLite
│   │   │   └── models.py         # ORM Models: Agent, Team, Project, Workflow, Ticket, Message
│   │   ├── schemas/              # Pydantic Request/Response models
│   │   ├── engine/
│   │   │   ├── state.py          # BugFixWorkflowState cho LangGraph
│   │   │   ├── nodes.py          # Node runners cho Agent, Condition, HumanApproval
│   │   │   └── compiler.py       # Biên dịch JSON React Flow -> Compiled LangGraph
│   │   ├── api/
│   │   │   ├── agents.py         # CRUD Agent & Test CLI Connection
│   │   │   ├── teams.py          # CRUD Team
│   │   │   ├── projects.py       # CRUD Project, Members, Save Workflow, Seed Default Team
│   │   │   └── tickets.py        # CRUD Ticket & Trigger Workflow
│   │   ├── ws/
│   │   │   ├── ticket_stream.py  # WebSocket phòng chat Ticket & Human Approval
│   │   │   └── terminal_ws.py    # WebSocket stream raw bytes cho Xterm.js
│   │   └── main.py               # Entrypoint FastAPI
│   ├── alembic/
│   ├── requirements.txt
│   └── tests/
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── agents/page.tsx                     # Quản lý & tạo mới Custom Agent
│   │   │   ├── teams/page.tsx                      # Quản lý Team
│   │   │   └── projects/
│   │   │       ├── page.tsx                        # Danh sách Project
│   │   │       └── [id]/
│   │   │           ├── page.tsx                    # Danh sách Ticket trong Project
│   │   │           ├── members/page.tsx            # Thêm User / Agent / Team vào Project
│   │   │           ├── workflow/page.tsx           # Trình kéo thả Workflow (React Flow)
│   │   │           └── tickets/[ticketId]/page.tsx # Phòng Chat Ticket + Live Graph + Terminal
│   │   └── components/
│   │       ├── agents/CreateAgentDialog.tsx
│   │       ├── workflow/
│   │       │   ├── WorkflowCanvas.tsx
│   │       │   ├── SidebarPalette.tsx
│   │       │   └── nodes/ (StartNode, AgentNode, ConditionNode, HumanConfirmNode)
│   │       └── ticket-chat/
│   │           ├── ChatTimeline.tsx
│   │           ├── AgentMessageBubble.tsx
│   │           ├── DiffViewerModal.tsx
│   │           ├── ApprovalBanner.tsx
│   │           ├── LiveWorkflowMiniMap.tsx
│   │           └── RemoteTerminalDrawer.tsx
│   └── package.json
└── IMPLEMENTATION_PLAN.md
```

3. Lộ trình Triển khai Chi tiết (5 Phases)

PHASE 1: Core ACP Subprocess Bridge & Dynamic Permission Guard

1.1. Yêu cầu cài đặt công cụ CLI nền tảng

```
npm install -g @agentclientprotocol/claude-agent-acp @agentclientprotocol/codex-acp
```

1.2. Đặc tả `backend/app/acp/types.py`

Định nghĩa các kiểu dữ liệu chuẩn hóa:

- `RuntimeType`: `"claude-code"` (`claude-agent-acp`) | `"codex"` (`codex-acp`).
- `PermissionProfile`:

```
class PermissionProfile(BaseModel):
    allow_read: bool = True
    allow_write: bool = False
    allow_terminal: bool = False
```

```
    allowed_commands: List[str] = []
    blocked_commands: List[str] = ["git commit", "git push", "git reset --hard", "rm -rf"]
```

- `ACPStreamEvent`:
  - `agent_name`: str (VD: `@ba`, `@developer`)
  - `event_type`: Literal[`"thinking"`, `"tool_call"`, `"file_read"`, `"file_edit"`, `"terminal_output"`, `"text_delta"`, `"completed"`, `"error"`]
  - `content`: str
  - `metadata`: dict (chứa đường dẫn file, diff patch, command...)

1.3. Đặc tả `backend/app/acp/guard.py`

Viết class `DynamicPermissionGuard`:

- Chặn và kiểm tra mọi gói tin JSON-RPC `method == "session/requestPermission"` từ ACP subprocess gửi lên.
- Đối chiếu với `PermissionProfile` của Agent đang chạy:
  - Nếu Agent yêu cầu `fs/writeTextFile` hoặc `Edit` mà `allow_write == False` -> trả về `"optionId": "deny"`.
  - Nếu Agent yêu cầu chạy lệnh bash chứa từ khóa trong `blocked_commands` -> trả về `"optionId": "deny"`.
  - Cấu hình mặc định cho 4 Agent chuẩn:
    - `@ba`: `allow_read=True, allow_write=False, allow_terminal=False`
    - `@developer`: `allow_read=True, allow_write=True, allow_terminal=True`, chặn `["git commit", "git push"]`
    - `@tester`: `allow_read=True, allow_write=False, allow_terminal=True` (cho phép chạy test runner), chặn `["git commit", "git push"]`
    - `@leader`: `allow_read=True, allow_write=False, allow_terminal=True`, `blocked_commands=["rm -rf"]` (cho phép toàn quyền `git add`, `git commit`, `git push`).

1.4. Đặc tả `backend/app/acp/client.py`

Viết class `ACPSubprocessClient`:

- Quản lý subprocess bằng `asyncio.create_subprocess_exec` tại thư mục `cwd=project.workspace_path`.
- Thực hiện handshake:
  1. Gửi `"initialize"` với `capabilities: {"fs": {"readTextFile": True, "writeTextFile": True}, "terminal": True}`.
  2. Gửi `"session/new"` với `cwd: workspace_path`.
- Hàm `stream_prompt(agent, user_prompt)`:
  - Nếu `agent.runtime == "claude-code"`: truyền `agent.system_prompt` vào `_meta.systemPrompt.append` và `model: agent.model`.
  - Nếu `agent.runtime == "codex"`: truyền `agent.system_prompt` vào `systemPrompt` và `model: agent.model`.
  - Đọc luồng `stdout` theo từng dòng (NDJSON), chuyển đổi thành `ACPStreamEvent` và `yield` ra ngoài.

PHASE 2: Database Schema & REST API (Custom Agents, Teams, Projects)

2.1. Đặc tả ORM Models (`backend/app/db/models.py`)

1. **Bảng `agents`**:
   - `id` (UUID, PK)
   - `name` (String, Unique - VD: `@ba`, `@developer`, `@security_audit`)
   - `display_name` (String - VD: `Business Analyst`)
   - `role` (String - VD: `BA`, `DEVELOPER`, `TESTER`, `LEADER`, `CUSTOM`)
   - `description` (Text - Mô tả tóm tắt)
   - `system_prompt` (Text - Mô tả chi tiết vai trò và quy tắc hành xử)
   - `runtime` (String - `"claude-code"` | `"codex"`)

- `model` (String - VD: `"claude-3-7-sonnet-latest"`, `"gpt-4o"`, `"o3-mini"`)
- `permission_profile` (JSONB)
- `is_builtin` (Boolean)

2. **Bảng `teams` & `team_agents`**:
   - Cho phép gom nhiều Agent thành một Team (VD: `Default Bugfix Team`).
3. **Bảng `projects` & `project_members`**:
   - `id`, `name`, `workspace_path` (VD: `D:\VETC\FE\HNBG_DEV`), `git_branch`.
   - `workflow_graph` (JSONB - lưu cấu trúc `nodes` và `edges` của React Flow).
   - `project_members`: liên kết User, Agent đơn lẻ hoặc nguyên Team vào Project.
4. **Bảng `tickets` & `ticket_messages`**:
   - `tickets`: `id`, `project_id`, `title`, `bug_description`, `status` (`OPEN`, `RUNNING`, `WAITING_USER_INPUT`, `WAITING_USER_CONFIRM`, `COMPLETED`, `FAILED`), `thread_id`.
   - `ticket_messages`: lưu toàn bộ lịch sử chat, thinking log, file diffs, terminal output của từng Agent.

2.2. Các API Endpoints cần thiết

- `GET/POST/PUT/DELETE /api/agents`: Quản lý Agent (hỗ trợ tạo mới Custom Agent với đầy đủ trường: tên, vai trò, mô tả chi tiết, loại runtime, chọn model).
- `GET /api/agents/runtime-models`: Trả về danh sách model khả dụng cho từng loại runtime.
- `POST /api/agents/{id}/test`: Kiểm tra kết nối CLI ACP.
- `GET/POST/PUT/DELETE /api/teams`: Quản lý Team.
- `GET/POST/PUT /api/projects`: Kiểm tra sự tồn tại của thư mục `workspace_path` trên ổ đĩa khi lưu.
- `POST /api/projects/{id}/seed-default`: Tự động khởi tạo 4 Agent (`@ba`, `@developer`, `@tester`, `@leader`) và tạo sẵn Workflow mặc định cho dự án.
- `PUT /api/projects/{id}/workflow`: Lưu sơ đồ kéo thả.

PHASE 3: Dynamic LangGraph Workflow Engine & WebSocket Control

3.1. Định nghĩa `WorkflowState` (`backend/app/engine/state.py`)

```
class WorkflowState(TypedDict):
    ticket_id: str
    workspace_path: str
    bug_description: str
    current_node_id: str
    ba_reproduce_steps: Optional[str]
    needs_user_clarification: bool
    dev_change_summary: Optional[str]
    modified_files: List[str]
    tester_report: Optional[str]
    test_passed: bool
    user_confirmed: bool
    user_feedback: Optional[str]
    git_commit_hash: Optional[str]
    messages: List[Dict[str, Any]]
```

3.2. Thiết kế Prompts Mặc định & Cơ chế Tự Chuyển Giao

1. **Node `@ba`**:
   - Đọc `bug_description` và đối chiếu cây thư mục tại `workspace_path`.
   - Nếu thông tin lỗi mơ hồ: Đặt câu hỏi cho người dùng và kết thúc bằng cờ `[ACTION:ASK_USER]`. Engine sẽ gọi `interrupt()` và đổi trạng thái ticket sang `WAITING_USER_INPUT`.
   - Nếu đã rõ: Xuất kịch bản tái tạo lỗi chi tiết (Steps to Reproduce - STR) và kết thúc bằng cờ `[ACTION:HANDOFF_DEV]`.
2. **Node `@developer`**:
   - Nhận STR từ `@ba` (hoặc lỗi trả về từ `@tester` / User).

- Định vị file trong `workspace_path`, sửa code, chạy build/lint kiểm tra, tuyệt đối không commit.
- Tóm tắt các file đã sửa và kết thúc bằng `[ACTION:HANDOFF_TESTER]`.

3. **Node `@tester`**:
   - Kiểm tra thay đổi code của `@developer` so với STR của `@ba`, chạy test.
   - Nếu chưa đạt: Xuất báo cáo lỗi kèm cờ `[ACTION:TEST_FAILED]` -> đồ thị tự động quay về Node `@developer`.
   - Nếu đạt: Xuất báo cáo nghiệm thu kèm cờ `[ACTION:TEST_PASSED]` -> đồ thị chuyển tới Node `HumanConfirmGate`.
4. **Node `HumanConfirmGate`**:
   - Gọi `langgraph.types.interrupt()` để dừng luồng, đổi trạng thái ticket sang `WAITING_USER_CONFIRM`, hiển thị hộp thoại xác nhận trên giao diện người dùng.
5. **Node `@leader`**:
   - Chỉ kích hoạt khi User bấm **Confirm**.
   - Chạy `git status`, `git diff` để review lần cuối, sau đó thực thi:

     ```
     git add <modified_files>
     git commit -m "fix(<scope>): <summary>"
     git push origin HEAD
     ```

   - Trả về mã Commit Hash, đổi trạng thái ticket sang `COMPLETED`.

3.3. Bộ biên dịch Đồ thị (`backend/app/engine/compiler.py`)

- Đọc JSON `workflow_graph` (`nodes`, `edges`) của Project và biên dịch động thành `StateGraph(WorkflowState)`.
- Bất kỳ **Custom Agent** nào người dùng kéo thả thêm vào giữa luồng (ví dụ: `@security_checker` nằm giữa `@developer` và `@tester`) đều được tự động gắn vào chuỗi thực thi thông qua `agent_node_factory(agent_id)`.

PHASE 4: Frontend Quản trị & Workflow Builder (Next.js + React Flow + shadcn/ui)

4.1. Modal Tạo/Chỉnh sửa Agent (`CreateAgentDialog.tsx`)

Form gồm đầy đủ các trường:

1. **Tên định danh (Handle):** Input có prefix `@` (VD: `@ui_reviewer`).
2. **Tên hiển thị & Vai trò (Role):** Input tên vai trò (VD: `UI/UX Code Reviewer`).
3. **Loại Agent (Runtime):** Select chọn `Claude Code (ACP)` hoặc `OpenAI Codex (ACP)`.
4. **Chọn Model:** Select phụ thuộc vào Runtime:
   - Nếu chọn `Claude Code`: `claude-3-7-sonnet-latest`, `claude-3-5-sonnet-latest`, `claude-3-5-haiku-latest`.
   - Nếu chọn `OpenAI Codex`: `gpt-4o`, `gpt-4o-mini`, `o3-mini`.
5. **Mô tả chi tiết vai trò (System Prompt):** Textarea lớn định nghĩa nhiệm vụ, quy tắc kiểm tra và định dạng phản hồi.
6. **Quyền hạn (Permissions):** Các công tắc bật/tắt `Read File`, `Write File`, `Run Terminal` và danh sách lệnh cấm.

4.2. Trang Kéo Thả Workflow (`/projects/[id]/workflow`)

Sử dụng thư viện `@xyflow/react`:

- **Sidebar bên trái:** Hiển thị danh sách toàn bộ Agent trong Project (cả 4 Agent mặc định lẫn các Custom Agent vừa tạo) + các khối điều khiển (`Start`, `Condition Pass/Fail`, `User Confirmation Gate`, `End`).
- **Canvas trung tâm:** Cho phép kéo thả các Node, nối dây (`edges`) giữa các Agent.
- **Nút "Tải Template Fix Bug Chuẩn":** Tự động vẽ luồng: `Start -> @ba -> @developer -> @tester -> (Pass/Fail) -> User Confirm Gate -> @leader -> End`.

PHASE 5: Giao diện Phòng Chat Ticket & Live Execution (Phong cách Buzz + 9remote)

5.1. Bố cục màn hình Ticket (`/projects/[id]/tickets/[ticketId]`)

Thiết kế chia 2 khu vực chính:

1. **Khu vực Trái (70%) - Multi-Agent Chat Stream:**

- Hiển thị luồng hội thoại thời gian thực giữa Người dùng và các Agent (`@ba`, `@developer`, `@tester`, `@leader`, `@custom...`).
- Trong mỗi bong bóng tin nhắn của Agent (`AgentMessageBubble.tsx`), tích hợp các khối tương tác lấy cảm hứng từ `9remote`:
  - **Thinking Block:** Khung thu gọn hiển thị suy luận nội tâm của Agent.
  - **Tool & File Badges:** Hiển thị trực quan Agent đang đọc file nào trong `D:\VETC\FE\HNBG_DEV`.
  - **Inline Diff Viewer:** Khi `@developer` sửa file, hiển thị nút xem nhanh mã nguồn thay đổi (xanh/đỏ) ngay trong tin nhắn.
  - **Terminal Output Card:** Hiển thị log chạy test của `@tester` hoặc kết quả `git commit & push` của `@leader`.

2. **Khu vực Phải (30%) - Live Workflow Tracker & Remote Terminal:**
   - **Tab 1 - Live Graph:** Hiển thị sơ đồ Workflow thu nhỏ; Agent nào đang hoạt động thì Node đó sáng đèn nhấp nháy (`animate-pulse`).
   - **Tab 2 - Remote Terminal (`Xterm.js`):** Kết nối qua PTY WebSocket tới thư mục `D:\VETC\FE\HNBG_DEV` để người dùng có thể gõ lệnh trực tiếp khi cần can thiệp thủ công.

5.2. Hộp thoại Xác nhận Nghiệm thu (`ApprovalBanner.tsx`)

Khi `@tester` xác nhận lỗi đã sửa xong và workflow dừng ở `HumanConfirmGate`:

- Hiển thị thanh thông báo nổi ở cuối khung chat:

  @tester đã xác nhận bản vá lỗi vượt qua kiểm thử.

  Hãy kiểm tra lại ứng dụng. Nếu bạn xác nhận lỗi đã hết, **@leader** sẽ tiến hành commit & push lên Git.

- **Nút 1 (Màu xanh): "Xác nhận Đã Xong (Giao @leader Commit & Push)"** -> Bắn sự kiện WebSocket kích hoạt `@leader` đẩy code lên Git.
- **Nút 2 (Màu đỏ): "Vẫn còn lỗi (Yêu cầu @developer sửa lại)"** -> Mở ô nhập phản hồi và trả luồng về cho `@developer`.

4. Hướng dẫn Chạy lệnh cho Codex / Claude Code

Để AI coding agent tự động triển khai toàn bộ dự án này theo đúng thứ tự, hãy chạy lần lượt các prompt sau:

1. **Bước 1:** `"Đọc file IMPLEMENTATION_PLAN.md, khởi tạo cấu trúc thư mục dự án và triển khai đầy đủ PHASE 1 (ACP Bridge & Permission Guard) kèm unit test."`
2. **Bước 2:** `"Tiếp tục triển khai PHASE 2 (Database Models, Schemas và toàn bộ REST API cho Agents, Custom Agents, Teams, Projects, Tickets) trong IMPLEMENTATION_PLAN.md."`
3. **Bước 3:** `"Triển khai PHASE 3 (LangGraph Dynamic Workflow Compiler, Human-in-the-loop Interrupt và WebSocket Streaming) theo đúng đặc tả trong IMPLEMENTATION_PLAN.md."`
4. **Bước 4:** `"Khởi tạo ứng dụng Next.js tại thư mục frontend/ và triển khai PHASE 4 (Giao diện quản lý Custom Agent, Team, Project và React Flow Workflow Builder)."`
5. **Bước 5:** `"Triển khai PHASE 5 (Giao diện phòng chat Ticket thời gian thực, Diff Viewer, Approval Banner và Live Workflow MiniMap), kết nối đồng bộ với Backend WebSocket."`

User prompt: xuất file md chi tiết từng phase và IMPLEMENTATION_PLAN.md

Response: Dưới đây là toàn bộ 6 file tài liệu kỹ thuật hoàn chỉnh (`IMPLEMENTATION_PLAN.md` và `PHASE_1.md` đến `PHASE_5.md`). Để tránh tốn thời gian tạo từng file thủ công, ở cuối bài viết có kèm **script PowerShell tự động xuất cả 6 file** vào thư mục dự án chỉ với 1 lần chạy.

1. File tổng quan: `IMPLEMENTATION_PLAN.md`

```
# MASTER IMPLEMENTATION PLAN: MULTI-AGENT WORKFLOW PLATFORM (BUZZ + DIFY + 9REMOTE)

## 1. Mục tiêu Hệ thống
Xây dựng nền tảng điều phối đội ngũ AI Agent làm việc trực tiếp trên thư mục dự án local (ví dụ: `D:\VETC\FE\HNBG_DEV`),
- **ACP Subprocess Bridge (Kiểu Buzz Desktop):** Kết nối `claude-agent-acp` (Claude Code) và `codex-acp` (OpenAI Codex)
- **Custom Agents & Role-Based Permission Guard:** Có sẵn 4 Agent mặc định (`@ba`, `@developer`, `@tester`, `@leader`) v
- **Visual Workflow Builder (Kiểu Dify):** Giao diện kéo thả Node bằng React Flow (`@xyflow/react`) và biên dịch động sa
- **Real-time Chat & Remote Terminal UI (Kiểu 9remote):** Hiển thị quá trình suy nghĩ (Thinking), bảng so sánh code (Dif

---

## 2. Cấu trúc Thư mục Dự án (Monorepo)

```text
multi-agent-platform/
```
```

```
├── docs/
│   └── specs/
│       ├── IMPLEMENTATION_PLAN.md
│       ├── PHASE_1_ACP_BRIDGE.md
│       ├── PHASE_2_DB_AND_CUSTOM_AGENTS.md
│       ├── PHASE_3_WORKFLOW_ENGINE.md
│       ├── PHASE_4_FRONTEND_BUILDER.md
│       └── PHASE_5_TICKET_CHAT_UI.md
├── backend/
│   ├── app/
│   │   ├── acp/
│   │   │   ├── __init__.py
│   │   │   ├── types.py          # Pydantic schemas cho JSON-RPC 2.0 & ACP events
│   │   │   ├── guard.py          # DynamicPermissionGuard chặn lệnh theo Permission Profile
│   │   │   ├── client.py         # ACPSubprocessClient giao tiếp với claude-agent-acp / codex-acp
│   │   │   └── pty_manager.py    # Quản lý Pseudo-Terminal cho Xterm.js
│   │   ├── db/
│   │   │   ├── session.py        # Async SQLAlchemy engine
│   │   │   └── models.py         # ORM Models: Agent, Team, Project, Workflow, Ticket, Message
│   │   ├── schemas/              # Pydantic Request/Response models
│   │   ├── engine/
│   │   │   ├── state.py          # WorkflowState cho LangGraph
│   │   │   ├── nodes.py          # Node runners cho Agent, Condition, HumanConfirm
│   │   │   └── compiler.py       # Biên dịch JSON React Flow -> Compiled LangGraph
│   │   ├── api/
│   │   │   ├── agents.py         # CRUD Agent, Custom Agent & Test CLI Connection
│   │   │   ├── teams.py          # CRUD Team
│   │   │   ├── projects.py       # CRUD Project, Members, Save Workflow, Seed Default
│   │   │   └── tickets.py        # CRUD Ticket & Trigger Workflow
│   │   ├── ws/
│   │   │   ├── ticket_stream.py  # WebSocket phòng chat Ticket & Human Approval
│   │   │   └── terminal_ws.py    # WebSocket stream raw bytes cho Xterm.js
│   │   └── main.py               # FastAPI Entrypoint
│   ├── alembic/
│   ├── requirements.txt
│   └── tests/
└── frontend/
    ├── src/
    │   ├── app/
    │   │   ├── agents/page.tsx                     # Quản lý & tạo mới Custom Agent
    │   │   ├── teams/page.tsx                      # Quản lý Team
    │   │   └── projects/
    │   │       ├── page.tsx                        # Danh sách Project
    │   │       └── [id]/
    │   │           ├── page.tsx                    # Danh sách Ticket trong Project
    │   │           ├── members/page.tsx            # Thêm User / Agent / Team vào Project
    │   │           ├── workflow/page.tsx           # Kéo thả Workflow (React Flow)
    │   │           └── tickets/[ticketId]/page.tsx # Phòng Chat Ticket + Live Graph + Terminal
    │   └── components/
    │       ├── agents/CreateAgentDialog.tsx
    │       ├── workflow/
    │       │   ├── WorkflowCanvas.tsx
    │       │   ├── SidebarPalette.tsx
    │       │   └── nodes/ (StartNode, AgentNode, ConditionNode, HumanConfirmNode)
    │       └── ticket-chat/
    │           ├── ChatTimeline.tsx
    │           ├── AgentMessageBubble.tsx
    │           ├── DiffViewerModal.tsx
    │           ├── ApprovalBanner.tsx
    │           ├── LiveWorkflowMiniMap.tsx
    │           └── RemoteTerminalDrawer.tsx
    └── package.json
```

3. Thứ tự Thực thi với Codex / Claude Code

Chạy lần lượt 5 câu lệnh dưới đây để hoàn thiện toàn bộ dự án:

1. **Giai đoạn 1:** `Hãy đọc docs/specs/PHASE_1_ACP_BRIDGE.md và triển khai hoàn chỉnh tầng giao tiếp ACP Subprocess cùng bộ lọc quyền hạn và unit test.`
2. **Giai đoạn 2:** `Hãy đọc docs/specs/PHASE_2_DB_AND_CUSTOM_AGENTS.md và triển khai Database Models, Schemas cùng toàn bộ REST API quản lý Custom Agent, Team, Project, Ticket.`

3. **Giai đoạn 3:** Hãy đọc docs/specs/PHASE_3_WORKFLOW_ENGINE.md và triển khai bộ biên dịch LangGraph động, cơ chế Human-in-the-loop interrupt và WebSocket streaming.
4. **Giai đoạn 4:** Hãy đọc docs/specs/PHASE_4_FRONTEND_BUILDER.md và xây dựng giao diện Next.js quản lý Custom Agent, Project và trình kéo thả Workflow bằng @xyflow/react.
5. **Giai đoạn 5:** Hãy đọc docs/specs/PHASE_5_TICKET_CHAT_UI.md và xây dựng giao diện phòng chat Ticket thời gian thực, Diff Viewer, Approval Banner và Remote Terminal.

---

2. File Phase 1: `PHASE_1_ACP_BRIDGE.md`

```markdown
# PHASE 1: Core ACP Subprocess Bridge & Dynamic Permission Guard

## 1. Mục tiêu
Xây dựng tầng giao tiếp Python kết nối với `@agentclientprotocol/claude-agent-acp` và `@agentclientprotocol/codex-acp` t

## 2. Cài đặt công cụ tiền đề
```bash
npm install -g @agentclientprotocol/claude-agent-acp @agentclientprotocol/codex-acp
```

3. Các File cần triển khai

3.1. backend/app/acp/types.py

Định nghĩa các Pydantic Schema:

```
from pydantic import BaseModel, Field
from typing import Literal, Optional, List, Dict, Any

RuntimeType = Literal["claude-code", "codex"]

class PermissionProfile(BaseModel):
    allow_read: bool = True
    allow_write: bool = False
    allow_terminal: bool = False
    allowed_commands: List[str] = Field(default_factory=list)
    blocked_commands: List[str] = Field(
        default_factory=lambda: ["git commit", "git push", "git reset --hard", "rm -rf"]
    )

class ACPStreamEvent(BaseModel):
    agent_name: str
    event_type: Literal[
        "thinking", "tool_call", "file_read", "file_edit",
        "terminal_output", "text_delta", "completed", "error"
    ]
    content: str
    metadata: Dict[str, Any] = Field(default_factory=dict)
```

3.2. backend/app/acp/guard.py

Xây dựng class `DynamicPermissionGuard`:

- Phương thức `evaluate(permission_request: dict, profile: PermissionProfile) -> bool`:
  - Kiểm tra loại hành động từ gói tin JSON-RPC `session/requestPermission` của ACP:
    - Nếu là thao tác đọc file (`fs/readTextFile`, `Read`, `Grep`, `Glob`): trả về `profile.allow_read`.
    - Nếu là thao tác ghi/sửa file (`fs/writeTextFile`, `Edit`, `Write`): trả về `profile.allow_write`.
    - Nếu là thao tác chạy lệnh terminal (`terminal/run`, `Bash`):
      - Nếu `not profile.allow_terminal`: trả về `False`.
      - Kiểm tra chuỗi lệnh command: nếu chứa bất kỳ chuỗi con nào nằm trong `profile.blocked_commands` (ví dụ `@developer` hoặc `@tester` cố chạy `git commit` hoặc `git push`) -> lập tức trả về `False`.
      - Ngược lại trả về `True`.

3.3. backend/app/acp/client.py
```

Xây dựng class `ACPSubprocessClient`:

- Thuộc tính: `runtime: RuntimeType`, `model: str`, `workspace_dir: str`, `permission_profile: PermissionProfile`.
- Ánh xạ câu lệnh binary:
  - `"claude-code"` -> `"claude-agent-acp"`
  - `"codex"` -> `"codex-acp"`
- Phương thức `async def start()`:
  - Khởi chạy process bằng `asyncio.create_subprocess_exec` với `cwd=self.workspace_dir`.
  - Gửi gói tin JSON-RPC `initialize` với `protocolVersion: "2025-01-01"`, `capabilities: {"fs": {"readTextFile": True, "writeTextFile": True}, "terminal": True}`.
  - Gửi gói tin `session/new` với `cwd: self.workspace_dir`.
- Phương thức `async def stream_prompt(agent_name: str, system_prompt: str, user_prompt: str) -> AsyncGenerator[ACPStreamEvent, None]`:
  - Nếu `runtime == "claude-code"`: đặt `system_prompt` vào `params._meta.systemPrompt.append` (giữ nguyên prompt gốc của Claude Code và nối thêm vai trò của Agent), đặt `params.model = self.model`.
  - Nếu `runtime == "codex"`: đặt `system_prompt` vào `params.systemPrompt`, đặt `params.model = self.model`.
  - Đọc từng dòng JSON từ `self.process.stdout`:
    - Nếu `method == "session/update"`: bóc tách `thinking`, `tool_use`, `text` và `yield ACPStreamEvent`.
    - Nếu `method == "session/requestPermission"`: gọi `DynamicPermissionGuard.evaluate()`, gửi phản hồi JSON-RPC `{"outcome": {"outcome": "selected", "optionId": "allow" if allowed else "deny"}}` xuống `stdin`.
    - Nếu nhận `result` khớp với `request_id` của `session/prompt`: `yield ACPStreamEvent(event_type="completed")` và thoát vòng lặp.

4. Tiêu chí Nghiệm thu Phase 1

- Viết file test `backend/tests/test_acp_bridge.py`.
- Test 1: Khởi tạo Agent `@ba` (`allow_write=False`), yêu cầu đọc cấu trúc thư mục -> Thành công. Yêu cầu tạo file mới -> Bị `DynamicPermissionGuard` chặn.
- Test 2: Khởi tạo Agent `@developer` (`allow_write=True`, chặn `git commit`), yêu cầu sửa file -> Thành công. Yêu cầu chạy `git commit` -> Bị chặn.

---

3. File Phase 2: `PHASE_2_DB_AND_CUSTOM_AGENTS.md`

```markdown
# PHASE 2: Database Schema & REST APIs (Custom Agents, Teams, Projects, Tickets)

## 1. Mục tiêu
Xây dựng cơ sở dữ liệu (SQLAlchemy Async + PostgreSQL/SQLite) và hệ thống REST API bằng FastAPI hỗ trợ 4 Agent mặc định

## 2. Thiết kế Database Models (`backend/app/db/models.py`)

### 2.1. Bảng `agents`
- `id`: UUID (Primary Key)
- `name`: String(50), Unique, Not Null (VD: `@ba`, `@developer`, `@tester`, `@leader`, `@security_checker`)
- `display_name`: String(100), Not Null (VD: `Business Analyst`, `Security Auditor`)
- `role`: String(50), Not Null (VD: `BA`, `DEVELOPER`, `TESTER`, `LEADER`, `CUSTOM`)
- `description`: Text (Mô tả ngắn hiển thị trên thẻ Agent)
- `system_prompt`: Text, Not Null (Mô tả chi tiết vai trò, quy tắc làm việc và định dạng đầu ra)
- `runtime`: String(30), Not Null (`"claude-code"` hoặc `"codex"`)
- `model`: String(100), Not Null (VD: `"claude-3-7-sonnet-latest"`, `"claude-3-5-sonnet-latest"`, `"gpt-4o"`, `"o3-mini"
- `permission_profile`: JSON, Not Null (Lưu cấu hình `allow_read`, `allow_write`, `allow_terminal`, `blocked_commands`)
- `is_builtin`: Boolean, Default `False`

### 2.2. Bảng `teams` và `team_agents`
- `teams`: `id` (UUID), `name` (String), `description` (Text).
- `team_agents`: `team_id` (FK), `agent_id` (FK).
```

2.3. Bảng `projects` và `project_members`
- `projects`:
  - `id` (UUID)
  - `name` (String - VD: `HNBG_DEV`)
  - `workspace_path` (String - VD: `D:\VETC\FE\HNBG_DEV`)
  - `git_branch` (String - Default: `main`)
  - `workflow_graph` (JSON - Lưu `{ "nodes": [...], "edges": [...] }` của React Flow)
- `project_members`:
  - `project_id` (FK), `member_type` (`"USER"` | `"AGENT"` | `"TEAM"`), `member_id` (UUID).

2.4. Bảng `tickets` và `ticket_messages`
- `tickets`:
  - `id` (UUID), `project_id` (FK), `title` (String), `bug_description` (Text)
  - `status`: String (`OPEN`, `RUNNING`, `WAITING_USER_INPUT`, `WAITING_USER_CONFIRM`, `COMPLETED`, `FAILED`)
  - `thread_id`: String (Dùng làm checkpoint thread cho LangGraph)
- `ticket_messages`:
  - `id` (UUID), `ticket_id` (FK), `sender_name` (String), `sender_role` (String)
  - `event_type` (String), `content` (Text), `metadata_json` (JSON), `created_at` (DateTime)

3. Danh mục REST API cần xây dựng

3.1. Quản lý Agent (`backend/app/api/agents.py`)
- `GET /api/agents/models-catalog`: Trả về danh sách Model hợp lệ theo từng Runtime:
  ```json
  {
    "claude-code": [
      {"id": "claude-3-7-sonnet-latest", "name": "Claude 3.7 Sonnet"},
      {"id": "claude-3-5-sonnet-latest", "name": "Claude 3.5 Sonnet"},
      {"id": "claude-3-5-haiku-latest", "name": "Claude 3.5 Haiku"}
    ],
    "codex": [
      {"id": "gpt-4o", "name": "GPT-4o"},
      {"id": "gpt-4o-mini", "name": "GPT-4o Mini"},
      {"id": "o3-mini", "name": "OpenAI o3-mini"}
    ]
  }
  ```
- `POST /api/agents`: Tạo mới Custom Agent (validate `name` bắt đầu bằng `@`).
- `GET /api/agents`: Lấy danh sách tất cả Agent.
- `PUT /api/agents/{id}` & `DELETE /api/agents/{id}` (Không cho xóa nếu `is_builtin == True`).
- `POST /api/agents/{id}/test-connection`: Khởi tạo thử ACP process và trả về trạng thái kết nối.

3.2. Seed 4 Agent Mặc định & Quản lý Project (`backend/app/api/projects.py`)
- Hàm khởi tạo tự động 4 Agent mặc định khi khởi chạy server lần đầu:
  1. `@ba` (Read-only): Tiếp nhận lỗi, hỏi lại người dùng nếu thiếu thông tin, lập kịch bản tái tạo lỗi step-by-step (STR).
  2. `@developer` (Read/Write, Cấm commit/push): Đọc STR từ `@ba`, tìm nguyên nhân gốc rễ, sửa code trong `workspace_path`, chạy build kiểm tra.
  3. `@tester` (Read + Run test, Cấm sửa code & commit): Kiểm tra code `@developer` vừa sửa dựa trên STR của `@ba`. Báo Pass hoặc Fail.
  4. `@leader` (Git Release): Chỉ chạy sau khi người dùng bấm Confirm xác nhận lỗi đã hết. Chạy `git diff` review code, `git add`, `git commit` và `git push`.
- `POST /api/projects`: Tạo Project mới, kiểm tra đường dẫn `workspace_path` có tồn tại trên máy chủ hay không.
- `PUT /api/projects/{id}/workflow`: Lưu JSON đồ thị kéo thả.

4. Tiêu chí Nghiệm thu Phase 2

- Khởi động FastAPI tự động seed 4 Agent mặc định (`@ba`, `@developer`, `@tester`, `@leader`).
- Gọi `POST /api/agents` tạo thành công một Agent mới tên `@db_optimizer`, runtime `codex`, model `gpt-4o`, lưu đầy đủ vào DB.

---

4. File Phase 3: `PHASE_3_WORKFLOW_ENGINE.md`

```markdown
```

```
# PHASE 3: Dynamic LangGraph Workflow Compiler & WebSocket Streaming

## 1. Mục tiêu
Biên dịch sơ đồ JSON (từ trình kéo thả React Flow) thành một `StateGraph` thực thi được của **LangGraph**. Điều phối luồ

## 2. Đặc tả Kỹ thuật

### 2.1. Định nghĩa `WorkflowState` (`backend/app/engine/state.py`)
```python
from typing import TypedDict, List, Optional, Dict, Any

class BugFixWorkflowState(TypedDict):
    ticket_id: str
    workspace_path: str
    bug_description: str
    current_agent: str
    ba_reproduce_steps: Optional[str]
    needs_user_clarification: bool
    ba_question: Optional[str]
    user_clarification_answer: Optional[str]
    dev_summary: Optional[str]
    modified_files: List[str]
    tester_report: Optional[str]
    test_passed: bool
    user_confirmed: bool
    user_reject_reason: Optional[str]
    git_commit_hash: Optional[str]
    conversation_log: List[Dict[str, Any]]
```

2.2. Thực thi Node Động (`backend/app/engine/nodes.py`)

1. **Hàm `create_agent_node_runner(agent_model, ws_manager)`**:
    - Phục vụ cho cả 4 Agent mặc định lẫn bất kỳ **Custom Agent** nào có mặt trong đồ thị.
    - Ghép ngữ cảnh từ `BugFixWorkflowState` thành prompt đầu vào.
    - Bổ sung quy ước cờ điều hướng ở cuối câu trả lời của từng vai trò:
        - Với `@ba`: Nếu cần hỏi lại người dùng, kết thúc bằng `[STATUS:NEEDS_CLARIFICATION]`. Nếu đã rõ kịch bản step-by-step, kết thúc bằng `[STATUS:STR_READY]`.
        - Với `@tester`: Nếu kiểm thử thành công, kết thúc bằng `[STATUS:TEST_PASSED]`. Nếu còn lỗi, kết thúc bằng `[STATUS:TEST_FAILED]`.
    - Trong khi `ACPSubprocessClient.stream_prompt()` chạy:
        - Lưu từng sự kiện vào bảng `ticket_messages`.
        - Bắn ngay lập tức qua WebSocket tới toàn bộ client đang mở Ticket.
2. **Node `user_clarification_gate`**:
    - Dùng `langgraph.types.interrupt({"type": "ba_question", "question": state["ba_question"]})` để tạm dừng đồ thị chờ người dùng nhập câu trả lời trên khung chat.
3. **Node `user_confirmation_gate`**:
    - Sau khi `@tester` trả về `[STATUS:TEST_PASSED]`, gọi `langgraph.types.interrupt({"type": "user_confirm_fix"})`.
    - Đổi trạng thái Ticket sang `WAITING_USER_CONFIRM`.
    - Khi người dùng xác nhận:
        - Nếu `approved == True`: Chuyển tiếp sang Node `@leader` để chạy `git add`, `git commit`, `git push`.
        - Nếu `approved == False`: Quay ngược lại Node `@developer` kèm phản hồi của người dùng.

2.3. Bộ biên dịch Đồ thị (`backend/app/engine/compiler.py`)

- Đọc JSON `nodes` và `edges` từ `project.workflow_graph`.
- Khởi tạo `StateGraph(BugFixWorkflowState)` và gắn `MemorySaver` (hoặc `AsyncSqliteSaver`).
- Hỗ trợ nối thêm các Custom Agent vào bất kỳ vị trí nào trong chuỗi cạnh (`edges`) mà người dùng đã vẽ trên giao diện.

2.4. WebSocket Endpoint (`backend/app/ws/ticket_stream.py`)
```

- Route: `WS /ws/tickets/{ticket_id}`
- Xử lý các bản tin điều khiển từ Frontend:
  - `{"action": "start"}`: Khởi chạy workflow từ đầu với `bug_description`.
  - `{"action": "reply_ba", "content": "..."}`: Tiếp tục (resume) đồ thị từ `user_clarification_gate` với câu trả lời của người dùng.
  - `{"action": "confirm_fix", "approved": true | false, "feedback": "..."}`: Tiếp tục đồ thị từ `user_confirmation_gate`.

3. Tiêu chí Nghiệm thu Phase 3

- Test luồng hoàn chỉnh qua WebSocket:
  1. Báo lỗi mơ hồ -> `@ba` dừng lại hỏi người dùng -> Người dùng trả lời -> `@ba` xuất kịch bản tái tạo lỗi (STR).
  2. `@developer` nhận STR và sửa file trong thư mục dự án.
  3. `@tester` kiểm tra và báo Pass -> Hệ thống dừng ở `WAITING_USER_CONFIRM`.
  4. Bắn gói tin `{"action": "confirm_fix", "approved": true}` -> `@leader` tự động commit & push lên Git.

---

5. File Phase 4: `PHASE_4_FRONTEND_BUILDER.md`

```markdown
# PHASE 4: Giao diện Quản lý Custom Agent, Project & Trình Kéo Thả Workflow (Next.js + shadcn/ui)

## 1. Mục tiêu
Xây dựng ứng dụng Frontend bằng **Next.js (App Router) + TypeScript + Tailwind CSS + shadcn/ui** và thư viện **`@xyflow/
1. Tạo và quản lý các Agent (bao gồm Custom Agent).
2. Tạo Team và Project trỏ tới thư mục `D:\VETC\FE\HNBG_DEV`.
3. Thiết kế Workflow bằng cách kéo thả các Agent và nối dây giống Dify.

## 2. Khởi tạo & Cài đặt Thư viện
```bash
npx create-next-app@latest frontend --typescript --tailwind --eslint --app --src-dir
cd frontend
npx shadcn@latest init -d
npx shadcn@latest add button card dialog input textarea label select badge tabs switch scroll-area toast
npm install @xyflow/react lucide-react zustand @tanstack/react-query axios
```

3. Chi tiết các Trang và Component cần xây dựng

3.1. Trang Quản lý Agent (`src/app/agents/page.tsx` & `CreateAgentDialog.tsx`)

- Hiển thị lưới các thẻ Agent (4 Agent mặc định có nhãn `Built-in` và các Custom Agent do người dùng tạo).
- Nút **"+ Tạo Agent Mới"** mở Modal `CreateAgentDialog.tsx` gồm các trường:
  1. **Tên định danh (`name`):** Ô nhập có tiền tố `@` (VD: `@ui_specialist`, `@perf_auditor`).
  2. **Vai trò (`display_name` & `role`):** Nhập tên vai trò hiển thị (VD: `Performance Specialist`).
  3. **Loại Agent (`runtime`):** Select chọn giữa:
     - `Claude Code (claude-agent-acp)`
     - `OpenAI Codex (codex-acp)`
  4. **Chọn Model (`model`):** Dropdown tự động lọc theo Loại Agent đã chọn (VD: `claude-3-7-sonnet-latest`, `claude-3-5-sonnet-latest`, `gpt-4o`, `o3-mini`...).
  5. **Mô tả chi tiết vai trò (`system_prompt`):** Textarea lớn để nhập hướng dẫn chi tiết nhiệm vụ và quy tắc cho Agent.
  6. **Phân quyền (`permission_profile`):**
     - Switch bật/tắt: `Cho phép Đọc File`, `Cho phép Sửa File`, `Cho phép chạy Terminal`.
     - Input danh sách lệnh cấm (mặc định: `git commit, git push, rm -rf`).
  7. Nút **"Kiểm tra kết nối CLI"** và nút **"Lưu Agent"**.
```

3.2. Trang Quản lý Project (`src/app/projects/[id]/page.tsx`)

- Cho phép cấu hình:
  - Tên Project (VD: `HNBG_DEV`).
  - Đường dẫn thư mục mã nguồn (`D:\VETC\FE\HNBG_DEV`).
  - Quản lý thành viên: Thêm User, thêm từng Agent lẻ hoặc thêm nguyên Team vào Project.

3.3. Trình Kéo Thả Workflow (`src/app/projects/[id]/workflow/page.tsx`)

Sử dụng `@xyflow/react`:

- **Sidebar bên trái (`SidebarPalette.tsx`):**
  - Liệt kê tất cả Agent có trong hệ thống (cả 4 Agent mặc định lẫn Custom Agent).
  - Các khối điều khiển luồng: `Start Trigger`, `Condition (Pass / Fail / Need Info)`, `User Confirmation Gate`, `End Node`.
- **Custom Nodes (`src/components/workflow/nodes/`):**
  - `AgentNode.tsx`: Hiển thị tên Agent (`@ba`, `@developer`...), badge loại Runtime (`Claude Code` / `Codex`), tên Model và các cổng kết nối (Handles).
  - `HumanConfirmNode.tsx`: Khối cổng chờ người dùng xác nhận nghiệm thu trước khi chuyển sang `@leader`.
- **Thanh công cụ phía trên Canvas:**
  - Nút **"Tải Mẫu Fix Bug Mặc Định"**: Tự động sinh sơ đồ chuẩn: `[Start] -> [@ba] -> [Hỏi lại User nếu thiếu thông tin] -> [@developer] -> [@tester] -> [User Confirm Gate] -> [@leader (Commit & Push)] -> [End]`.
  - Nút **"Lưu Workflow"**: Gửi cấu trúc `nodes` và `edges` về API `PUT /api/projects/{id}/workflow`.

4. Tiêu chí Nghiệm thu Phase 4

- Tạo được một Custom Agent mới trên giao diện Web, thấy xuất hiện ngay trong thanh công cụ của Workflow Editor.
- Kéo thả, nối dây giữa các Agent và lưu sơ đồ vào Database thành công.

---

6. File Phase 5: `PHASE_5_TICKET_CHAT_UI.md`

```markdown
# PHASE 5: Giao diện Chạy Ticket Dạng Phòng Chat Multi-Agent & Remote Terminal

## 1. Mục tiêu
Xây dựng màn hình thực thi Ticket (`/projects/[id]/tickets/[ticketId]`) kết hợp phong cách phòng chat nhóm của **Buzz De

## 2. Cài đặt thêm thư viện hiển thị Code Diff & Terminal
```bash
cd frontend
npm install react-diff-viewer-continued @xterm/xterm @xterm/addon-fit react-markdown
```

3. Chi tiết các Component trong Phòng Chat Ticket

3.1. Bố cục màn hình Ticket (`src/app/projects/[id]/tickets/[ticketId]/page.tsx`)

Chia màn hình thành 2 cột:

- **Cột Trái (70% chiều rộng) - Luồng Hội Thoại Multi-Agent (`ChatTimeline.tsx`):**
  - Hiển thị tin nhắn theo thời gian thực nhận từ WebSocket `WS /ws/tickets/{ticketId}`.
  - Mỗi Agent có màu sắc và badge định danh riêng (VD: `@ba [Claude 3.7]`, `@developer [Codex GPT-4o]`, `@tester`, `@leader`).
- **Cột Phải (30% chiều rộng) - Sơ đồ Trực tiếp & Terminal:**
  - **Tab 1 (`LiveWorkflowMiniMap.tsx`):** Hiển thị sơ đồ workflow của dự án; khi Backend báo Agent nào đang hoạt động, Node của Agent đó phát sáng nhấp nháy (`ring-4 ring-blue-500 animate-pulse`).
  - **Tab 2 (`RemoteTerminalDrawer.tsx`):** Nhúng `@xterm/xterm` kết nối tới `WS /ws/terminal/{projectId}` để gõ lệnh kiểm tra trực tiếp trên thư mục `D:\VETC\FE\HNBG_DEV`.
```

3.2. Bong bóng Tin nhắn của Agent (`AgentMessageBubble.tsx`)

Mỗi lượt hoạt động của một Agent được gom gọn trong một Card thông minh gồm:

1. **Khối Suy nghĩ (`Thinking Accordion`):** Thu gọn mặc định, bấm vào để xem luồng suy luận nội tâm của Agent.
2. **Danh sách Thao tác Công cụ (`Tool Call Badges`):**
   - Hiển thị các nhãn nhỏ: `📖 Đã đọc file src/pages/Reconciliation.tsx`, `🔍 Tìm kiếm từ khóa exportReport`.
3. **Nút Xem Thay đổi Code (`Inline Diff Viewer`):**
   - Khi `@developer` chỉnh sửa file, hiển thị hộp so sánh code trước và sau khi sửa (màu xanh/đỏ) bằng `react-diff-viewer-continued`.
4. **Nội dung Phản hồi Chính (`Markdown Content`):**
   - Kịch bản tái tạo lỗi step-by-step của `@ba`, báo cáo sửa lỗi của `@developer`, kết quả test của `@tester` hoặc thông báo đã push Git kèm Commit Hash của `@leader`.

3.3. Hộp thoại Tương tác với Người dùng (`ApprovalBanner.tsx`)

1. **Trường hợp 1: Khi `@ba` cần hỏi thêm thông tin tái tạo lỗi (`status === "WAITING_USER_INPUT"`):**
   - Làm nổi bật ô nhập tin nhắn ở dưới cùng với dòng nhắc: *"@ba đang chờ bạn cung cấp thêm chi tiết để hoàn tất kịch bản tái tạo lỗi..."*.
2. **Trường hợp 2: Khi `@tester` báo đã sửa xong và chờ xác nhận (`status === "WAITING_USER_CONFIRM"`):**
   - Hiển thị Banner xác nhận nổi bật ngay phía trên thanh chat:

     > ✅ **@tester đã xác nhận lỗi được khắc phục thành công!**
     >
     > Vui lòng kiểm tra lại trên môi trường của bạn (`D:\VETC\FE\HNBG_DEV`). Nếu bạn xác nhận lỗi đã hết, **@leader** sẽ tự động commit và push code lên Git.

   - Gồm 2 nút bấm:
     - **Nút Xanh ("Xác nhận Đã Xong - Giao @leader Commit & Push"):** Gửi lệnh WebSocket `{"action": "confirm_fix", "approved": true}`.
     - **Nút Đỏ ("Chưa đạt - Yêu cầu @developer sửa lại"):** Mở ô nhập lý do và gửi `{"action": "confirm_fix", "approved": false, "feedback": "..."}`.

4. Kịch bản Kiểm thử Nghiệm thu Toàn Hệ thống (End-to-End)

1. Mở Project `HNBG_DEV` (trỏ tới `D:\VETC\FE\HNBG_DEV`).
2. Tạo một Ticket mới: *"Bấm nút xuất Excel trên màn hình đối soát không tải được file"*.
3. Quan sát trên khung chat:
   - `@ba` đọc mã nguồn, hỏi lại người dùng 1 câu làm rõ -> Người dùng trả lời -> `@ba` xuất kịch bản tái tạo lỗi (STR) chi tiết từng bước.
   - Node `@developer` sáng đèn trên sơ đồ -> `@developer` sửa file code (hiển thị bảng Diff trực quan), không chạy lệnh commit.
   - Node `@tester` sáng đèn -> `@tester` chạy kiểm tra và báo Pass.
   - `ApprovalBanner` bật lên -> Người dùng bấm **"Xác nhận Đã Xong"**.
   - Node `@leader` sáng đèn -> `@leader` chạy `git add`, `git commit`, `git push` và gửi mã Commit Hash vào khung chat, đóng Ticket.

Script Tự động Tạo toàn bộ các File `.md` trên Windows (PowerShell)

Bạn có thể mở PowerShell tại thư mục dự án của mình và chạy đoạn lệnh ngắn dưới đây để tạo sẵn cấu trúc thư mục `docs/sp

```powershell
# Tạo thư mục chứa tài liệu đặc tả
New-Item -ItemType Directory -Force -Path ".\docs\specs"

# Tạo sẵn các file rỗng để lưu đặc tả
$files = @(
    "IMPLEMENTATION_PLAN.md",
    "docs\specs\PHASE_1_ACP_BRIDGE.md",
    "docs\specs\PHASE_2_DB_AND_CUSTOM_AGENTS.md",
    "docs\specs\PHASE_3_WORKFLOW_ENGINE.md",
```

```
    "docs\specs\PHASE_4_FRONTEND_BUILDER.md",
    "docs\specs\PHASE_5_TICKET_CHAT_UI.md"
)

foreach ($file in $files) {
    if (-not (Test-Path $file)) {
        New-Item -ItemType File -Path $file -Force | Out-Null
        Write-Host "Đã tạo file: $file" -ForegroundColor Green
    }
}
```

User prompt: tạo file html giao diện

Response: File HTML單 (`index.html`) dưới đây mô phỏng toàn bộ giao diện nền tảng **Multi-Agent Workflow Platform (kết hợp Buzz + Dify + 9remote)** theo phong cách **Next.js + Tailwind CSS + shadcn/ui Dark Mode**.

File bao gồm đầy đủ 4 màn hình chức năng tương tác trực tiếp:

1. **Ticket Chat & Live Execution (Mặc định):** Luồng hội thoại giữa User và `@ba`, `@developer`, `@tester`, `@leader`, tích hợp khối Thinking thu gọn, bảng xem Code Diff xanh/đỏ, nút bấm **Xác nhận để `@leader` Commit & Push Git**, sơ đồ tiến độ thời gian thực và khung Remote Terminal.
2. **Workflow Builder (Giống Dify):** Giao diện Canvas kéo thả các khối Agent, nối dây điều kiện và tải mẫu workflow.
3. **Quản lý & Tạo mới Custom Agent:** Danh sách 4 Agent mặc định và Modal tạo thêm Agent tùy chỉnh (Tên, Vai trò, Mô tả chi tiết, Loại Agent Claude Code / Codex, Chọn Model, Phân quyền).
4. **Quản lý Project & Team:** Cấu hình thư mục `D:\VETC\FE\HNBG_DEV`, nhánh Git và thành viên.

Lưu đoạn mã dưới đây thành file **`index.html`** và mở trực tiếp bằng trình duyệt:

```
<!DOCTYPE html>
<html lang="vi" class="dark">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>BuzzFlow AI — Multi-Agent Coding & Workflow Platform</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/lucide@latest"></script>
  <script>
    tailwind.config = {
      darkMode: 'class',
      theme: {
        extend: {
          colors: {
            border: "hsl(240 3.7% 15.9%)",
            background: "hsl(240 10% 3.9%)",
            foreground: "hsl(0 0% 98%)",
            card: "hsl(240 10% 6%)",
            muted: "hsl(240 3.7% 15.9%)",
            accent: "hsl(240 3.7% 15.9%)",
          }
        }
      }
    }
  </script>
  <style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@400;500;
    body { font-family: 'Inter', sans-serif; background-color: #09090b; color: #fafafa; }
    .font-mono { font-family: 'JetBrains Mono', monospace; }
    .canvas-grid {
      background-size: 24px 24px;
      background-image: radial-gradient(circle, #27272a 1px, transparent 1px);
    }
    ::-webkit-scrollbar { width: 6px; height: 6px; }
    ::-webkit-scrollbar-track { background: #09090b; }
    ::-webkit-scrollbar-thumb { background: #27272a; border-radius: 3px; }
    ::-webkit-scrollbar-thumb:hover { background: #3f3f46; }
  </style>
</head>
<body class="h-screen w-screen overflow-hidden flex flex-col select-none">

  <!-- TOP HEADER -->
  <header class="h-14 border-b border-zinc-800 bg-zinc-950/90 px-4 flex items-center justify-between shrink-0 z-20">
```

```
  <div class="flex items-center gap-3">
    <div class="w-8 h-8 rounded-lg bg-indigo-600 flex items-center justify-center font-bold text-white shadow-lg shado
      <i data-lucide="zap" class="w-4 h-4"></i>
    </div>
    <div class="flex items-center gap-2">
      <span class="font-bold tracking-tight text-sm text-white">BuzzFlow Studio</span>
      <span class="text-xs px-2 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">ACP v2025.1</span>
    </div>
    <div class="h-4 w-[1px] bg-zinc-800 mx-2"></div>
    <!-- Project Selector -->
    <div class="flex items-center gap-2 bg-zinc-900 border border-zinc-800 rounded-md px-3 py-1 text-xs">
      <i data-lucide="folder-git-2" class="w-3.5 h-3.5 text-emerald-400"></i>
      <span class="font-medium text-zinc-200">HNBG_DEV</span>
      <span class="text-zinc-500 font-mono">D:\VETC\FE\HNBG_DEV</span>
      <span class="px-1.5 py-0.5 bg-emerald-500/10 text-emerald-400 rounded text-[10px] font-mono flex items-center ga
        <i data-lucide="git-branch" class="w-2.5 h-2.5"></i> hotfix/reconciliation
      </span>
    </div>
  </div>

  <!-- Navigation Tabs -->
  <nav class="flex items-center gap-1 bg-zinc-900 p-1 rounded-lg border border-zinc-800">
    <button onclick="switchTab('ticket')" id="tab-ticket" class="nav-btn px-3 py-1.5 rounded-md text-xs font-medium fl
      <i data-lucide="message-square-code" class="w-3.5 h-3.5 text-indigo-400"></i>
      Ticket Chat & Run
    </button>
    <button onclick="switchTab('workflow')" id="tab-workflow" class="nav-btn px-3 py-1.5 rounded-md text-xs font-mediu
      <i data-lucide="git-compare" class="w-3.5 h-3.5 text-amber-400"></i>
      Workflow Builder (Dify)
    </button>
    <button onclick="switchTab('agents')" id="tab-agents" class="nav-btn px-3 py-1.5 rounded-md text-xs font-medium fl
      <i data-lucide="bot" class="w-3.5 h-3.5 text-emerald-400"></i>
      Agents & Custom Roles
    </button>
    <button onclick="switchTab('project')" id="tab-project" class="nav-btn px-3 py-1.5 rounded-md text-xs font-medium
      <i data-lucide="settings" class="w-3.5 h-3.5 text-zinc-400"></i>
      Project & Teams
    </button>
  </nav>

  <!-- Right Status -->
  <div class="flex items-center gap-3">
    <div class="flex items-center gap-1.5 text-xs text-zinc-400 bg-zinc-900 border border-zinc-800 px-2.5 py-1 rounded
      <span class="w-2 h-2 rounded-full bg-emerald-500 animate-pulse"></span>
      <span>claude-agent-acp & codex-acp: Ready</span>
    </div>
  </div>
</header>

<!-- MAIN CONTENT AREA -->
<div class="flex-1 flex overflow-hidden">

  <!-- ============================================================ -->
  <!-- VIEW 1: TICKET CHAT & LIVE EXECUTION (DEFAULT)               -->
  <!-- ============================================================ -->
  <main id="view-ticket" class="flex-1 flex overflow-hidden">

    <!-- Left Sidebar: Ticket List -->
    <aside class="w-64 border-r border-zinc-800 bg-zinc-950/50 flex flex-col shrink-0">
      <div class="p-3 border-b border-zinc-800 flex items-center justify-between">
        <span class="text-xs font-semibold uppercase tracking-wider text-zinc-400">Tickets (HNBG_DEV)</span>
        <button onclick="createNewTicketPrompt()" class="p-1 rounded bg-indigo-600 hover:bg-indigo-500 text-white text
          <i data-lucide="plus" class="w-3 h-3"></i> Tạo mới
        </button>
      </div>
      <div class="flex-1 overflow-y-auto p-2 space-y-1.5">
        <div class="p-2.5 rounded-lg bg-zinc-900 border border-indigo-500/40 cursor-pointer">
          <div class="flex items-center justify-between mb-1">
            <span class="text-[11px] font-mono text-indigo-400">#BUG-104</span>
            <span id="sidebar-ticket-badge" class="text-[10px] px-1.5 py-0.5 rounded bg-amber-500/10 text-amber-400 bo
          </div>
          <p class="text-xs font-medium text-zinc-100 line-clamp-2">Đơ nút Xuất Excel báo cáo đối soát khi chọn khoảng
          <div class="mt-2 flex items-center gap-1 text-[10px] text-zinc-500">
            <i data-lucide="users" class="w-3 h-3"></i> @ba, @developer, @tester, @leader
```

```
      </div>
    </div>

    <div class="p-2.5 rounded-lg hover:bg-zinc-900/60 border border-transparent cursor-pointer transition">
      <div class="flex items-center justify-between mb-1">
        <span class="text-[11px] font-mono text-zinc-500">#BUG-103</span>
        <span class="text-[10px] px-1.5 py-0.5 rounded bg-emerald-500/10 text-emerald-400">Đã Push Git</span>
      </div>
      <p class="text-xs font-medium text-zinc-400 line-clamp-1">Sai định dạng tiền tệ VNĐ ở bảng tổng hợp</p>
      <div class="mt-1.5 text-[10px] font-mono text-zinc-600">Commit: 7a8f12c</div>
    </div>
  </div>
</aside>

<!-- Center: Multi-Agent Chat Timeline (70%) -->
<section class="flex-1 flex flex-col bg-zinc-950 overflow-hidden">
  <!-- Ticket Header -->
  <div class="h-12 border-b border-zinc-800 px-4 flex items-center justify-between bg-zinc-900/30">
    <div class="flex items-center gap-2">
      <span class="text-xs font-mono px-2 py-0.5 rounded bg-zinc-800 text-zinc-300">#BUG-104</span>
      <h2 class="text-sm font-semibold text-white">Đơ nút Xuất Excel báo cáo đối soát khi chọn khoảng ngày lớn</h2
    </div>
    <div class="flex items-center gap-2">
      <button onclick="simulateFullDemo()" class="text-xs px-2.5 py-1 rounded bg-zinc-800 hover:bg-zinc-700 text-z
        <i data-lucide="play" class="w-3 h-3 text-emerald-400"></i> Chạy mô phỏng từ đầu
      </button>
    </div>
  </div>

  <!-- Chat Messages Scroll Area -->
  <div id="chat-container" class="flex-1 overflow-y-auto p-4 space-y-4 select-text">

    <!-- 1. User Report Bug -->
    <div class="flex justify-end">
      <div class="max-w-xl bg-indigo-600/20 border border-indigo-500/30 rounded-xl p-3 text-xs text-zinc-100">
        <div class="flex items-center justify-between gap-4 mb-1 text-[11px] text-indigo-300 font-medium">
          <span>Nguyen Manh Hung (Reporter)</span>
          <span class="font-mono text-[10px] opacity-70">09:00 AM</span>
        </div>
        <p>@ba Ở màn hình Đối soát doanh thu, khi mình chọn ngày từ đầu tháng đến cuối tháng rồi bấm nút "Xuất Exc
      </div>
    </div>

    <!-- 2. @ba Response (Steps to Reproduce) -->
    <div class="flex gap-3 items-start max-w-3xl">
      <div class="w-8 h-8 rounded-lg bg-sky-500/20 border border-sky-500/40 flex items-center justify-center text-
      <div class="flex-1 bg-zinc-900 border border-zinc-800 rounded-xl p-3.5 space-y-2.5">
        <div class="flex items-center justify-between">
          <div class="flex items-center gap-2">
            <span class="text-xs font-bold text-sky-400">@ba</span>
            <span class="text-[10px] px-1.5 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">claude-3-5-sonnet
          </div>
          <span class="text-[10px] font-mono text-zinc-500">09:01 AM</span>
        </div>

        <!-- Tool Badge -->
        <div class="flex flex-wrap gap-1.5">
          <span class="text-[11px] font-mono px-2 py-0.5 rounded bg-zinc-950 border border-zinc-800 text-zinc-400
            <i data-lucide="file-search" class="w-3 h-3 text-sky-400"></i> Đã đọc: src/pages/Reconciliation/index.
          </span>
        </div>

        <div class="text-xs text-zinc-200 space-y-1.5 leading-relaxed">
          <p class="font-semibold text-white">📋 Kịch bản tái tạo lỗi chi tiết (Steps to Reproduce - STR):</p>
          <ul class="list-disc pl-4 space-y-1 text-zinc-300">
            <li><strong>Môi trường:</strong> Thư mục <code class="text-amber-300">D:\VETC\FE\HNBG_DEV</code> — Rou
            <li><strong>Bước 1:</strong> Chọn bộ lọc Từ ngày <code class="font-mono">01/10/2026</code> đến <code c
            <li><strong>Bước 2:</strong> Bấm nút <code class="font-mono">Xuất Excel (ExportButton.tsx)</code>.</li
            <li><strong>Kết quả thực tế (Actual):</strong> Hàm <code class="font-mono">XLSX.utils.json_to_sheet</c
            <li><strong>Kết quả kỳ vọng (Expected):</strong> Xử lý bất đồng bộ có trạng thái loading hoặc stream b
          </ul>
          <p class="text-sky-400 pt-1">👉 Chuyển giao <span class="underline">@developer</span> thực hiện sửa lỗi
        </div>
      </div>
```

```
</div>

<!-- 3. @developer Response (With Thinking & Diff Viewer) -->
<div class="flex gap-3 items-start max-w-3xl">
  <div class="w-8 h-8 rounded-lg bg-amber-500/20 border border-amber-500/40 flex items-center justify-center t
  <div class="flex-1 bg-zinc-900 border border-zinc-800 rounded-xl p-3.5 space-y-2.5">
    <div class="flex items-center justify-between">
      <div class="flex items-center gap-2">
        <span class="text-xs font-bold text-amber-400">@developer</span>
        <span class="text-[10px] px-1.5 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">claude-3-7-sonnet
      </div>
      <span class="text-[10px] font-mono text-zinc-500">09:03 AM</span>
    </div>

    <!-- Collapsible Thinking Block (9remote style) -->
    <details class="group bg-zinc-950/80 border border-zinc-800/80 rounded-lg px-3 py-1.5 text-xs">
      <summary class="cursor-pointer text-zinc-400 flex items-center justify-between font-mono text-[11px]">
        <span class="flex items-center gap-1.5">
          <i data-lucide="cpu" class="w-3.5 h-3.5 text-amber-400"></i>
          Quá trình suy nghĩ & phân tích nguyên nhân (Thinking...)
        </span>
        <i data-lucide="chevron-down" class="w-3.5 h-3.5 group-open:rotate-180 transition"></i>
      </summary>
      <p class="mt-2 text-zinc-400 font-mono text-[11px] leading-relaxed border-t border-zinc-800 pt-2">
        Kiểm tra file D:\VETC\FE\HNBG_DEV\src\pages\Reconciliation\ExportButton.tsx. Phát hiện vòng lặp map()
      </p>
    </details>

    <!-- Inline Code Diff Viewer -->
    <div class="rounded-lg border border-zinc-800 overflow-hidden font-mono text-[11px]">
      <div class="bg-zinc-950 px-3 py-1.5 border-b border-zinc-800 flex items-center justify-between text-zinc
        <span class="flex items-center gap-1.5">
          <i data-lucide="file-diff" class="w-3.5 h-3.5 text-emerald-400"></i>
          src/pages/Reconciliation/ExportButton.tsx
        </span>
        <span class="text-[10px] text-emerald-400">+12 -4 lines</span>
      </div>
      <div class="p-2.5 bg-zinc-950/60 space-y-0.5 overflow-x-auto">
        <div class="text-rose-400 bg-rose-500/10 px-2 py-0.5">- const handleExport = () => {</div>
        <div class="text-rose-400 bg-rose-500/10 px-2 py-0.5">-   const sheet = XLSX.utils.json_to_sheet(rawLa
        <div class="text-emerald-400 bg-emerald-500/10 px-2 py-0.5">+ const handleExport = async () => {</div>
        <div class="text-emerald-400 bg-emerald-500/10 px-2 py-0.5">+   setIsExporting(true);</div>
        <div class="text-emerald-400 bg-emerald-500/10 px-2 py-0.5">+   const blob = await exportService.strea
        <div class="text-emerald-400 bg-emerald-500/10 px-2 py-0.5">+   downloadFile(blob, `DoiSoat_${fromDate
        <div class="text-emerald-400 bg-emerald-500/10 px-2 py-0.5">+   setIsExporting(false);</div>
      </div>
    </div>

    <p class="text-xs text-zinc-200">Đã sửa xong logic xuất báo cáo và chạy <code class="text-emerald-400 font
  </div>
</div>

<!-- 4. @tester Response -->
<div class="flex gap-3 items-start max-w-3xl">
  <div class="w-8 h-8 rounded-lg bg-emerald-500/20 border border-emerald-500/40 flex items-center justify-cent
  <div class="flex-1 bg-zinc-900 border border-zinc-800 rounded-xl p-3.5 space-y-2.5">
    <div class="flex items-center justify-between">
      <div class="flex items-center gap-2">
        <span class="text-xs font-bold text-emerald-400">@tester</span>
        <span class="text-[10px] px-1.5 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">codex-acp (gpt-4o)
      </div>
      <span class="text-[10px] font-mono text-zinc-500">09:05 AM</span>
    </div>

    <div class="bg-zinc-950 border border-zinc-800 rounded-lg p-2.5 font-mono text-[11px] text-zinc-300">
      <div class="text-zinc-500">$ npm test -- src/pages/Reconciliation/ExportButton.test.tsx</div>
      <div class="text-emerald-400 mt-1">✓ PASS  src/pages/Reconciliation/ExportButton.test.tsx (1.42s)</div>
      <div class="text-zinc-400">  ✓ should stream excel blob without blocking UI thread (310ms)</div>
      <div class="text-zinc-400">  ✓ should disable export button while loading (85ms)</div>
    </div>

    <p class="text-xs text-zinc-200">
      <strong>Kết quả nghiệm thu: ĐẠT (PASSED).</strong> Lỗi đơ trình duyệt đã được xử lý triệt để.
      <span class="text-indigo-400">@Nguyen Manh Hung</span> vui lòng kiểm tra lại trên môi trường của bạn và
```

```
        </p>
      </div>
    </div>

  </div>

  <!-- HUMAN-IN-THE-LOOP APPROVAL BANNER -->
  <div id="approval-banner" class="mx-4 mb-3 p-3.5 rounded-xl bg-gradient-to-r from-indigo-950/90 via-zinc-900 to-
    <div class="flex items-center gap-3">
      <div class="w-9 h-9 rounded-full bg-indigo-500/20 border border-indigo-400/40 flex items-center justify-cent
        <i data-lucide="shield-check" class="w-5 h-5"></i>
      </div>
      <div>
        <div class="text-xs font-bold text-white flex items-center gap-2">
          <span>Xác nhận Nghiệm thu từ Người dùng (Human-in-the-loop Gate)</span>
          <span class="text-[10px] px-2 py-0.5 rounded-full bg-amber-500/20 text-amber-300 font-mono">PAUSED_BEFOR
        </div>
        <p class="text-xs text-zinc-300 mt-0.5">Khi bạn bấm xác nhận đã xong, <strong>@leader</strong> sẽ tự động
      </div>
    </div>
    <div class="flex items-center gap-2 shrink-0">
      <button onclick="rejectFix()" class="px-3 py-1.5 rounded-lg bg-rose-500/10 hover:bg-rose-500/20 border borde
        <i data-lucide="rotate-ccw" class="w-3.5 h-3.5"></i> Vẫn còn lỗi (Báo @developer)
      </button>
      <button onclick="approveAndTriggerLeader()" class="px-3.5 py-1.5 rounded-lg bg-emerald-600 hover:bg-emerald-
        <i data-lucide="git-commit-horizontal" class="w-4 h-4"></i> Xác nhận Đã Xong (Giao @leader Push Git)
      </button>
    </div>
  </div>

  <!-- Chat Input Bar -->
  <div class="p-3 border-t border-zinc-800 bg-zinc-900/40 flex items-center gap-2">
    <div class="flex items-center gap-1">
      <button onclick="insertTag('@ba ')" class="text-[11px] font-mono px-2 py-1 rounded bg-zinc-800 hover:bg-zinc
      <button onclick="insertTag('@developer ')" class="text-[11px] font-mono px-2 py-1 rounded bg-zinc-800 hover:
      <button onclick="insertTag('@tester ')" class="text-[11px] font-mono px-2 py-1 rounded bg-zinc-800 hover:bg-
      <button onclick="insertTag('@leader ')" class="text-[11px] font-mono px-2 py-1 rounded bg-zinc-800 hover:bg-
    </div>
    <input id="chat-input" type="text" placeholder="Nhập phản hồi cho @ba hoặc gõ '@leader CONFIRM' để duyệt..."
           onkeydown="if(event.key==='Enter') sendUserMessage()"
           class="flex-1 bg-zinc-950 border border-zinc-800 rounded-lg px-3 py-2 text-xs text-white focus:outline-
    <button onclick="sendUserMessage()" class="px-4 py-2 rounded-lg bg-indigo-600 hover:bg-indigo-500 text-white t
      <i data-lucide="send" class="w-3.5 h-3.5"></i> Gửi
    </button>
  </div>
</section>

<!-- Right Sidebar: Live Workflow Graph & Remote PTY Terminal (30%) -->
<aside class="w-96 border-l border-zinc-800 bg-zinc-950 flex flex-col shrink-0">
  <!-- Top Half: Live Workflow Progress -->
  <div class="p-3 border-b border-zinc-800 flex items-center justify-between">
    <span class="text-xs font-semibold text-zinc-300 flex items-center gap-1.5">
      <i data-lucide="activity" class="w-3.5 h-3.5 text-indigo-400"></i> Tiến độ Workflow Trực tiếp
    </span>
    <span id="live-node-status" class="text-[10px] font-mono px-2 py-0.5 rounded bg-amber-500/10 text-amber-400">N
  </div>

  <div class="p-4 space-y-2.5 bg-zinc-950/50 canvas-grid border-b border-zinc-800 overflow-y-auto max-h-[310px]">
    <!-- Step 1: BA -->
    <div id="step-ba" class="p-2.5 rounded-lg bg-zinc-900 border border-emerald-500/40 flex items-center justify-b
      <div class="flex items-center gap-2">
        <span class="w-2 h-2 rounded-full bg-emerald-400"></span>
        <span class="font-semibold text-sky-400">1. @ba</span>
        <span class="text-[10px] text-zinc-400">Xác nhận kịch bản STR</span>
      </div>
      <i data-lucide="check-circle-2" class="w-4 h-4 text-emerald-400"></i>
    </div>
    <!-- Step 2: Dev -->
    <div id="step-dev" class="p-2.5 rounded-lg bg-zinc-900 border border-emerald-500/40 flex items-center justify-
      <div class="flex items-center gap-2">
        <span class="w-2 h-2 rounded-full bg-emerald-400"></span>
        <span class="font-semibold text-amber-400">2. @developer</span>
        <span class="text-[10px] text-zinc-400">Sửa code (No commit)</span>
      </div>
```

```
        <i data-lucide="check-circle-2" class="w-4 h-4 text-emerald-400"></i>
      </div>
      <!-- Step 3: Tester -->
      <div id="step-tester" class="p-2.5 rounded-lg bg-zinc-900 border border-emerald-500/40 flex items-center justi
        <div class="flex items-center gap-2">
          <span class="w-2 h-2 rounded-full bg-emerald-400"></span>
          <span class="font-semibold text-emerald-400">3. @tester</span>
          <span class="text-[10px] text-zinc-400">Kiểm thử nghiệm thu</span>
        </div>
        <i data-lucide="check-circle-2" class="w-4 h-4 text-emerald-400"></i>
      </div>
      <!-- Step 4: User Gate -->
      <div id="step-gate" class="p-2.5 rounded-lg bg-indigo-950/60 border-2 border-indigo-500 animate-pulse flex ite
        <div class="flex items-center gap-2">
          <span class="w-2 h-2 rounded-full bg-indigo-400"></span>
          <span class="font-semibold text-white">4. User Confirm Gate</span>
        </div>
        <span class="text-[10px] font-mono text-indigo-300">Đang chờ...</span>
      </div>
      <!-- Step 5: Leader -->
      <div id="step-leader" class="p-2.5 rounded-lg bg-zinc-900/50 border border-zinc-800 flex items-center justify-
        <div class="flex items-center gap-2">
          <span class="w-2 h-2 rounded-full bg-zinc-600"></span>
          <span class="font-semibold text-purple-400">5. @leader</span>
          <span class="text-[10px] text-zinc-400">Git Commit & Push</span>
        </div>
        <i data-lucide="clock" class="w-4 h-4 text-zinc-500"></i>
      </div>
    </div>

    <!-- Bottom Half: 9remote Style PTY Terminal -->
    <div class="flex-1 flex flex-col bg-black overflow-hidden">
      <div class="px-3 py-2 border-b border-zinc-800 bg-zinc-900/80 flex items-center justify-between">
        <span class="text-[11px] font-mono text-zinc-300 flex items-center gap-1.5">
          <i data-lucide="terminal" class="w-3.5 h-3.5 text-emerald-400"></i>
          ACP / PTY Stream (D:\VETC\FE\HNBG_DEV)
        </span>
        <span class="text-[10px] font-mono text-zinc-500">JSON-RPC stdio</span>
      </div>
      <div id="terminal-output" class="flex-1 p-3 font-mono text-[11px] text-zinc-300 overflow-y-auto space-y-1 sele
        <div class="text-zinc-500">[09:00:12] Spawning claude-agent-acp in D:\VETC\FE\HNBG_DEV</div>
        <div class="text-sky-400">[09:01:05] @ba -> fs/readTextFile: src/pages/Reconciliation/index.tsx (ALLOW)</div
        <div class="text-amber-400">[09:03:19] @developer -> fs/writeTextFile: ExportButton.tsx (ALLOW)</div>
        <div class="text-rose-400">[09:03:22] Guard: Blocked git commit permission for @developer</div>
        <div class="text-emerald-400">[09:05:01] @tester -> terminal/run: npm test (PASS)</div>
        <div class="text-indigo-400">[09:05:04] LangGraph Interrupt: Waiting for user_confirmation_gate...</div>
      </div>
    </div>
  </aside>
</main>

<!-- ========================================================== -->
<!-- VIEW 2: WORKFLOW BUILDER (DIFY STYLE DRAG & DROP CANVAS)   -->
<!-- ========================================================== -->
<main id="view-workflow" class="flex-1 hidden flex overflow-hidden">
  <!-- Left Palette -->
  <aside class="w-72 border-r border-zinc-800 bg-zinc-950 p-4 flex flex-col gap-4">
    <div>
      <h3 class="text-xs font-bold uppercase tracking-wider text-zinc-400 mb-2">Khối Điều Khiển</h3>
      <div class="space-y-2">
        <div class="p-2.5 rounded-lg bg-zinc-900 border border-zinc-800 text-xs flex items-center gap-2 cursor-grab"
          <i data-lucide="play-circle" class="w-4 h-4 text-emerald-400"></i> Start Trigger (Khi tạo Ticket)
        </div>
        <div class="p-2.5 rounded-lg bg-zinc-900 border border-zinc-800 text-xs flex items-center gap-2 cursor-grab"
          <i data-lucide="split" class="w-4 h-4 text-amber-400"></i> Điều kiện Rẽ nhánh (Pass / Fail)
        </div>
        <div class="p-2.5 rounded-lg bg-zinc-900 border border-indigo-500/40 text-xs flex items-center gap-2 cursor-
          <i data-lucide="user-check" class="w-4 h-4 text-indigo-400"></i> Cổng Người Dùng Xác Nhận (Interrupt)
        </div>
      </div>
    </div>

    <div class="flex-1 overflow-y-auto">
      <div class="flex items-center justify-between mb-2">
```

```
      <h3 class="text-xs font-bold uppercase tracking-wider text-zinc-400">Danh sách Agent (Kéo vào luồng)</h3>
    </div>
    <div id="workflow-agent-palette" class="space-y-2">
      <!-- Dynamically synced with Agents list -->
    </div>
  </div>

  <button onclick="alert('Đã lưu cấu trúc đồ thị JSON (Nodes & Edges) vào PostgreSQL thành công!')" class="w-full
    <i data-lucide="save" class="w-4 h-4"></i> Lưu Đồ Thị Workflow
  </button>
</aside>

<!-- Visual Canvas -->
<section class="flex-1 canvas-grid relative overflow-auto p-10">
  <div class="absolute top-4 right-4 flex items-center gap-2 bg-zinc-900/90 border border-zinc-800 p-1.5 rounded-l
    <span class="px-2 text-zinc-400">Template: <strong>Chu trình Fix Bug 4 Agent + Git Release</strong></span>
  </div>

  <!-- Horizontal Visual Flow Representation -->
  <div class="min-w-[1150px] h-full flex items-center justify-between gap-4 relative">

    <!-- Node 1: Start -->
    <div class="w-44 p-3 rounded-xl bg-zinc-900 border-2 border-emerald-500/50 shadow-xl">
      <div class="text-[10px] font-mono text-emerald-400 uppercase">Trigger</div>
      <div class="text-xs font-bold text-white mt-0.5">User Báo Lỗi Mới</div>
      <div class="text-[11px] text-zinc-400 mt-1">Input: bug_description</div>
    </div>

    <i data-lucide="arrow-right" class="w-5 h-5 text-zinc-600 shrink-0"></i>

    <!-- Node 2: @ba -->
    <div class="w-56 p-3.5 rounded-xl bg-zinc-900 border-2 border-sky-500/60 shadow-xl">
      <div class="flex items-center justify-between">
        <span class="text-xs font-bold text-sky-400">@ba</span>
        <span class="text-[10px] font-mono bg-zinc-800 px-1.5 py-0.5 rounded">Claude Code</span>
      </div>
      <p class="text-[11px] text-zinc-300 mt-1.5">Phân tích lỗi, hỏi lại User nếu thiếu thông tin, xuất kịch bản S
      <div class="mt-2 pt-2 border-t border-zinc-800 text-[10px] font-mono text-zinc-400">Quyền: Read-Only</div>
    </div>

    <i data-lucide="arrow-right" class="w-5 h-5 text-zinc-600 shrink-0"></i>

    <!-- Node 3: @developer -->
    <div class="w-56 p-3.5 rounded-xl bg-zinc-900 border-2 border-amber-500/60 shadow-xl">
      <div class="flex items-center justify-between">
        <span class="text-xs font-bold text-amber-400">@developer</span>
        <span class="text-[10px] font-mono bg-zinc-800 px-1.5 py-0.5 rounded">Claude 3.7</span>
      </div>
      <p class="text-[11px] text-zinc-300 mt-1.5">Sửa code trong D:\VETC\FE\HNBG_DEV, chạy build kiểm tra.</p>
      <div class="mt-2 pt-2 border-t border-zinc-800 text-[10px] font-mono text-rose-400">Cấm: git commit / push</
    </div>

    <i data-lucide="arrow-right" class="w-5 h-5 text-zinc-600 shrink-0"></i>

    <!-- Node 4: @tester -->
    <div class="w-56 p-3.5 rounded-xl bg-zinc-900 border-2 border-emerald-500/60 shadow-xl">
      <div class="flex items-center justify-between">
        <span class="text-xs font-bold text-emerald-400">@tester</span>
        <span class="text-[10px] font-mono bg-zinc-800 px-1.5 py-0.5 rounded">Codex GPT-4o</span>
      </div>
      <p class="text-[11px] text-zinc-300 mt-1.5">Chạy test theo STR. Nếu Fail trả lại @developer, nếu Pass sang c
      <div class="mt-2 pt-2 border-t border-zinc-800 text-[10px] font-mono text-emerald-400">Quyền: Read + Run Tes
    </div>

    <i data-lucide="arrow-right" class="w-5 h-5 text-zinc-600 shrink-0"></i>

    <!-- Node 5: User Confirm -->
    <div class="w-52 p-3.5 rounded-xl bg-indigo-950/70 border-2 border-indigo-400 shadow-xl">
      <div class="text-[10px] font-mono text-indigo-300 uppercase">LangGraph Interrupt</div>
      <div class="text-xs font-bold text-white mt-0.5">User Xác Nhận Đã Xong</div>
      <p class="text-[11px] text-indigo-200 mt-1">Chờ người dùng bấm Confirm trước khi commit.</p>
    </div>

    <i data-lucide="arrow-right" class="w-5 h-5 text-zinc-600 shrink-0"></i>
```

```
        <!-- Node 6: @leader -->
        <div class="w-56 p-3.5 rounded-xl bg-zinc-900 border-2 border-purple-500/60 shadow-xl">
          <div class="flex items-center justify-between">
            <span class="text-xs font-bold text-purple-400">@leader</span>
            <span class="text-[10px] font-mono bg-zinc-800 px-1.5 py-0.5 rounded">Claude Code</span>
          </div>
          <p class="text-[11px] text-zinc-300 mt-1.5">Review git diff lần cuối, thực hiện git add, git commit và git p
          <div class="mt-2 pt-2 border-t border-zinc-800 text-[10px] font-mono text-purple-400">Quyền: Full Git Releas
        </div>

      </div>
    </section>
  </main>

  <!-- ============================================================ -->
  <!-- VIEW 3: AGENTS & CUSTOM AGENT CREATOR                        -->
  <!-- ============================================================ -->
  <main id="view-agents" class="flex-1 hidden overflow-y-auto p-6 bg-zinc-950">
    <div class="max-w-6xl mx-auto space-y-6">
      <div class="flex items-center justify-between">
        <div>
          <h1 class="text-lg font-bold text-white">Quản lý Đội ngũ Agent (ACP Harnesses)</h1>
          <p class="text-xs text-zinc-400 mt-0.5">Tích hợp trực tiếp với Claude Code (<code class="font-mono">claude-a
        </div>
        <button onclick="openAgentModal()" class="px-4 py-2 rounded-lg bg-indigo-600 hover:bg-indigo-500 text-white te
          <i data-lucide="plus-circle" class="w-4 h-4"></i> Tạo thêm Agent mới
        </button>
      </div>

      <!-- Agent Cards Grid -->
      <div id="agents-grid" class="grid grid-cols-1 md:grid-cols-2 gap-4">
        <!-- Rendered by JS -->
      </div>
    </div>
  </main>

  <!-- ============================================================ -->
  <!-- VIEW 4: PROJECT & TEAM SETTINGS                              -->
  <!-- ============================================================ -->
  <main id="view-project" class="flex-1 hidden overflow-y-auto p-6 bg-zinc-950">
    <div class="max-w-4xl mx-auto space-y-6">
      <div class="p-5 rounded-xl bg-zinc-900 border border-zinc-800 space-y-4">
        <h2 class="text-sm font-bold text-white flex items-center gap-2">
          <i data-lucide="folder-cog" class="w-4 h-4 text-indigo-400"></i> Cấu hình Project & Thư mục Mã nguồn Local
        </h2>
        <div class="grid grid-cols-2 gap-4 text-xs">
          <div>
            <label class="text-zinc-400 block mb-1">Tên Project</label>
            <input type="text" value="HNBG_DEV (VETC Frontend)" class="w-full bg-zinc-950 border border-zinc-800 round
          </div>
          <div>
            <label class="text-zinc-400 block mb-1">Đường dẫn Thư mục Local (ACP Working Directory)</label>
            <input type="text" value="D:\VETC\FE\HNBG_DEV" class="w-full bg-zinc-950 border border-zinc-800 rounded-lg
          </div>
          <div>
            <label class="text-zinc-400 block mb-1">Nhánh Git Mặc định để @leader Push</label>
            <input type="text" value="hotfix/reconciliation" class="w-full bg-zinc-950 border border-zinc-800 rounded-
          </div>
          <div>
            <label class="text-zinc-400 block mb-1">Team Phụ trách</label>
            <input type="text" value="Core Bugfix Squad (@ba, @developer, @tester, @leader)" class="w-full bg-zinc-950
          </div>
        </div>
      </div>
    </div>
  </main>

</div>

<!-- ============================================================ -->
<!-- MODAL: CREATE CUSTOM AGENT                                   -->
<!-- ============================================================ -->
<div id="agent-modal" class="fixed inset-0 bg-black/80 backdrop-blur-sm hidden items-center justify-center z-50 p-4">
```

```
  <div class="bg-zinc-900 border border-zinc-800 rounded-2xl max-w-lg w-full p-5 space-y-4 shadow-2xl">
    <div class="flex items-center justify-between border-b border-zinc-800 pb-3">
      <h3 class="text-sm font-bold text-white flex items-center gap-2">
        <i data-lucide="bot" class="w-4 h-4 text-indigo-400"></i> Tạo Custom Agent Mới
      </h3>
      <button onclick="closeAgentModal()" class="text-zinc-400 hover:text-white">
        <i data-lucide="x" class="w-4 h-4"></i>
      </button>
    </div>

    <div class="space-y-3 text-xs">
      <div class="grid grid-cols-2 gap-3">
        <div>
          <label class="text-zinc-300 font-medium block mb-1">1. Tên định danh Agent *</label>
          <input id="new-agent-name" type="text" placeholder="@security_auditor" class="w-full bg-zinc-950 border bord
        </div>
        <div>
          <label class="text-zinc-300 font-medium block mb-1">2. Vai trò (Role) *</label>
          <input id="new-agent-role" type="text" placeholder="Chuyên gia Bảo mật / Review" class="w-full bg-zinc-950 b
        </div>
      </div>

      <div class="grid grid-cols-2 gap-3">
        <div>
          <label class="text-zinc-300 font-medium block mb-1">3. Loại Agent (ACP Runtime) *</label>
          <select id="new-agent-runtime" onchange="updateModelDropdown()" class="w-full bg-zinc-950 border border-zinc
            <option value="claude-code">Claude Code (claude-agent-acp)</option>
            <option value="codex">OpenAI Codex (codex-acp)</option>
          </select>
        </div>
        <div>
          <label class="text-zinc-300 font-medium block mb-1">4. Chọn Model *</label>
          <select id="new-agent-model" class="w-full bg-zinc-950 border border-zinc-800 rounded-lg px-3 py-2 text-whit
            <!-- Populated by JS -->
          </select>
        </div>
      </div>

      <div>
        <label class="text-zinc-300 font-medium block mb-1">5. Mô tả chi tiết vai trò (System Prompt) *</label>
        <textarea id="new-agent-prompt" rows="4" placeholder="Mô tả chi tiết trách nhiệm, quy tắc kiểm tra mã nguồn tạ
      </div>

      <div class="p-3 rounded-lg bg-zinc-950 border border-zinc-800 space-y-2">
        <div class="font-medium text-zinc-300">Phân quyền truy cập thư mục (Dynamic Permission Guard)</div>
        <div class="flex items-center gap-4 text-[11px] text-zinc-400">
          <label class="flex items-center gap-1.5"><input id="perm-read" type="checkbox" checked class="accent-indigo-
          <label class="flex items-center gap-1.5"><input id="perm-write" type="checkbox" class="accent-indigo-500" />
          <label class="flex items-center gap-1.5"><input id="perm-term" type="checkbox" checked class="accent-indigo-
          <label class="flex items-center gap-1.5 text-rose-400"><input id="perm-git" type="checkbox" disabled /> Cấm
        </div>
      </div>
    </div>

    <div class="flex items-center justify-end gap-2 pt-2 border-t border-zinc-800">
      <button onclick="closeAgentModal()" class="px-3 py-1.5 rounded-lg bg-zinc-800 text-zinc-300 text-xs">Hủy</button
      <button onclick="saveCustomAgent()" class="px-4 py-1.5 rounded-lg bg-indigo-600 hover:bg-indigo-500 text-white t
    </div>
  </div>
</div>

<!-- JAVASCRIPT LOGIC -->
<script>
  const MODELS_BY_RUNTIME = {
    "claude-code": [
      "claude-3-7-sonnet-latest",
      "claude-3-5-sonnet-latest",
      "claude-3-5-haiku-latest"
    ],
    "codex": [
      "gpt-4o",
      "gpt-4o-mini",
      "o3-mini"
    ]
```

```
};

let agents = [
  {
    name: "@ba",
    role: "Business Analyst",
    runtime: "claude-code",
    model: "claude-3-5-sonnet-latest",
    prompt: "Tiếp nhận lỗi từ người dùng, hỏi lại nếu cần làm rõ. Xuất kịch bản tái tạo lỗi (STR) chi tiết từng bước
    permissions: "Read-Only (Cấm sửa file & Git)",
    builtin: true,
    color: "sky"
  },
  {
    name: "@developer",
    role: "Frontend Developer",
    runtime: "claude-code",
    model: "claude-3-7-sonnet-latest",
    prompt: "Đọc kịch bản STR từ @ba, định vị mã nguồn trong D:\\VETC\\FE\\HNBG_DEV, sửa lỗi và chạy build kiểm tra.
    permissions: "Read & Write (Cấm Git Commit/Push)",
    builtin: true,
    color: "amber"
  },
  {
    name: "@tester",
    role: "QA / Automation Tester",
    runtime: "codex",
    model: "gpt-4o",
    prompt: "Kiểm tra code @developer vừa sửa đối chiếu với STR của @ba. Chạy test suite, nếu Pass thì báo người dùn
    permissions: "Read & Run Test (Cấm sửa code & Git)",
    builtin: true,
    color: "emerald"
  },
  {
    name: "@leader",
    role: "Tech Lead / Git Release",
    runtime: "claude-code",
    model: "claude-3-7-sonnet-latest",
    prompt: "Chỉ hoạt động sau khi người dùng xác nhận lỗi đã sửa xong. Review git diff lần cuối, thực hiện git add,
    permissions: "Full Git Commit & Push Access",
    builtin: true,
    color: "purple"
  }
];

function renderAgents() {
  const grid = document.getElementById("agents-grid");
  const palette = document.getElementById("workflow-agent-palette");
  grid.innerHTML = "";
  palette.innerHTML = "";

  agents.forEach((ag) => {
    grid.innerHTML += `
      <div class="p-4 rounded-xl bg-zinc-900 border border-zinc-800 flex flex-col justify-between space-y-3">
        <div>
          <div class="flex items-center justify-between">
            <div class="flex items-center gap-2">
              <span class="text-sm font-bold text-${ag.color}-400 font-mono">${ag.name}</span>
              <span class="text-xs font-semibold text-white">— ${ag.role}</span>
            </div>
            <span class="text-[10px] px-2 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">${ag.builtin ? 'Mặc đị
          </div>
          <p class="text-xs text-zinc-400 mt-2 leading-relaxed">${ag.prompt}</p>
        </div>
        <div class="pt-3 border-t border-zinc-800 flex items-center justify-between text-[11px] font-mono">
          <span class="text-indigo-400">${ag.runtime} • ${ag.model}</span>
          <span class="text-zinc-500">${ag.permissions}</span>
        </div>
      </div>
    `;

    palette.innerHTML += `
      <div class="p-2.5 rounded-lg bg-zinc-900 border border-zinc-800 text-xs flex items-center justify-between curs
        <div>
```

```
          <div class="font-bold text-${ag.color}-400 font-mono">${ag.name}</div>
          <div class="text-[10px] text-zinc-400">${ag.role}</div>
        </div>
        <span class="text-[10px] font-mono px-1.5 py-0.5 bg-zinc-950 rounded text-zinc-400">${ag.model}</span>
      </div>
    `;
  });
  lucide.createIcons();
}

function switchTab(tab) {
  ['ticket', 'workflow', 'agents', 'project'].forEach(t => {
    document.getElementById(`view-${t}`).classList.add('hidden');
    const btn = document.getElementById(`tab-${t}`);
    btn.classList.remove('bg-zinc-800', 'text-white', 'shadow-sm');
    btn.classList.add('text-zinc-400');
  });
  document.getElementById(`view-${tab}`).classList.remove('hidden');
  const activeBtn = document.getElementById(`tab-${tab}`);
  activeBtn.classList.add('bg-zinc-800', 'text-white', 'shadow-sm');
  activeBtn.classList.remove('text-zinc-400');
}

function approveAndTriggerLeader() {
  const chat = document.getElementById("chat-container");
  const banner = document.getElementById("approval-banner");
  const term = document.getElementById("terminal-output");

  banner.classList.add("hidden");

  // 1. Add User Confirmation Bubble
  chat.innerHTML += `
    <div class="flex justify-end">
      <div class="max-w-xl bg-emerald-600/20 border border-emerald-500/40 rounded-xl p-3 text-xs text-zinc-100">
        <div class="flex items-center justify-between gap-4 mb-1 text-[11px] text-emerald-300 font-medium">
          <span>Nguyen Manh Hung (Confirmed)</span>
          <span class="font-mono text-[10px]">Vừa xong</span>
        </div>
        <p>@leader CONFIRM — Mình đã kiểm tra trên môi trường local, xuất file Excel mượt và không còn bị đơ nữa. Nh
      </div>
    </div>
  `;

  // Update right sidebar workflow tracker
  document.getElementById("step-gate").className = "p-2.5 rounded-lg bg-zinc-900 border border-emerald-500/40 flex i
  document.getElementById("step-gate").innerHTML = `<div class="flex items-center gap-2"><span class="w-2 h-2 rounde

  const leaderStep = document.getElementById("step-leader");
  leaderStep.className = "p-2.5 rounded-lg bg-purple-950/50 border-2 border-purple-500 animate-pulse flex items-cent
  document.getElementById("live-node-status").innerText = "Node: @leader (Git Push)";

  term.innerHTML += `<div class="text-purple-400">[09:08:10] User Approved! Resuming LangGraph -> Node @leader</div>
  term.innerHTML += `<div class="text-zinc-300">[09:08:11] @leader -> git add src/pages/Reconciliation/ExportButton.
  term.innerHTML += `<div class="text-emerald-400">[09:08:13] @leader -> git commit -m "fix(reconciliation): stream
  term.innerHTML += `<div class="text-emerald-400">[09:08:15] @leader -> git push origin hotfix/reconciliation (SUCC
  term.scrollTop = term.scrollHeight;

  setTimeout(() => {
    chat.innerHTML += `
      <div class="flex gap-3 items-start max-w-3xl">
        <div class="w-8 h-8 rounded-lg bg-purple-500/20 border border-purple-500/40 flex items-center justify-center
        <div class="flex-1 bg-zinc-900 border border-purple-500/40 rounded-xl p-3.5 space-y-2.5">
          <div class="flex items-center justify-between">
            <div class="flex items-center gap-2">
              <span class="text-xs font-bold text-purple-400">@leader</span>
              <span class="text-[10px] px-1.5 py-0.5 rounded bg-zinc-800 text-zinc-400 font-mono">claude-3-7-sonnet
            </div>
            <span class="text-[10px] font-mono text-emerald-400">COMPLETED</span>
          </div>
          <div class="bg-zinc-950 border border-zinc-800 rounded-lg p-2.5 font-mono text-[11px] text-zinc-300 space-
            <div class="text-zinc-500">$ git commit -m "fix(reconciliation): stream excel export to prevent UI freez
            <div class="text-emerald-400">[hotfix/reconciliation 9f4d21a] 1 file changed, 12 insertions(+), 4 deleti
            <div class="text-zinc-500">$ git push origin hotfix/reconciliation</div>
            <div class="text-emerald-400">To https://github.com/vetc/hnbg_fe.git (b21d45e..9f4d21a)</div>
```

```
          </div>
          <p class="text-xs text-zinc-200">✅ Đã review code, tạo commit <code class="font-mono text-purple-300">9f4
        </div>
      </div>
    `;
    leaderStep.className = "p-2.5 rounded-lg bg-zinc-900 border border-emerald-500/40 flex items-center justify-betw
    document.getElementById("live-node-status").innerText = "Status: COMPLETED";
    document.getElementById("sidebar-ticket-badge").innerText = "Đã Push Git";
    document.getElementById("sidebar-ticket-badge").className = "text-[10px] px-1.5 py-0.5 rounded bg-emerald-500/10
    chat.scrollTop = chat.scrollHeight;
    lucide.createIcons();
  }, 600);
}

function rejectFix() {
  const reason = prompt("Nhập chi tiết điểm chưa đạt để @developer sửa lại:");
  if (!reason) return;
  const chat = document.getElementById("chat-container");
  chat.innerHTML += `
    <div class="flex justify-end">
      <div class="max-w-xl bg-rose-600/20 border border-rose-500/40 rounded-xl p-3 text-xs text-zinc-100">
        <p><strong>Yêu cầu sửa lại:</strong> ${reason}</p>
      </div>
    </div>
    <div class="text-center text-[11px] font-mono text-amber-400 py-1">🔄 LangGraph đã lùi luồng về Node @developer
  `;
  chat.scrollTop = chat.scrollHeight;
}

function insertTag(tag) {
  const input = document.getElementById("chat-input");
  input.value = tag + input.value;
  input.focus();
}

function sendUserMessage() {
  const input = document.getElementById("chat-input");
  const text = input.value.trim();
  if (!text) return;

  if (text.toUpperCase().includes("CONFIRM")) {
    input.value = "";
    approveAndTriggerLeader();
    return;
  }

  const chat = document.getElementById("chat-container");
  chat.innerHTML += `
    <div class="flex justify-end">
      <div class="max-w-xl bg-indigo-600/20 border border-indigo-500/30 rounded-xl p-3 text-xs text-zinc-100">
        <p>${text}</p>
      </div>
    </div>
  `;
  input.value = "";
  chat.scrollTop = chat.scrollHeight;
}

function openAgentModal() {
  updateModelDropdown();
  document.getElementById("agent-modal").classList.remove("hidden");
  document.getElementById("agent-modal").classList.add("flex");
}

function closeAgentModal() {
  document.getElementById("agent-modal").classList.add("hidden");
  document.getElementById("agent-modal").classList.remove("flex");
}

function updateModelDropdown() {
  const runtime = document.getElementById("new-agent-runtime").value;
  const modelSelect = document.getElementById("new-agent-model");
  modelSelect.innerHTML = MODELS_BY_RUNTIME[runtime]
    .map(m => `<option value="${m}">${m}</option>`)
```

```
        .join("");
    }

    function saveCustomAgent() {
      let name = document.getElementById("new-agent-name").value.trim();
      const role = document.getElementById("new-agent-role").value.trim();
      const runtime = document.getElementById("new-agent-runtime").value;
      const model = document.getElementById("new-agent-model").value;
      const prompt = document.getElementById("new-agent-prompt").value.trim();

      if (!name || !role || !prompt) {
        alert("Vui lòng nhập đầy đủ Tên, Vai trò và Mô tả chi tiết vai trò!");
        return;
      }
      if (!name.startsWith("@")) name = "@" + name;

      agents.push({
        name,
        role,
        runtime,
        model,
        prompt,
        permissions: "Custom Guard (No Git Push)",
        builtin: false,
        color: "indigo"
      });

      renderAgents();
      closeAgentModal();
    }

    function createNewTicketPrompt() {
      const desc = prompt("Nhập mô tả lỗi mới cho @ba phân tích:");
      if (!desc) return;
      document.getElementById("chat-container").innerHTML = `
        <div class="flex justify-end">
          <div class="max-w-xl bg-indigo-600/20 border border-indigo-500/30 rounded-xl p-3 text-xs text-zinc-100">
            <p>@ba ${desc}</p>
          </div>
        </div>
        <div class="flex gap-3 items-start max-w-3xl">
          <div class="w-8 h-8 rounded-lg bg-sky-500/20 border border-sky-500/40 flex items-center justify-center text-sk
          <div class="flex-1 bg-zinc-900 border border-zinc-800 rounded-xl p-3.5 text-xs text-zinc-200">
            <div class="font-bold text-sky-400 mb-1">@ba đang đọc mã nguồn tại D:\\VETC\\FE\\HNBG_DEV...</div>
            <p>Chào bạn, để mình lập kịch bản tái tạo lỗi (STR) chính xác cho @developer, bạn cho mình hỏi thêm: Lỗi này
          </div>
        </div>
      `;
    }

    function simulateFullDemo() {
      location.reload();
    }

    window.onload = () => {
      renderAgents();
      lucide.createIcons();
    };
  </script>
</body>
</html>
```

Các tính năng nổi bật có thể bấm thử ngay trên file HTML này:

- **Duyệt & kích hoạt `@leader` đẩy Git:** Ở cuối màn hình chat, bấm nút màu xanh **"Xác nhận Đã Xong (Giao @leader Push Git)"** (hoặc gõ `@leader CONFIRM` vào ô chat), bạn sẽ thấy đồ thị bên phải nhảy sang Node `@leader`, cửa sổ Terminal chạy lệnh `git add/commit/push` và `@leader` báo cáo mã Commit Hash.
- **Tạo thêm Custom Agent:** Chuyển sang tab **"Agents & Custom Roles"** trên thanh menu -> bấm **"Tạo thêm Agent mới"**, chọn giữa `Claude Code` và `OpenAI Codex` (danh sách Model sẽ tự động đổi theo) -> sau khi lưu, Agent mới sẽ xuất hiện trong cả danh sách lẫn thanh kéo thả của **Workflow Builder**.

User prompt: tham khảo giao diện của buzz, thiết kế đơn giản hơn

Response: Giao diện dưới đây được thiết kế lại theo đúng phong cách tối giản, sáng sủa và liền mạch của **Buzz Desktop**:

- **Bố cục 2 cột tinh gọn:** Chỉ gồm thanh Sidebar màu kem ấm (`#EBE9D8`) bên trái và khung nội dung trắng tinh (`#FFFFFF`) bo góc bên phải, loại bỏ hoàn toàn các bảng điều khiển rườm rà.
- **Trải nghiệm hội thoại phẳng (Flat Timeline):** Tin nhắn của người dùng và các Agent (`@ba`, `@developer`, `@tester`, `@leader`) hiển thị tự nhiên như đồng nghiệp đang trò chuyện trong channel, sử dụng chip tag xám bo tròn (`@ba`, `@developer`) kèm nút emoji reaction (👀 1, 💬 1) và thanh trạng thái nhỏ gọn dưới đáy (🚧 `@leader: Ready to commit`).
- **Tích hợp đầy đủ tính năng trong giao diện tối giản:** Bấm vào mục **Agents** hoặc **Workflow** ở Sidebar bên trái để tạo thêm Custom Agent hoặc xem luồng xử lý của dự án `D:\VETC\FE\HNBG_DEV`.

Lưu đoạn mã dưới đây thành file **`buzz-minimal.html`** và mở trực tiếp trên trình duyệt:

```
<!DOCTYPE html>
<html lang="vi">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Buzz — Minimalist Multi-Agent Workspace</title>
  <script src="https://cdn.tailwindcss.com"></script>
  <script src="https://unpkg.com/lucide@latest"></script>
  <style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&family=JetBrains+Mono:wght@400;500&
    body {
      font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;
      background-color: #EBE9D8;
      color: #1C1C1A;
    }
    .font-mono { font-family: 'JetBrains Mono', monospace; }
    .agent-pill {
      display: inline-flex;
      align-items: center;
      gap: 3px;
      background-color: #EFECE1;
      border: 1px solid #DFDDD0;
      padding: 1px 6px;
      border-radius: 6px;
      font-weight: 600;
      font-size: 12px;
      color: #2C2C2A;
    }
    ::-webkit-scrollbar { width: 5px; height: 5px; }
    ::-webkit-scrollbar-track { background: transparent; }
    ::-webkit-scrollbar-thumb { background: #D5D3C2; border-radius: 99px; }
  </style>
</head>
<body class="h-screen w-screen overflow-hidden flex p-2.5 select-none">

  <!-- ========================================================= -->
  <!-- LEFT SIDEBAR (Buzz Warm Cream Style)                      -->
  <!-- ========================================================= -->
  <aside class="w-60 flex flex-col justify-between pr-2.5 shrink-0 text-[13px]">
    <div class="space-y-4">
      <!-- macOS Window Controls & Search -->
      <div class="space-y-3 px-2 pt-1">
        <div class="flex items-center justify-between">
          <div class="flex items-center gap-1.5">
            <span class="w-3 h-3 rounded-full bg-[#FF5F56] border border-black/10"></span>
            <span class="w-3 h-3 rounded-full bg-[#FFBD2E] border border-black/10"></span>
            <span class="w-3 h-3 rounded-full bg-[#27C93F] border border-black/10"></span>
          </div>
          <span class="text-[11px] font-mono text-stone-500">ACP Connected</span>
        </div>

        <div class="flex items-center justify-between bg-[#DFDDD0]/70 hover:bg-[#DFDDD0] transition px-2.5 py-1.5 rounde
          <div class="flex items-center gap-2 text-xs">
            <i data-lucide="search" class="w-3.5 h-3.5"></i>
            <span>Tìm kiếm mọi thứ</span>
          </div>
          <span class="text-[10px] font-mono text-stone-400">⌘K</span>
        </div>
      </div>
```

```
    <!-- Top Menu: Inbox & Agents -->
    <div class="space-y-0.5 px-1">
      <button onclick="switchView('chat')" id="nav-chat" class="w-full flex items-center justify-between px-2.5 py-1.5
        <span class="flex items-center gap-2"><i data-lucide="inbox" class="w-4 h-4 text-stone-600"></i> Hộp thư & Tic
        <span class="text-[11px] font-mono bg-stone-800 text-white px-1.5 py-0.2 rounded-full">1</span>
      </button>
      <button onclick="switchView('agents')" id="nav-agents" class="w-full flex items-center justify-between px-2.5 py
        <span class="flex items-center gap-2"><i data-lucide="bot" class="w-4 h-4 text-stone-600"></i> Quản lý Agents<
        <span id="agent-count-badge" class="text-[11px] text-stone-500 font-mono">4</span>
      </button>
      <button onclick="switchView('workflow')" id="nav-workflow" class="w-full flex items-center justify-between px-2.
        <span class="flex items-center gap-2"><i data-lucide="git-merge" class="w-4 h-4 text-stone-600"></i> Luồng Wor
      </button>
    </div>

    <!-- Project Section: HNBG_DEV -->
    <div class="space-y-1 px-1">
      <div class="px-2.5 flex items-center justify-between text-[11px] font-semibold text-stone-500">
        <span>🛠️ HNBG_DEV (VETC)</span>
        <button onclick="newTicket()" title="Tạo ticket báo lỗi mới" class="hover:text-stone-900"><i data-lucide="plus
      </div>
      <button onclick="switchView('chat')" class="w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg bg
        <span class="truncate"># bug-xuat-excel</span>
        <span id="channel-timer" class="text-[10px] font-mono bg-stone-700/15 px-1.5 py-0.5 rounded text-stone-700">Ch
      </button>
      <button onclick="switchView('chat')" class="w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg te
        <span class="truncate"># bug-format-tien-te</span>
        <span class="w-1.5 h-1.5 rounded-full bg-stone-500"></span>
      </button>
      <button onclick="switchView('chat')" class="w-full flex items-center justify-between px-2.5 py-1.5 rounded-lg te
        <span class="truncate"># general-dev</span>
      </button>
    </div>

    <!-- Active AI Team Members in Project -->
    <div class="space-y-1 px-1">
      <div class="px-2.5 text-[11px] font-semibold text-stone-500">🐝 Đội ngũ Agent trong kênh</div>
      <div id="sidebar-agents-list" class="space-y-0.5">
        <!-- Rendered by JS -->
      </div>
    </div>
  </div>

  <!-- Bottom User Profile & Workspace Path -->
  <div class="px-2.5 py-2 rounded-xl hover:bg-[#DFDDD0]/60 transition flex items-center justify-between cursor-pointer
    <div class="flex items-center gap-2.5 min-w-0">
      <div class="relative shrink-0">
        <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
        <span class="w-2.5 h-2.5 rounded-full bg-emerald-500 border-2 border-[#EBE9D8] absolute -bottom-0.5 -right-0.5
      </div>
      <div class="truncate">
        <div class="text-xs font-semibold text-stone-900 truncate">Nguyen Manh Hung</div>
        <div class="text-[10px] font-mono text-stone-500 truncate">D:\VETC\FE\HNBG_DEV</div>
      </div>
    </div>
  </div>
</aside>

<!-- ========================================================== -->
<!-- MAIN WHITE ROUNDED PANEL (Buzz Clean Content Card)        -->
<!-- ========================================================== -->
<main class="flex-1 bg-white rounded-2xl border border-stone-300/80 shadow-sm flex flex-col overflow-hidden">

  <!-- ---------------------------------------------------- -->
  <!-- VIEW 1: CHANNEL CHAT (DEFAULT BUZZ VIEW)               -->
  <!-- ---------------------------------------------------- -->
  <div id="view-chat" class="flex-1 flex flex-col overflow-hidden">

    <!-- Top Channel Bar -->
    <header class="h-12 border-b border-stone-200 px-5 flex items-center justify-between shrink-0">
      <div class="flex items-center gap-2">
        <span class="font-bold text-sm text-stone-900"># bug-xuat-excel</span>
        <span class="text-xs text-stone-400">•</span>
```

```
    <span class="text-xs font-mono text-stone-500 bg-stone-100 px-2 py-0.5 rounded">D:\VETC\FE\HNBG_DEV (hotfix/re
  </div>
  <div class="flex items-center gap-2">
    <div class="flex items-center gap-1 px-2.5 py-1 rounded-lg border border-stone-200 text-xs text-stone-600">
      <i data-lucide="users" class="w-3.5 h-3.5"></i>
      <span id="member-count">5</span>
    </div>
    <button onclick="switchView('workflow')" title="Xem quy trình" class="p-1.5 rounded-lg border border-stone-200
      <i data-lucide="sliders-horizontal" class="w-3.5 h-3.5"></i>
    </button>
  </div>
</header>

<!-- Flat Message Stream (Like Buzz / Slack) -->
<div id="messages-stream" class="flex-1 overflow-y-auto px-6 py-4 space-y-5 select-text text-[13.5px] leading-rela

  <!-- User Message -->
  <div class="flex items-start gap-3">
    <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
    <div class="flex-1 space-y-1">
      <div class="flex items-baseline gap-2">
        <span class="font-bold text-stone-900 text-xs">Nguyen Manh Hung</span>
        <span class="text-[11px] text-stone-400">9:15 AM</span>
      </div>
      <p class="text-stone-800">
        <span class="agent-pill"><i data-lucide="bot" class="w-3 h-3"></i> ba</span>
        Ở màn hình Đối soát doanh thu, khi chọn ngày từ đầu tháng đến cuối tháng rồi bấm nút "Xuất Excel" thì trìn
      </p>
    </div>
  </div>

  <!-- Divider NEW -->
  <div class="relative flex items-center py-1">
    <div class="flex-grow border-t border-stone-200"></div>
    <span class="shrink-0 mx-3 text-[10px] font-bold tracking-wider text-stone-400 uppercase">Tiến trình Đội ngũ A
    <div class="flex-grow border-t border-stone-200"></div>
  </div>

  <!-- @ba Message -->
  <div class="flex items-start gap-3">
    <div class="w-8 h-8 rounded-full bg-amber-100 border border-amber-300 text-amber-800 flex items-center justify
    <div class="flex-1 space-y-1.5">
      <div class="flex items-baseline gap-2">
        <span class="font-bold text-stone-900 text-xs">ba</span>
        <span class="text-[11px] font-mono text-stone-400">claude-3-5-sonnet</span>
        <span class="text-[11px] text-stone-400">9:16 AM</span>
      </div>
      <p class="text-stone-800">Mình đã kiểm tra mã nguồn trong <code class="bg-stone-100 px-1.5 py-0.5 rounded fo
      <ol class="list-decimal pl-5 space-y-1.5 text-stone-800">
        <li>
          <strong>Điều kiện & Màn hình</strong>
          <ul class="list-disc pl-5 text-stone-600 text-xs mt-0.5 space-y-0.5">
            <li>Truy cập đường dẫn <code class="font-mono">/reconciliation/report</code>.</li>
            <li>Chọn khoảng thời gian từ <code class="font-mono">01/10/2026</code> đến <code class="font-mono">22/
          </ul>
        </li>
        <li>
          <strong>Thao tác gây lỗi</strong>
          <ul class="list-disc pl-5 text-stone-600 text-xs mt-0.5 space-y-0.5">
            <li>Bấm vào nút <strong>Xuất Excel</strong> (<code class="font-mono">src/pages/Reconciliation/ExportBu
            <li>Hàm <code class="font-mono">XLSX.utils.json_to_sheet</code> xử lý đồng bộ trên Main Thread gây tre
          </ul>
        </li>
        <li>
          <strong>Kết quả mong muốn</strong>
          <ul class="list-disc pl-5 text-stone-600 text-xs mt-0.5 space-y-0.5">
            <li>Chuyển sang xử lý bất đồng bộ (async stream blob) và khóa nút bấm kèm trạng thái loading trong lúc
          </ul>
        </li>
      </ol>
      <p class="pt-1 text-stone-800">
        <span class="agent-pill"><i data-lucide="bot" class="w-3 h-3"></i> developer</span>
        bàn giao cho bạn sửa lỗi theo kịch bản trên nhé.
      </p>
```

```
      <div class="flex items-center gap-1.5 pt-1">
        <span class="px-2 py-0.5 rounded-full bg-stone-100 border border-stone-200 text-[11px] text-stone-600">👀
        <span class="px-2 py-0.5 rounded-full bg-stone-100 border border-stone-200 text-[11px] text-stone-600">✅
      </div>
    </div>
  </div>

  <!-- @developer Message -->
  <div class="flex items-start gap-3">
    <div class="w-8 h-8 rounded-full bg-sky-100 border border-sky-300 text-sky-800 flex items-center justify-cente
    <div class="flex-1 space-y-1.5">
      <div class="flex items-baseline gap-2">
        <span class="font-bold text-stone-900 text-xs">developer</span>
        <span class="text-[11px] font-mono text-stone-400">claude-3-7-sonnet</span>
        <span class="text-[11px] text-stone-400">9:18 AM</span>
      </div>
      <p class="text-stone-800">Đã sửa xong lỗi trên file <code class="bg-stone-100 px-1.5 py-0.5 rounded font-mon

      <!-- Clean Minimal Diff Collapsible -->
      <details class="bg-stone-50 border border-stone-200 rounded-xl px-3.5 py-2 text-xs max-w-2xl">
        <summary class="cursor-pointer font-medium text-stone-700 flex items-center justify-between">
          <span class="font-mono text-[11px]">📄 Xem thay đổi mã nguồn: ExportButton.tsx (+8 -2)</span>
          <span class="text-[11px] text-stone-400">Bấm để mở</span>
        </summary>
        <div class="mt-2 pt-2 border-t border-stone-200 font-mono text-[11px] space-y-0.5">
          <div class="text-rose-600 bg-rose-50 px-2 py-0.5 rounded">- const handleExport = () => XLSX.utils.json_t
          <div class="text-emerald-700 bg-emerald-50 px-2 py-0.5 rounded">+ const handleExport = async () => {</di
          <div class="text-emerald-700 bg-emerald-50 px-2 py-0.5 rounded">+   setIsExporting(true);</div>
          <div class="text-emerald-700 bg-emerald-50 px-2 py-0.5 rounded">+   await exportService.downloadStream({
          <div class="text-emerald-700 bg-emerald-50 px-2 py-0.5 rounded">+   setIsExporting(false);</div>
          <div class="text-emerald-700 bg-emerald-50 px-2 py-0.5 rounded">+ };</div>
        </div>
      </details>

      <p class="pt-1 text-stone-800">
        <span class="agent-pill"><i data-lucide="bot" class="w-3 h-3"></i> tester</span>
        nhờ bạn kiểm tra nghiệm thu lại bản sửa này.
      </p>
    </div>
  </div>

  <!-- @tester Message -->
  <div class="flex items-start gap-3">
    <div class="w-8 h-8 rounded-full bg-emerald-100 border border-emerald-300 text-emerald-800 flex items-center j
    <div class="flex-1 space-y-1.5">
      <div class="flex items-baseline gap-2">
        <span class="font-bold text-stone-900 text-xs">tester</span>
        <span class="text-[11px] font-mono text-stone-400">codex-acp (gpt-4o)</span>
        <span class="text-[11px] text-stone-400">9:20 AM</span>
      </div>
      <p class="text-stone-800">
        Đã chạy kiểm thử tự động trên <code class="bg-stone-100 px-1.5 py-0.5 rounded font-mono text-xs">ExportBut
      </p>
      <p class="text-stone-800">
        Anh <span class="agent-pill">@Nguyen Manh Hung</span> kiểm tra lại nhé. Nếu đã ổn, hãy bấm nút xác nhận bê
      </p>

      <!-- Minimalist Inline Confirmation Action Bar -->
      <div id="confirm-box" class="mt-2 p-3 rounded-xl bg-[#F7F6EE] border border-[#DFDDD0] flex items-center just
        <span class="text-xs font-medium text-stone-700">Bạn xác nhận lỗi đã được sửa xong?</span>
        <div class="flex items-center gap-2">
          <button onclick="rejectBugFix()" class="px-2.5 py-1 rounded-lg border border-stone-300 bg-white hover:bg
            Chưa đạt, sửa lại
          </button>
          <button onclick="confirmAndTriggerLeader()" class="px-3 py-1 rounded-lg bg-stone-900 hover:bg-stone-800
            <i data-lucide="check" class="w-3.5 h-3.5"></i> Xác nhận (Báo @leader Push Git)
          </button>
        </div>
      </div>
    </div>
  </div>

</div>
```

```
  <!-- Bottom Compose Box (Exact Buzz Style) -->
  <div class="px-5 pt-2 pb-1">
    <div class="border border-stone-300 rounded-xl bg-white p-3 focus-within:border-stone-500 transition shadow-sm">
      <input id="chat-input" type="text" placeholder="Nhắn tin vào #bug-xuat-excel (gõ @ba, @developer, @tester, @le
             onkeydown="if(event.key==='Enter') sendChatMessage()"
             class="w-full text-sm text-stone-900 placeholder-stone-400 focus:outline-none" />
      <div class="flex items-center justify-between mt-2.5 pt-1">
        <div class="flex items-center gap-2 text-stone-500">
          <button onclick="quickMention('@ba ')" title="Tag @ba" class="hover:text-stone-900 text-xs font-mono px-1.
          <button onclick="quickMention('@developer ')" title="Tag @developer" class="hover:text-stone-900 text-xs f
          <button onclick="quickMention('@tester ')" title="Tag @tester" class="hover:text-stone-900 text-xs font-mo
          <button onclick="quickMention('@leader ')" title="Tag @leader" class="hover:text-stone-900 text-xs font-mo
        </div>
        <button onclick="sendChatMessage()" class="w-7 h-7 rounded-full bg-stone-400 hover:bg-stone-900 text-white f
          <i data-lucide="arrow-up" class="w-4 h-4"></i>
        </button>
      </div>
    </div>

    <!-- Bottom Agent Working Status Footer (Exact Buzz Bottom Bar) -->
    <div class="h-7 flex items-center justify-between px-1 text-[11px] text-stone-600 font-medium">
      <div class="flex items-center gap-1.5">
        <span>🚀</span>
        <span id="footer-agent-status"><strong>leader:</strong> Đang chờ người dùng xác nhận để commit & push Git</s
      </div>
      <span class="font-mono text-[10px] text-stone-400">claude-agent-acp • codex-acp</span>
    </div>
  </div>

</div>

<!-- ---------------------------------------------------------- -->
<!-- VIEW 2: MANAGE & CREATE CUSTOM AGENTS                      -->
<!-- ---------------------------------------------------------- -->
<div id="view-agents" class="flex-1 hidden flex-col overflow-y-auto p-6 space-y-6">
  <div class="flex items-center justify-between border-b border-stone-200 pb-4">
    <div>
      <h2 class="text-base font-bold text-stone-900">Danh sách Agents trong Workspace</h2>
      <p class="text-xs text-stone-500 mt-0.5">Ngoài 4 Agent mặc định, bạn có thể tạo thêm Agent mới nối với Claude
    </div>
    <button onclick="toggleCreateAgentForm(true)" class="px-3.5 py-2 rounded-xl bg-stone-900 hover:bg-stone-800 text
      <i data-lucide="plus" class="w-4 h-4"></i> Tạo thêm Agent
    </button>
  </div>

  <!-- Simple Create Agent Form (Hidden by default) -->
  <div id="create-agent-card" class="hidden p-5 rounded-2xl bg-[#F7F6EE] border border-[#DFDDD0] space-y-4">
    <div class="flex items-center justify-between">
      <span class="text-xs font-bold uppercase tracking-wider text-stone-600">Thêm Agent Tùy Chỉnh Mới</span>
      <button onclick="toggleCreateAgentForm(false)" class="text-xs text-stone-500 hover:text-stone-900">Đóng</butto
    </div>
    <div class="grid grid-cols-2 gap-3 text-xs">
      <div>
        <label class="block text-stone-600 font-medium mb-1">1. Tên Agent (Handle)</label>
        <input id="f-name" type="text" placeholder="@security_review" class="w-full px-3 py-2 rounded-lg border bord
      </div>
      <div>
        <label class="block text-stone-600 font-medium mb-1">2. Vai trò</label>
        <input id="f-role" type="text" placeholder="Kiểm tra bảo mật & hiệu năng" class="w-full px-3 py-2 rounded-lg
      </div>
      <div>
        <label class="block text-stone-600 font-medium mb-1">3. Loại Agent (ACP)</label>
        <select id="f-runtime" onchange="syncModels()" class="w-full px-3 py-2 rounded-lg border border-stone-300 bg
          <option value="claude-code">Claude Code (claude-agent-acp)</option>
          <option value="codex">OpenAI Codex (codex-acp)</option>
        </select>
      </div>
      <div>
        <label class="block text-stone-600 font-medium mb-1">4. Chọn Model</label>
        <select id="f-model" class="w-full px-3 py-2 rounded-lg border border-stone-300 bg-white font-mono"></select
      </div>
    </div>
    <div class="text-xs">
```

```
        <label class="block text-stone-600 font-medium mb-1">5. Mô tả chi tiết vai trò (System Prompt)</label>
        <textarea id="f-prompt" rows="3" placeholder="Nhập hướng dẫn chi tiết cách Agent làm việc trên thư mục D:\VETC
      </div>
      <div class="flex justify-end gap-2">
        <button onclick="toggleCreateAgentForm(false)" class="px-3 py-1.5 rounded-lg border border-stone-300 bg-white
        <button onclick="addCustomAgent()" class="px-4 py-1.5 rounded-lg bg-stone-900 text-white text-xs font-medium">
      </div>
    </div>

    <!-- Agent List Cards -->
    <div id="agents-list-grid" class="grid grid-cols-1 md:grid-cols-2 gap-3.5"></div>
  </div>

  <!-- ------------------------------------------------------- -->
  <!-- VIEW 3: MINIMAL WORKFLOW PIPELINE                       -->
  <!-- ------------------------------------------------------- -->
  <div id="view-workflow" class="flex-1 hidden flex-col overflow-y-auto p-6 space-y-6">
    <div class="border-b border-stone-200 pb-4">
      <h2 class="text-base font-bold text-stone-900">Quy trình Phối hợp Tự động (Workflow)</h2>
      <p class="text-xs text-stone-500 mt-0.5">Thư mục áp dụng: <code class="font-mono bg-stone-100 px-1.5 py-0.5 roun
    </div>

    <div class="space-y-3 max-w-2xl">
      <div class="p-4 rounded-xl border border-stone-200 bg-stone-50/60 flex items-center justify-between">
        <div class="flex items-center gap-3">
          <span class="w-6 h-6 rounded-full bg-stone-900 text-white text-xs flex items-center justify-center font-bold
          <div>
            <div class="text-xs font-bold text-stone-900">@ba — Xác nhận kịch bản tái tạo lỗi (Step-by-step)</div>
            <div class="text-[11px] text-stone-500">Hỏi lại người dùng nếu chưa rõ, xuất tài liệu STR chi tiết.</div>
          </div>
        </div>
        <span class="text-[11px] font-mono bg-white border border-stone-200 px-2 py-0.5 rounded">Read-only</span>
      </div>

      <div class="p-4 rounded-xl border border-stone-200 bg-stone-50/60 flex items-center justify-between">
        <div class="flex items-center gap-3">
          <span class="w-6 h-6 rounded-full bg-stone-900 text-white text-xs flex items-center justify-center font-bold
          <div>
            <div class="text-xs font-bold text-stone-900">@developer — Sửa lỗi trực tiếp trên mã nguồn</div>
            <div class="text-[11px] text-stone-500">Sửa file trong D:\VETC\FE\HNBG_DEV, không được phép commit.</div>
          </div>
        </div>
        <span class="text-[11px] font-mono bg-white border border-stone-200 px-2 py-0.5 rounded">Read / Write</span>
      </div>

      <div class="p-4 rounded-xl border border-stone-200 bg-stone-50/60 flex items-center justify-between">
        <div class="flex items-center gap-3">
          <span class="w-6 h-6 rounded-full bg-stone-900 text-white text-xs flex items-center justify-center font-bold
          <div>
            <div class="text-xs font-bold text-stone-900">@tester — Kiểm thử nghiệm thu & Báo người dùng</div>
            <div class="text-[11px] text-stone-500">Chạy test theo kịch bản của BA, nếu đạt thì mời người dùng xác nhậ
          </div>
        </div>
        <span class="text-[11px] font-mono bg-white border border-stone-200 px-2 py-0.5 rounded">Run Test</span>
      </div>

      <div class="p-4 rounded-xl border-2 border-stone-900 bg-[#F7F6EE] flex items-center justify-between">
        <div class="flex items-center gap-3">
          <span class="w-6 h-6 rounded-full bg-stone-900 text-white text-xs flex items-center justify-center font-bold
          <div>
            <div class="text-xs font-bold text-stone-900">@leader — Review & Git Commit / Push</div>
            <div class="text-[11px] text-stone-600">Chỉ kích hoạt khi người dùng xác nhận lỗi đã hết.</div>
          </div>
        </div>
        <span class="text-[11px] font-mono bg-stone-900 text-white px-2 py-0.5 rounded">Git Push</span>
      </div>
    </div>
  </div>

</main>

<script>
  const MODELS = {
    "claude-code": ["claude-3-7-sonnet-latest", "claude-3-5-sonnet-latest", "claude-3-5-haiku-latest"],
```

```
  "codex": ["gpt-4o", "gpt-4o-mini", "o3-mini"]
};

let agents = [
  { name: "ba", role: "Business Analyst", runtime: "claude-code", model: "claude-3-5-sonnet-latest", desc: "Làm rõ l
  { name: "developer", role: "Frontend Developer", runtime: "claude-code", model: "claude-3-7-sonnet-latest", desc:
  { name: "tester", role: "QA Tester", runtime: "codex", model: "gpt-4o", desc: "Kiểm thử lại lỗi theo STR và báo cá
  { name: "leader", role: "Tech Lead (Git Release)", runtime: "claude-code", model: "claude-3-7-sonnet-latest", desc
];

function renderAllAgents() {
  const sidebar = document.getElementById("sidebar-agents-list");
  const grid = document.getElementById("agents-list-grid");
  sidebar.innerHTML = "";
  grid.innerHTML = "";

  agents.forEach(a => {
    sidebar.innerHTML += `
      <div onclick="quickMention('@${a.name} ')" class="flex items-center justify-between px-2.5 py-1.5 rounded-lg h
        <div class="flex items-center gap-2">
          <span class="text-sm">${a.icon}</span>
          <span class="text-xs font-medium text-stone-800">${a.name}</span>
        </div>
        <span class="text-[10px] font-mono text-stone-500">${a.runtime === 'claude-code' ? 'Claude' : 'Codex'}</span
      </div>
    `;

    grid.innerHTML += `
      <div class="p-4 rounded-xl border border-stone-200 bg-stone-50/40 flex flex-col justify-between space-y-3">
        <div>
          <div class="flex items-center justify-between">
            <span class="font-bold text-sm text-stone-900">${a.icon} @${a.name} — ${a.role}</span>
            <span class="text-[11px] font-mono bg-stone-200/70 px-2 py-0.5 rounded text-stone-700">${a.runtime}</spa
          </div>
          <p class="text-xs text-stone-600 mt-1.5 leading-relaxed">${a.desc}</p>
        </div>
        <div class="text-[11px] font-mono text-stone-400 pt-2 border-t border-stone-200/70">Model: ${a.model}</div>
      </div>
    `;
  });

  document.getElementById("agent-count-badge").innerText = agents.length;
  document.getElementById("member-count").innerText = agents.length + 1;
  lucide.createIcons();
}

function switchView(view) {
  ['chat', 'agents', 'workflow'].forEach(v => {
    const el = document.getElementById(`view-${v}`);
    el.classList.add('hidden');
    el.classList.remove('flex');
  });
  const target = document.getElementById(`view-${view}`);
  target.classList.remove('hidden');
  target.classList.add('flex');
}

function confirmAndTriggerLeader() {
  const box = document.getElementById("confirm-box");
  if (box) box.remove();

  const stream = document.getElementById("messages-stream");

  // 1. User confirm message
  stream.innerHTML += `
    <div class="flex items-start gap-3">
      <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
      <div class="flex-1 space-y-1">
        <div class="flex items-baseline gap-2">
          <span class="font-bold text-stone-900 text-xs">Nguyen Manh Hung</span>
          <span class="text-[11px] text-stone-400">Vừa xong</span>
        </div>
        <p class="text-stone-800">
          <span class="agent-pill"><i data-lucide="bot" class="w-3 h-3"></i> leader</span>
```

```
          Mình xác nhận lỗi xuất Excel đã sửa xong rồi nhé, nhờ Leader commit và push lên Git.
        </p>
      </div>
    </div>
  `;

  document.getElementById("footer-agent-status").innerHTML = `<strong>leader:</strong> Đang chạy git commit & push l

  // 2. Leader Commit & Push response
  setTimeout(() => {
    stream.innerHTML += `
      <div class="flex items-start gap-3">
        <div class="w-8 h-8 rounded-full bg-purple-100 border border-purple-300 text-purple-800 flex items-center ju
        <div class="flex-1 space-y-1.5">
          <div class="flex items-baseline gap-2">
            <span class="font-bold text-stone-900 text-xs">leader</span>
            <span class="text-[11px] font-mono text-stone-400">claude-3-7-sonnet</span>
            <span class="text-[11px] text-stone-400">Vừa xong</span>
          </div>
          <p class="text-stone-800">Đã kiểm tra <code class="font-mono text-xs bg-stone-100 px-1.5 py-0.5 rounded">g
          <div class="bg-stone-900 text-stone-100 rounded-xl p-3 font-mono text-xs max-w-xl space-y-0.5">
            <div class="text-stone-400">$ git add src/pages/Reconciliation/ExportButton.tsx</div>
            <div class="text-stone-400">$ git commit -m "fix(reconciliation): async stream excel export"</div>
            <div class="text-emerald-400">[hotfix/reconciliation 8c3d91a] 1 file changed, 8 insertions(+), 2 deletio
            <div class="text-emerald-400">To https://github.com/vetc/hnbg_fe.git -> hotfix/reconciliation</div>
          </div>
          <div class="flex items-center gap-1.5 pt-1">
            <span class="px-2 py-0.5 rounded-full bg-stone-100 border border-stone-200 text-[11px] text-stone-600">
          </div>
        </div>
      </div>
    `;
    document.getElementById("channel-timer").innerText = "Đã xong";
    document.getElementById("footer-agent-status").innerHTML = `<strong>Hoàn tất:</strong> Commit 8c3d91a đã được đẩ
    stream.scrollTop = stream.scrollHeight;
    lucide.createIcons();
  }, 500);
}

function rejectBugFix() {
  const feedback = prompt("Nhập phản hồi chi tiết để @developer sửa lại:");
  if (!feedback) return;
  const stream = document.getElementById("messages-stream");
  stream.innerHTML += `
    <div class="flex items-start gap-3">
      <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
      <div class="flex-1 space-y-1">
        <div class="flex items-baseline gap-2">
          <span class="font-bold text-stone-900 text-xs">Nguyen Manh Hung</span>
          <span class="text-[11px] text-stone-400">Vừa xong</span>
        </div>
        <p class="text-stone-800"><span class="agent-pill">@developer</span> Chưa đạt: ${feedback}</p>
      </div>
    </div>
  `;
  stream.scrollTop = stream.scrollHeight;
}

function quickMention(tag) {
  switchView('chat');
  const input = document.getElementById("chat-input");
  input.value = tag + input.value;
  input.focus();
}

function sendChatMessage() {
  const input = document.getElementById("chat-input");
  const text = input.value.trim();
  if (!text) return;
  if (text.toLowerCase().includes("confirm") || text.toLowerCase().includes("xác nhận")) {
    input.value = "";
    confirmAndTriggerLeader();
    return;
  }
```

```
      const stream = document.getElementById("messages-stream");
      stream.innerHTML += `
        <div class="flex items-start gap-3">
          <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
          <div class="flex-1 space-y-1">
            <div class="flex items-baseline gap-2">
              <span class="font-bold text-stone-900 text-xs">Nguyen Manh Hung</span>
              <span class="text-[11px] text-stone-400">Vừa xong</span>
            </div>
            <p class="text-stone-800">${text}</p>
          </div>
        </div>
      `;
      input.value = "";
      stream.scrollTop = stream.scrollHeight;
    }

    function toggleCreateAgentForm(show) {
      syncModels();
      document.getElementById("create-agent-card").classList.toggle("hidden", !show);
    }

    function syncModels() {
      const rt = document.getElementById("f-runtime").value;
      document.getElementById("f-model").innerHTML = MODELS[rt].map(m => `<option value="${m}">${m}</option>`).join("");
    }

    function addCustomAgent() {
      let name = document.getElementById("f-name").value.trim().replace(/^@/, "");
      const role = document.getElementById("f-role").value.trim();
      const runtime = document.getElementById("f-runtime").value;
      const model = document.getElementById("f-model").value;
      const desc = document.getElementById("f-prompt").value.trim();
      if (!name || !role || !desc) return alert("Vui lòng điền đủ thông tin Agent!");
      agents.push({ name, role, runtime, model, desc, icon: "👹" });
      renderAllAgents();
      toggleCreateAgentForm(false);
    }

    function newTicket() {
      const msg = prompt("Nhập nội dung lỗi mới cần gửi cho @ba:");
      if (!msg) return;
      switchView('chat');
      document.getElementById("messages-stream").innerHTML = `
        <div class="flex items-start gap-3">
          <div class="w-8 h-8 rounded-full bg-stone-800 text-white flex items-center justify-center font-semibold text-x
          <div class="flex-1 space-y-1">
            <div class="flex items-baseline gap-2">
              <span class="font-bold text-stone-900 text-xs">Nguyen Manh Hung</span>
              <span class="text-[11px] text-stone-400">Vừa xong</span>
            </div>
            <p class="text-stone-800"><span class="agent-pill">@ba</span> ${msg}</p>
          </div>
        </div>
        <div class="flex items-start gap-3">
          <div class="w-8 h-8 rounded-full bg-amber-100 border border-amber-300 text-amber-800 flex items-center justify
          <div class="flex-1 space-y-1">
            <div class="flex items-baseline gap-2">
              <span class="font-bold text-stone-900 text-xs">ba</span>
              <span class="text-[11px] text-stone-400">Vừa xong</span>
            </div>
            <p class="text-stone-800">Mình đã nhận thông tin lỗi. Để lập kịch bản tái tạo lỗi (STR) chi tiết cho <span c
          </div>
        </div>
      `;
    }

    window.onload = () => {
      renderAllAgents();
      lucide.createIcons();
    };
  </script>
</body>
</html>
```